Formal verification report — RoycoDayAccountant

53 properties · 92 CVL rules · 1 high-level properties

Contract	`RoycoDayAccountant`
Run timestamp	`2026-08-19T10:33:27.557778+00:00`
Prover runs	Capacity_MathParameter_GovernanceStructural_Invariants

Rule outcomes:

VERIFIED 74 VIOLATED 18

High-level property outcomes:

VIOLATED 1

Coverage warnings:

- FALLBACK GROUPING APPLIED
- 1 component(s) were cut short by the run budget; their properties are excluded from the groupings above (see the budget appendix).

Findings 18

Violated CVL rules surfaced as audit issues — fields follow the standard issue-submission format.

MEDIUM

dustTolerance pad is deducted in collateral-scaled NAV units, so it cannot absorb junior-side drift: operations sized at the reported maximum revert on the post-op coverage gate

rule `dust_pad_dominates_dust_sized_senior_share_mint` in `autospec_Capacity_Math.spec` prover run

`_maxSTDeposit` subtracts the raw `dustTolerance` pad `D` from the *collateral* side of the coverage requirement, where the inequality is scaled by `minCoverageWAD` (`c`), while the drift the pad is meant to absorb — the sync's fee / liquidity-premium senior share mint, which moves NAV from junior to senior — lands on the *junior* side, scaled by `WAD`. Because the configuration invariant forces `c < WAD`, the pad is always worth strictly less than `D` units of junior-side slack (only `D·c/WAD`), so a deposit sized at exactly the advertised maximum can fail the authoritative post-op coverage gate. The Prover exhibited a concrete state where the post-op utilization is `MAX_UINT256`, i.e. a maximal, not marginal, rejection.

Severity rationale

Impact medium × Likelihood medium — The mismatch is structural — it holds for every configuration because `minCoverageWAD < WAD` is an invariant — so any pending fee or liquidity-premium senior mint at the boundary makes the advertised maximum unachievable; no privileged access is needed to submit a max-sized deposit. Severity is bounded because the failure mode is a revert on a gate that fails closed (denial of service / incorrect advertised capacity) rather than a loss of funds, and the most extreme witness needs a small `minCoverageWAD` and a nearly exhausted junior tranche.

Impact

Operations sized at exactly the reported maximum can revert on the authoritative post-op coverage (or liquidity) gate, breaking maximal-fill behaviour for keepers and entry points that trust `inkindMaxDeposit` / `inkindMaxRedeemable` / `lptMaxRedeemableMultiAsset`. Because the discrepancy is deterministic in the state, an actor who can influence the pending fee / liquidity-premium senior mint (or simply times an operation around a sync) can make advertised-maximum operations fail, i.e. grief the gate boundary and cause a transient denial of service for max-sized deposits/redemptions. In the witness the post-op utilization saturates to `MAX_UINT256`, so the rejection is maximal rather than a near-miss. No funds are directly lost or stolen: the gate fails closed and the operation reverts, and callers can retry with a smaller size.

Description

The kernel derives the advertised capacity (`inkindMaxDeposit` and friends) from a PRE-op previewed sync state, while the authoritative coverage/liquidity requirement is re-evaluated on the POST-op state — after the sync has minted senior shares for fees and the liquidity premium and after a fresh venue mark. `dustTolerance` (`D`) exists to make the advertised maximum robust to that drift.

In `_maxSTDeposit`, the coverage leg is computed as

`maxSTDepositGivenCoverage = saturatingSub( floor(J·WAD / c), addNAVUnits(C, D) )`

so `D` is deducted on the collateral/senior side of the coverage inequality. That side of the inequality is scaled by `c = minCoverageWAD`; the junior side is scaled by `WAD`. The guarantee the pad therefore buys is only `(C + m + D)·c ≤ J·WAD`. A fee / liquidity-premium senior share mint of size `f` has `ΔcollateralNAV = 0`, `ΔstEffectiveNAV = +f`, `ΔjtEffectiveNAV = −f`, i.e. it consumes junior-side slack. Surviving it requires `f·WAD ≤ D·c`, i.e. `f ≤ D·c/WAD`, whereas the pad is nominally sized to license drift up to `D`. Since the configuration invariant is `c < WAD`, the pad is *always* worth strictly less than `D` units of the drift it is being asked to absorb, and the shortfall grows without bound as `c` shrinks.

The counterexample makes this concrete and exercises only real implementation code (`Math.saturatingSub`/`trySub`, `RoycoUnitsMath.min`, the NAV-unit wrappers, and a bit-exact `mulDiv` summary — no havoc, no unresolved calls):

- Pre-op state: `collateralNAV C = 196`, `stEffectiveNAV S = 164`, `jtEffectiveNAV J = 32` (NAV conservation `S + J = C` holds exactly), `lptRawNAV P = 3`, `minCoverageWAD c = 1` wei-WAD, `minLiquidityWAD l = 0`, `dustTolerance D ≈ 3.2e19`.
- With `l = 0` the liquidity leg is the `MAX_NAV_UNITS` sentinel, so the coverage leg alone is binding and the reported maximum is `m = saturatingSub(floor(32·WAD/1), 196 + D) = 1696`.

- A senior share mint of `f = 32` is fully within what the pad nominally licenses (`f ≤ D`, since `32 ≪ 3.2e19`) and within available junior NAV (`f ≤ J`, `32 = 32`).
- Post-op the collateral is `C + m = 1892` and the junior tranche is `J − f = 0`.
`UtilizationLogic._computeCoverageUtilization(1892, 1, 0)` takes the `jtEffectiveNAV == 0` branch and returns `MAX_UINT256`, far above `WAD`, so the post-op coverage gate rejects the operation that the kernel itself advertised as feasible.

Here the pad is worth only `D·c/WAD = 3.2e19/1e18 ≈ 32` junior-side units out of a nominal `3.2e19`, and the Prover located exactly that boundary. The `I = 0` setting is not incidental — it isolates the coverage leg and confirms the defect is not an artifact of the two legs interacting. The mirror defect exists on the liquidity leg, where `pDrift·WAD ≤ D·I` is needed but drift up to `D` is licensed. The honest budget the pad actually buys is `jDrift·WAD + cDrift·c ≤ D·c` (respectively `pDrift·WAD + sDrift·I ≤ D·I`), which is provable but is a much smaller allowance than the code's use of `D` suggests.

Attack path

1. The vault reaches a state with a small `minCoverageWAD` (`c = 1` wei-WAD in the witness) and a thin junior tranche relative to the pad: `C = 196`, `S = 164`, `J = 32`, `dustTolerance D ≈ 3.2e19`, `minLiquidityWAD I = 0` so the coverage leg is the only binding constraint.
2. A caller queries the advertised senior-deposit capacity; `_maxSTDeposit` returns `m = saturatingSub(floor(J·WAD/c), C + D) = 1696`.
3. Before/with the operation, the sync mints `f = 32` senior shares for accrued fees / liquidity premium — well within the drift the pad nominally licenses (`f ≤ D`) and within available junior NAV (`f = J`). This shifts NAV from junior to senior: collateral unchanged, `J → 0`.
4. The deposit of exactly `m = 1696` is attempted. The post-op check calls `_computeCoverageUtilization(C + m = 1892, c = 1, J − f = 0)`, which hits the `jtEffectiveNAV == 0` clamp and returns `MAX_UINT256 > WAD`.
5. The coverage gate rejects, so the operation the protocol advertised as exactly fillable reverts.

Assumptions & uncertainties

The finding is grounded in the harness-level model of the drift channel: the senior share mint is modelled as an abstract `f` constrained only by `f ≤ dustTolerance` and `f ≤ jtEffectiveNAV`, not by the concrete fee/liquidity-premium accrual arithmetic, so whether an attacker can steer the real sync to produce exactly the boundary `f` in a given deployment depends on accrual rates and timing. The unit/scale mismatch itself is unconditional and independent of that modelling. The witness uses extreme-but-admitted parameters (`minCoverageWAD = 1` wei, `minLiquidityWAD = 0`, junior NAV equal to the drift); with more typical `c` the gap between the advertised maximum and the true boundary shrinks proportionally to `c/WAD` but never closes. Remediation is a protocol-design choice for the authors — either deduct `ceil(D·WAD/c)` rather than `D` on the coverage leg (and the analogous scaling on the liquidity leg), or size operations off the post-mint previewed state so the drift channel is zero by construction; note that either change would invalidate the exact-closed-form tightness properties proved against the current implementation.

MEDIUM

Dust-tolerance pad in `_maxLPTWithdrawal` is applied on the senior side and scaled down by `minLiquidityWAD`, so the advertised max LPT withdrawal can revert on the post-op liquidity gate

rule `dust_pad_dominates_dust_sized_venue_mark_drift` in `autospec_Capacity_Math.spec` prover run

`_maxLPTWithdrawal` subtracts the dust-tolerance pad `D` inside the senior term `(S + D)` before scaling by `I / WAD`, so the pad only buys `D·I/WAD` units of slack on the venue (LPT) side. The post-op liquidity gate `U_liq = ceil(S·I / P)` is sensitive to venue-mark movement in *raw* NAV units, so any downward mark drift `pDrift` with `D·I/WAD < pDrift <= D` is "dust-sized" by the protocol's own tolerance yet is not covered by the pad — a withdrawal sized at exactly the reported maximum then reverts on the post-op gate. The Prover found a concrete witness in which the drift drives `lptRawNAV` to exactly `0`, where `_computeLiquidityUtilization` saturates to `type(uint256).max`.

Severity rationale

Impact medium × Likelihood medium — The gap is structural, not incidental: since `minLiquidityWAD < WAD` always holds, the uncovered drift band `(D·I/WAD, D]` is non-empty for every admissible configuration, so the failure is reachable with an ordinary max-sized redemption plus a small, entirely normal venue-mark move between the preview sync and the post-op gate — no privileged access is required, but it does need that specific ordering and a mark move inside the band, and the width of the band shrinks as `I` approaches `WAD`. The consequence is a revert / denial of maximal fills and griefable gate boundary rather than a loss of funds, and the caller can recover by sizing below the advertised maximum.

Impact

An operation sized at exactly the reported `lptMaxRedeemableMultiAsset` can revert on the post-op liquidity gate whenever the venue mark moves down by an amount the protocol itself classifies as dust. This breaks maximal fills for keepers and entry points that size against the advertised maximum (wasted gas, failed liquidations/rebalances, stuck automation that does not back off), and gives an adversary a gate-boundary griefing lever: nudging the venue mark by a small, dust-classified amount is enough to make every max-sized withdrawal revert. No funds are directly lost — the shortfall is recoverable by resubmitting a smaller size — but the advertised capacity is unreliable and the failure is unbounded at the `lptRawNAV == 0` edge, where utilization saturates to `type(uint256).max`.

Description

The kernel publishes `lptMaxRedeemableMultiAsset` (via `_maxLPTWithdrawal`) from a PRE-op preview sync state, while the authoritative liquidity requirement is enforced on the POST-op state, after the sync's fee / liquidity-premium senior share mints and after a freshly committed venue mark. The dust tolerance `D` exists to absorb exactly that drift. The rule `dust_pad_dominates_dust_sized_venue_mark_drift` asserts that a mark move no larger than the dust tolerance (`pDrift <= D`) is always absorbed; the Prover refutes it.

The defect is a unit/scale mismatch between where the pad is subtracted and the ratio that scales it. `_maxLPTWithdrawal` computes:

```
requiredLPTValue = mulDiv(addNAVUnits(stEffectiveNAV, dustTolerance), minLiquidityWAD, WAD, Ceil)
z = saturatingSub(lptRawNAV, requiredLPTValue)
```

The pad is added on the *senior* side, inside `(S + D)`, and the whole term is then multiplied by `I / WAD`. A pad of `D` raw NAV

units therefore purchases only about `D · I / WAD` NAV units of headroom on the *venue* side. The post-op gate, by contrast, evaluates `U_liq = ceil(S · I / P)` and is sensitive to venue-side movement in full raw NAV units: a downward mark move of `pDrift` consumes `pDrift` whole units of venue headroom. Surviving `P -> P - pDrift` genuinely requires `pDrift · WAD <= D · I`, whereas the tolerance knob only promises `pDrift <= D`. Because the configuration invariant is `I < WAD`, the inequality `D·I/WAD < D` is strict for *every* admissible `minLiquidityWAD`, so there is always a non-empty band of drifts that the protocol itself classifies as dust yet the published maximum does not tolerate.

Counterexample trace (all arithmetic is the implementation's own — `Math.mulDiv` is the bit-exact `mathint` model, `Math.saturatingSub`, `RoycoUnitsMath` and the `NAV_UNIT` wrappers execute as real code; no havoc, no unresolved calls):

- `collateralNAV C = 2`, `lptRawNAV P = 14`, `stEffectiveNAV S = 8`, `jtEffectiveNAV J = 1`, `minCoverageWAD c = 5`, `minLiquidityWAD I = 12`, `dustTolerance D = 0xca6268669e7254d ≈ 9.13e17`.
- `addNAVUnits(8, D) = 0xca6268669e72555`; `mulDiv(..., 12, 1e18, Ceil)` floors to `10` and the `unsignedRoundsUp(Ceil)` correction yields `requiredLPTValue = 11`.
- `z = saturatingSub(14, 11) = 3` — the published maximum LPT withdrawal.
- A mark drift of `pDrift = 11` satisfies the dust hypothesis `pDrift <= D` trivially (11 vs ~9.13e17), yet `postP = P - z - pDrift = 14 - 3 - 11 = 0`.
- `_computeLiquidityUtilization(S = 8, I = 12, P = 0)` takes its `lptRawNAV == 0` branch and returns `type(uint256).max`, far above `WAD`, so the post-op liquidity gate rejects the operation.

Here `D ≈ 9.13e17` buys only `D·I/WAD ≈ 10.96 ≈ 11` NAV units of venue-side slack, so a drift of just 11 raw units exhausts the entire budget and lands on the saturating `postP == 0` edge — the liquidity-side mirror of the `jtEffectiveNAV == 0 => MAX_UINT256` clamp on the coverage side. The companion rule `gate_admits_max_lpt_withdrawal_within_drift_budget`, which carries the *correct* hypothesis `pDrift · WAD <= D · I`, is VERIFIED, confirming that the true tolerated budget is the rescaled one and that the published figure is advertised against the wrong budget.

Remediation is a protocol-design decision: either scale the pad by the requirement ratio when it is applied on the senior side (i.e. deduct `ceil(D·I/WAD)` extra on the venue side, or equivalently size the pad so that `pDrift·WAD <= D·I` holds for the intended drift), or commit the venue mark before the maximum is published so that no pre/post divergence exists.

Attack path

1. The kernel publishes a maximum LPT withdrawal from a pre-op preview sync: with `S = 8`, `P = 14`, `I = 12`, `D ≈ 9.13e17`, `_maxLPTWithdrawal` returns `z = 3`.
2. A caller (or keeper/entry point) submits a redemption sized at exactly the reported maximum `z = 3`.
3. Between the preview and settlement, the freshly committed venue mark moves down by `pDrift = 11` raw NAV units — comfortably within the configured dust tolerance `D ≈ 9.13e17`, and therefore "dust-sized" by the protocol's own definition.
4. Post-op venue NAV is `P - z - pDrift = 0`.
5. `_computeLiquidityUtilization(8, 12, 0)` hits the `lptRawNAV == 0` branch and returns `type(uint256).max`, which exceeds the `WAD` liquidity limit, so the post-op gate rejects and the transaction reverts despite being sized at the advertised maximum. An adversary able to influence the venue mark by any amount in `(D·I/WAD, D]` can reproduce this on demand.

Assumptions & uncertainties

The counterexample uses deliberately small, degenerate magnitudes (`I = 12` wei-WAD, `P = 14`) that a production configuration would not use; the significance is the general characterization, not the specific numbers — for any `minLiquidityWAD = I < WAD` the uncovered band is `(D·I/WAD, D]`, whose relative width is `1 - I/WAD` (e.g. ~10% of `D` at `I = 0.9e18`). The finding assumes the venue mark can move between the pre-op preview that produces the published maximum and the post-op gate evaluation, which is the premise the dust tolerance itself is designed for. Exploitability as griefing further assumes an actor can influence or time the venue mark move; absent that, the issue manifests as intermittent reverts on maximal fills. The rule was authored as an intentional attack-vector probe and registered as an expected failure, so the violation is the documented refutation of the "pad dominates dust-sized drift" claim rather than a regression.

LOW

Unbounded, absolute-valued `dustTolerance` can zero out `maxSTDeposit`/`maxJTWithdrawal`/`maxLPTWithdrawal` on a fully healthy market, freezing all size-limited flows

rule `healthy_market_reports_positive_capacity_for_any_dust_tolerance` in `autospec_Capacity_Math.spec` prover run

`dustTolerance` is added unconditionally, additively and in absolute NAV units to the requirement side of every capacity formula in `RoycoDayAccountant`, and `setDustTolerance` accepts any `uint256` with no cap. The Prover produced a concrete counterexample in which a market with strictly positive coverage and liquidity slack reports `maxSTDeposit == 0` purely because the pad exceeds that slack, so every consumer that sizes itself off these views (kernel `inkindMaxDeposit`, entry-point ST fills, keepers, the async queue) silently stops accepting flow.

Severity rationale

Impact medium × Likelihood low — Impact is a broad but recoverable denial of service — every capacity-sized deposit and withdrawal path reports zero on a healthy market, temporarily blocking withdrawals, but the state is reversible by resetting the parameter and no funds are directly stolen. Likelihood is low because reaching the state requires the privileged `setDustTolerance` role (held separately from the coverage/liquidity parameter setters), so it needs an admin mistake, a compromised or overly broad role, or a market whose true slack is small relative to a legitimately-chosen absolute pad — the counterexample only needed `D = 12` against 2 and 6 units of slack, which the total absence of setter validation makes easy to hit.

Impact

All flows that size themselves off `maxSTDeposit`, `maxJTWithdrawal` and `maxLPTWithdrawal` — the kernel's `inkindMaxDeposit`, entry-point ST fills, keepers and the async queue — see a capacity of zero and silently halt on a market that is in fact perfectly healthy. That is a protocol-wide denial of service on deposits and on JT/LPT withdrawals: user assets become temporarily unwithdrawable through the size-limited paths until the parameter is corrected, with no error signal distinguishing the padded-out state from a genuine requirement breach. Because the pad is absolute rather than proportional, the same freeze applies at any market size. The mirror-image setting (`dustTolerance = 0`) removes the safety margin and lets operations sized at exactly the reported maximum revert on post-operation rounding.

Description

The rule `healthy_market_reports_positive_capacity_for_any_dust_tolerance` asserts that when a market is strictly healthy — i.e. it has genuine slack against both the minimum coverage and the minimum liquidity requirement, measured without the dust pad — the reported capacity must be strictly positive for any admin-set `dustTolerance`. The Prover refuted it with a fully concrete, unsummarized-arithmetic trace (`Math.mulDiv` evaluated by a bit-exact summary; `Math.saturatingSub`, `RoycoUnitsMath.min`, `addNAVUnits` and the NAV-unit wrappers all executed as real code; no havoc, no NONDET).

Witness state: `collateralNAV C = 4`, `lptRawNAV P = 14`, `stEffectiveNAV S = 8`, `jtEffectiveNAV J = 1`, `minCoverageWAD c ≈ 1.548e17` (≈0.155 WAD), `minLiquidityWAD l ≈ 9.51e17` (≈0.951 WAD), `dustTolerance D = 12`.

Both healthy-market preconditions hold strictly when measured without the pad:
- coverage: `floor(J·WAD/c) = floor(1e18 / 1.548e17) = 6 > C = 4` → two NAV units of genuine coverage slack;
- liquidity: `floor(P·WAD/l) = floor(14e18 / 9.51e17) = 14 > S = 8` → six NAV units of genuine liquidity slack.

The implementation then computes, inside `_maxSTDeposit`:
- coverage leg: `mulDiv(1, 1e18, 1.548e17, Floor) = 6`; `addNAVUnits(C = 4, D = 12) = 16`; `saturatingSub(6, 16) = 0`;
- liquidity leg: `mulDiv(14, 1e18, 9.51e17, Floor) = 14`; `addNAVUnits(S = 8, D = 12) = 20`; `saturatingSub(14, 20) = 0`;
- `min(0, 0) = 0` ⇒ `maxSTDeposit = 0`, violating the `m > 0` assertion.

The root cause is structural and lives in the admin knob, not in the capacity arithmetic. `dustTolerance` enters the capacity formulas as `saturatingSub(floor(J·WAD/c), C + D)` on the coverage leg and `saturatingSub(floor(P·WAD/l), S + D)` on the liquidity leg — additively, unconditionally, and as an **absolute** NAV figure with no proportionality to `collateralNAV` and no upper bound in `setDustTolerance` (which performs no validation at all and sits under a role separate from the coverage/liquidity parameter setters). The pad is therefore a monotone, unbounded reducer of all three capacity views (`maxSTDeposit`, `maxJTWithdrawal`, `maxLPTWithdrawal`): as soon as it reaches the market's true slack, every reported capacity saturates to zero even though the market satisfies its requirements with room to spare. Nothing in the witness is scale-specific — because the pad is absolute, an arbitrarily large and arbitrarily healthy market can be frozen identically by a sufficiently large `dustTolerance`.

`saturatingSub` is itself behaving correctly: it is what prevents an underflow or a wrap to a huge capacity figure. The hazard is that its output is a bare `0`, indistinguishable from "genuinely at the requirement boundary", so downstream consumers cannot tell a padded-out healthy market from a constrained one and simply stop accepting flow with no diagnostic signal.

The same knob is dangerous at the other extreme: setting `dustTolerance = 0` removes the padding that is the sole reason these figures deliberately under-report, re-opening the "operation sized at exactly the reported maximum reverts on post-operation rounding" failure the pad exists to prevent.

Suggested remediation (a protocol-design decision for the authors): validate `dustTolerance` in its setter — cap it as a small absolute figure or as a small fraction of `collateralNAV`, and consider a non-zero lower bound — and/or expose pad-induced saturation as a distinguishable signal rather than a bare zero so consumers can tell "padded out" from "at the requirement boundary".

Attack path

1. The account holding the dust-tolerance role calls `setDustTolerance(12)` (harnessed in the trace as `harnessSetDustTolerance`); no validation is performed, so any `uint256` is accepted.
2. The market is in a strictly healthy state: `collateralNAV = 4`, `lptRawNAV = 14`, `stEffectiveNAV = 8`, `jtEffectiveNAV = 1`, `minCoverageWAD ≈ 0.155e18`, `minLiquidityWAD ≈ 0.951e18` — coverage slack 2 NAV units, liquidity slack 6 NAV units.
3. A consumer queries `maxSTDeposit`. `_maxSTDeposit` computes the coverage leg as `saturatingSub(6, 4 + 12) = 0` and the liquidity leg as `saturatingSub(14, 8 + 12) = 0`, then `min(0, 0) = 0`.
4. The view returns `0`, indistinguishable from a market genuinely at its requirement boundary, and every flow sized off it (kernel `inkindMaxDeposit`, entry-point ST fills, keepers, async queue) declines to proceed. The identical mechanism zeroes `maxJTWithdrawal` and `maxLPTWithdrawal`, which share the additive pad.

Assumptions & uncertainties

The counterexample is fully concrete with no havoc, no unresolved calls and no NONDET, so the arithmetic break is certain. The severity rests on two assumptions drawn from the analysis rather than re-derived here: that downstream consumers (kernel `inkindMaxDeposit`, entry-point ST fills, keepers, the async queue) treat a zero capacity as "no room" and decline rather than reverting or falling back, and that `maxJTWithdrawal`/`maxLPTWithdrawal` use the same additive absolute pad on their requirement side (the analysis states all three formulas do). The rule is registered as an expected failure, so the specification is intentionally strong here; the finding is about the unbounded knob, not about a defect in `saturatingSub`, `mulDiv` or the capacity arithmetic itself. Whether a caller can also reach a harmful state via `dustTolerance = 0` is asserted from the property text and is not exercised by this particular trace.

MEDIUM

Junior deposits are advertised as unbounded but silently require a full one-shot recapitalization once coverage utilization breaches the liquidation threshold (no `minJTDeposit` advisory view)

rule `jt_deposit_of_any_size_admitted_when_coverage_breached` in `autospec_Capacity_Math.spec` prover run

The kernel's `JT_DEPOSIT` post-op gate requires post-deposit coverage utilization to settle strictly *below* the liquidation threshold, but `RoycoDayAccountant` publishes no corresponding advisory figure (it exposes only `maxSTDeposit`, `maxJTWithdrawal`, `maxLPTWithdrawal`). The Prover produced a concrete counterexample where a junior deposit that more than halves coverage utilization is still rejected, proving that whenever coverage has breached the threshold there is an unadvertised *minimum* junior deposit size below which every deposit reverts — exactly during the recovery window when junior capital is most needed.

Severity rationale

Impact medium × Likelihood medium — The counterexample is concrete, uses the implementation's own gate arithmetic with no havoc, and satisfies NAV conservation and all realistic-magnitude bounds, so the break is proven rather than hypothetical; the consequence is denial of service on junior deposits plus stuck escrowed/queued fills rather than direct theft, which places

impact at medium. Reachability requires the market to have already breached the coverage liquidation threshold — a reachable but not everyday state — and the adversarial variant additionally requires shaving coverage in the request-to-execution window, so likelihood is medium.

Impact

During the recovery window after coverage utilization breaches the liquidation threshold — precisely when junior capital is most needed — every junior deposit smaller than the full recapitalization size reverts on the `JT_DEPOSIT` post-op gate, while the accountant advertises no bound at all. Keepers, integrators, and the async queue cannot size a fillable junior deposit, so partial junior fills queued at a healthier state can never complete and remain stuck. Because the required minimum depends on live coverage state, an adversary who shaves coverage between request and execution can push the threshold above an already-escrowed junior deposit, rendering it permanently unexecutable and leaving the escrowed junior capital blocked while the market stays unrecapitalized.

Description

`RoycoDayAccountant` is the advisory capacity surface for the kernel's four post-op utilization gates. Three of those gates are upper-bound gates ("post-op utilization must settle at or below WAD" for `ST_DEPOSIT`/`JT_REDEMPTION` against coverage and for `ST_DEPOSIT`/`LPT_REDEMPTION` against liquidity), and each has a matching published maximum: `maxSTDeposit`, `maxJTWithdrawal`, `maxLPTWithdrawal`. The fourth gate, `JT_DEPOSIT`, has the opposite polarity — post-op coverage utilization must settle *strictly below* `coverageLiquidationUtilizationWAD`, a "restore health in one shot" rule — and the accountant publishes nothing for it. Junior deposits therefore appear to callers, keepers, and the async queue as unbounded and unconditional.

The Prover refuted the property that a junior deposit of any size is admitted once coverage utilization has breached the liquidation threshold. The counterexample runs the real `UtilizationLogic._computeCoverageUtilization` (reached through the `coverageUtilizationRaw` harness wrapper, with `Math.mulDiv` evaluated bit-exactly via `mulDivDownSummary` plus the `SafeCast.toUint(unsignedRoundsUp)` ceiling correction — no havoc, no NONDET), on a state satisfying NAV conservation (`S + J == C`) and all realistic-magnitude hypotheses:

- `collateralNAV C = 0x4fc15904d345078f ≈ 5.746e18`
- `stEffectiveNAV S = C − 8`, `jtEffectiveNAV J = 8` (junior buffer nearly wiped out)
- `minCoverageWAD c = 15`, `coverageLiquidationUtilizationWAD theta ≈ 1.1066e18`

Pre-op utilization is `ceil(C·c/J) ≈ 1.0776e19`, far above `theta`: the market has breached liquidation. A junior deposit of `x = 11` increases both `C` and `J`, yielding `uPost = ceil((C+11)·c/(J+11)) ≈ 4.5364e18` — a reduction of more than a factor of two, i.e. a deposit that unambiguously *helps* — yet `uPost` is still above `theta`, so the `JT_DEPOSIT` gate rejects it and the assertion `uPost < theta` fails.

The arithmetic is not the defect; the missing advisory member is. Because post-op utilization is monotonically nonincreasing in the deposit size (established by the companion rule `jt_deposit_monotonically_restores_coverage`), the gate is a single threshold in deposit size: there exists a well-defined minimum fillable junior deposit, computable from the same packet these views already consume. Nothing surfaces it. This is the inverse of the failure mode the `dustTolerance` padding exists to prevent: instead of an advertised maximum that cannot be consumed, it is an unadvertised minimum below which nothing can be consumed.

Attack path

1. Coverage utilization breaches `coverageLiquidationUtilizationWAD` — in the witness, `jtEffectiveNAV` is reduced to 8 wei against `collateralNAV ≈ 5.746e18` with `minCoverageWAD = 15`, giving pre-op `u = ceil(C·c/J) ≈ 1.0776e19 >> theta ≈ 1.1066e18`.
2. A keeper or user queries the accountant's capacity surface for junior deposits. No bound is published for the `JT_DEPOSIT` gate, so a junior deposit appears unbounded and unconditional.
3. The caller submits a junior deposit of size `x = 11`, which raises both `collateralNAV` and `jtEffectiveNAV` and reduces post-op utilization to `≈ 4.5364e18` — more than a 2x improvement in coverage health.
4. The `JT_DEPOSIT` post-op gate nevertheless requires `uPost < theta`; since `4.5364e18 > 1.1066e18`, the deposit reverts. Every deposit smaller than the full recapitalization size fails identically, with no advertised figure telling the caller what that size is.
5. For an escrowed/async junior request sized against an earlier, healthier state, an adversary who shaves coverage before execution raises the required minimum past the escrowed amount, so the queued fill can never execute.

Assumptions & uncertainties

The finding relies on the accountant being the intended advisory source for kernel gate capacities (it already publishes `maxSTDeposit`, `maxJTWithdrawal`, `maxLPTWithdrawal`) and on the kernel enforcing the `JT_DEPOSIT` gate as `uPost < coverageLiquidationUtilizationWAD`. The witness uses extreme but admissible parameters (`minCoverageWAD = 15` wei-WAD, `jtEffectiveNAV = 8` wei); the structural gap — a gate of opposite polarity with no corresponding advisory view and hence an unadvertised minimum deposit size — is parameter-independent, though the practical size of the shortfall in production configurations is not quantified here. The remediation direction indicated by the analysis is a new advisory view (a `minJTDeposit` / junior recapitalization-shortfall figure derived from `coverageLiquidationUtilizationWAD` and padded by `dustTolerance` in the senior-protective direction) surfaced through the kernel's junior-tranche `inkindMaxDeposit`; the specification itself should not be weakened, as the assertion correctly captures the intended guarantee.

LOW

Permissionless capacity views accept unvalidated SyncedAccountingState: maxJTWithdrawal can report a junior withdrawal exceeding the entire collateral pool

rule `max_jt_withdrawal_bounded_for_unvalidated_caller_state` in `autospec_Capacity_Math.spec` prover run

`maxJTWithdrawal` (and the sibling views `maxSTDeposit` / `maxLPTWithdrawal`) are `external view` functions that accept a caller-supplied in-memory `SyncedAccountingState` and never validate the NAV-conservation invariant `collateralNAV == stEffectiveNAV + jtEffectiveNAV` that every sync-produced packet satisfies. The Prover produced a concrete packet with `collateralNAV = 0` and `jtEffectiveNAV = 25` for which `maxJTWithdrawal` returns `25` — a reported junior withdrawal capacity larger than the entire collateral pool, i.e. an overstated and unfulfillable capacity for any integrator that reads the accountant directly.

Severity rationale	Impact low × Likelihood high — The counterexample requires no privileges, no unusual configuration and no specific chain state — any actor can call the permissionless `external view` with a fabricated struct and immediately obtain a capacity exceeding the collateral pool, so reachability of the property break is high. Impact is limited because the misreport only affects consumers of the view: the kernel's own call paths always pass conservation-enforced packets, so no protocol funds are moved or frozen by the break itself.
Impact	The accountant's public capacity views can return figures that are not backed by any collateral, exceeding the junior claim and potentially the tranche's total supply once converted to shares. Downstream integrators relying on these views get overstated, unfulfillable capacity, leading to reverting user withdrawals or incorrect off-chain/on-chain accounting. No protocol-internal funds are directly at risk, because the kernel only ever supplies sync-produced, conservation-checked packets.
Description	The rule `max_jt_withdrawal_bounded_for_unvalidated_caller_state` asserts that the value returned by `maxJTWithdrawal` is bounded both by the junior effective NAV and by the collateral NAV (`y <= J && y <= C`), *without* assuming NAV conservation on the caller-supplied packet. The Prover refuted the collateral bound with a fully concrete trace (no havoc, no unresolved calls, no NONDET returns). Counterexample packet: `collateralNAV = 0`, `stEffectiveNAV = 4`, `jtEffectiveNAV = 25`, `lptRawNAV = 5`, `minCoverageWAD = 0`, `minLiquidityWAD = 3`, storage `dustTolerance = 0`. Conservation is violated by construction (`4 + 25 = 29 != 0`). Execution inside `_maxJTWithdrawal`: 1. `addNAVUnits(collateralNAV, dustTolerance) = 0`; `RoycoUnitsMath.mulDiv(0, minCoverageWAD = 0, 1e18, Ceil) = 0`, so `requiredJTValue = 0`. Because `minCoverageWAD == 0`, the required-coverage term collapses entirely — there is nothing the junior buffer must cover. 2. `saturatingSub(jtEffectiveNAV = 25, 0) = 25`, so `surplusJTValue = 25`. 3. `mulDiv(25, 1e18, 1e18 - 0, Floor) = 25`, so the function returns `25`. 4. The assertion splits: `25 <= J = 25` holds, but `25 <= C = 0` fails. The arithmetic itself is correct; the closed form is sound *conditional on* the packet describing a real accounting state. The root cause is a missing trust boundary: the three capacity views are permissionless and perform no input validation — neither the full invariant `collateralNAV == stEffectiveNAV + jtEffectiveNAV` nor any weaker relation such as `jtEffectiveNAV <= collateralNAV`. The very notion that makes the answer meaningful — a junior claim backed by collateral — is never established inside the function, so with `J > C` the view reports a junior withdrawal size exceeding the whole collateral pool. A second, sharper variant amplifies the overstatement without bound rather than merely capping it at `J`: with `collateralNAV = 0`, `jtEffectiveNAV = 100`, `minCoverageWAD = 0.5e18`, the `WAD / (WAD - c)` factor yields `y = 200`. The basic failure, however, needs no amplification at all. Inside the protocol the hazard is latent: the kernel only ever passes packets produced by `preOpSyncTrancheAccounting` / `previewPreOpSyncTrancheAccounting`, post-op sync enforces `NAV_CONSERVATION_VIOLATION`, and the `nav_conservation` invariant is proved. The exposure is confined to the public API surface — a third-party integrator (or wrapper) that constructs, caches, or partially populates a packet and converts the returned NAV figure into shares can be handed a `maxRedeem`-style capacity exceeding the tranche's total supply.
Attack path	1. Any caller (no privileges required) invokes `maxJTWithdrawal` with a self-constructed `SyncedAccountingState`: `collateralNAV = 0`, `stEffectiveNAV = 4`, `jtEffectiveNAV = 25`, `minCoverageWAD = 0`. 2. `requiredJTValue` evaluates to `0` because both `collateralNAV + dustTolerance` and `minCoverageWAD` are zero. 3. `surplusJTValue = saturatingSub(25, 0) = 25`, and the coverage-amplification division by `WAD - minCoverageWAD` is a no-op. 4. The view returns `25`, a junior withdrawal capacity strictly greater than the reported collateral pool of `0`. 5. An integrator that trusts this figure (e.g. converting it into a redeemable share amount for an ERC-4626-style `maxRedeem`) publishes or acts on a capacity that cannot be honoured, producing reverting withdrawals or mispriced downstream decisions.
Assumptions & uncertainties	The trace is fully concrete apart from `Math.mulDiv`, which is replaced by a bit-exact `mathint` model whose intermediate values (`0`, `25`) are arithmetically correct; `saturatingSub` executes real code. The finding assumes the three views are part of the externally consumable API. If they are in fact intended only as kernel-internal advisory helpers, the residual risk reduces to a documentation gap. Remediation options: validate `collateralNAV == stEffectiveNAV + jtEffectiveNAV` (or at minimum `jtEffectiveNAV <= collateralNAV`) at the head of each capacity view and revert otherwise, or explicitly document the views as kernel-internal and unsafe to call with self-constructed packets. Note that the corresponding rule with the conservation hypothesis added (`max_jt_withdrawal_bounded_by_junior_claim`) verifies, confirming the arithmetic is correct for well-formed inputs.

MEDIUM

Unvalidated `dustTolerance` overflows checked NAV addition in the capacity views, bricking every capacity quote the market gates on

rule `capacity_views_never_revert` in `autospec_Parameter_Governance.spec` prover run

`dustTolerance` is written verbatim by both `initialize` and `setDustTolerance` with no bound, unlike every sibling economic parameter in `RoycoDayAccountant`. The capacity views then feed it into *checked* NAV additions (`collateralNAV + dustTolerance`, `stEffectiveNAV + dustTolerance`), so a sufficiently large value makes `maxSTDeposit`, `maxJTWithdrawal` and `maxLPTWithdrawal` revert unconditionally on overflow — a protocol-wide denial of service on the quotes the kernel and entry point use for gating and for max-deposit/withdraw reporting.

Severity rationale	Impact high × Likelihood low — Impact is high because the break freezes the entire market's deposit *and* withdrawal paths — users cannot exit — and the only in-protocol remedy (`setDustTolerance`) is itself routed through `withSyncedAccounting` and so may be blocked by the same reverting quotes, making the frozen state hard to unwind. Likelihood is low because reaching the state requires a privileged write to `dustTolerance` via `initialize` or `setDustTolerance`; it is not attacker-reachable, but it is entirely reachable by operator error or a misconfigured deployment since no bound exists on either write path and no sibling parameter's validation covers it.

Impact	Every capacity quote the protocol relies on reverts unconditionally. `maxSTDeposit`, `maxJTWithdrawal` and (via its `stEffectiveNAV + dustTolerance` leg) `maxLPTWithdrawal` are used by the kernel and entry point both for gating and for max-deposit/withdraw reporting, so all deposit and withdrawal flows that consult them fail: users cannot enter and, critically, cannot exit their positions. This is a market-wide denial of service with assets stuck in the market for as long as the parameter stands. Recovery is not straightforward, because the natural fix — calling `setDustTolerance` to lower the value — is wrapped in `withSyncedAccounting`, whose two mandatory syncs can consume the very quotes that are now reverting, so there may be no working path to restore the parameter through normal operation. Off-chain integrators and UIs reading the max-* views also break outright rather than degrading.
Description	The rule `capacity_views_never_revert` asserts that, for any well-formed and realistically sized `SyncedAccountingState`, the three capacity views return rather than revert. The Prover refuted it with a concrete counterexample. **What the counterexample shows.** The accounting packet in the witness is tiny and entirely plausible, and honours every well-formedness constraint the rule imposes (`minCoverageWAD = 1 < WAD`, `minLiquidityWAD = 0 < WAD`, and all four NAV legs far below the sane-NAV ceiling): `collateralNAV = 15`, `lptRawNAV = 4`, `stEffectiveNAV = 3`, `jtEffectiveNAV = 1204`, `marketState = PERPETUAL`. Nothing about the state is saturated or adversarially extreme. The single unconstrained quantity is the accountant's own stored parameter, loaded in the trace from `ext_Royco_storage_RoycoDayAccountantState.dustTolerance` as `2^256 - 15`. Both observed reverts land on the same site, `addNAVUnits(_a = 15, _b = 2^256 - 15)` — i.e. `collateralNAV + dustTolerance`: - `maxSTDeposit` first computes the coverage-bounded quote successfully (`RoycoUnitsMath.mulDiv(1204, WAD, 1)` returns `0x4144db44ef42500000` cleanly through `Math.mulDiv`), then reaches the checked NAV addition and reverts on overflow. - `maxJTWithdrawal` reverts at the identical site, before performing any other work. - `maxLPTWithdrawal` returned normally (`↪ 4`) in this particular witness **only** because the solver happened to pick `minLiquidityWAD = 0`, which short-circuits the view before it reaches its own `stEffectiveNAV + dustTolerance` addition. That leg contains the same unguarded checked addition and is equally exposed; it simply was not exercised by this witness. The issue must therefore not be read as affecting "only two of the three quotes." **Root cause.** `dustTolerance` is the only economic parameter in `RoycoDayAccountant` with no validation on any write path: `setDustTolerance` stores its argument verbatim, and `initialize` stores `_params.dustTolerance` with no `_validateDustTolerance` analogue — while every sibling parameter carries a bound (`< WAD`, `> WAD`, `<= MAX_PROTOCOL_FEE_WAD`, `_validateYieldShareConfig`). Because the value flows into checked (reverting) arithmetic rather than saturating arithmetic inside the *view/quote* paths, one bad parameter converts a soft "quote is unusable" condition into a hard revert on every capacity query. Note also that the overflow threshold is *scale-relative*: the addition overflows once `dustTolerance > type(uint256).max - collateralNAV` (or `- stEffectiveNAV`). "Dust" is inherently relative to market size, so an operator has no natural intuition that any particular absolute magnitude is unsafe, and the small NAV legs in the witness make the safe range effectively the whole uint256 range minus a handful of units. **Recovery is impaired.** The obvious remedy — lowering `dustTolerance` — is itself impaired, because `setDustTolerance` is wrapped in `withSyncedAccounting`, whose two mandatory syncs can consume the very quotes the bad parameter has bricked. There is no path guaranteed to be independent of the capacity views. **Trace quality.** No havoc, unresolved call, or modelling artifact appears anywhere in the trace: the storage load, the operand values (`15` and `2^256 - 15`), and the revert site are all explicit. There is no competing explanation for the violation. **Suggested remediation.** 1. Bound `dustTolerance` at *every* write site — a dedicated error plus a `_validateDustTolerance` internal (mirroring `_validateYieldShareConfig`) called from both `setDustTolerance` and `initialize`. A setter-only fix leaves the deployment path open. Prefer a bound expressed relative to `lastCollateralNAV`/`lastSTEffectiveNAV`, since dust is scale-relative. 2. Make the quote math degrade instead of bricking: replace the checked additions at all **three** view sites with a saturating NAV addition, so a bad parameter degrades a quote rather than reverting every gate. Keep checked arithmetic everywhere accounting correctness depends on it — this is defence-in-depth for view/quote paths only. 3. Guarantee a recovery route that does not depend on the capacity quotes: exempt `setDustTolerance` from `withSyncedAccounting` (the guard is structurally blind to that field anyway), or add a delay-gated emergency setter under a distinct role.
Attack path	1. During deployment `initialize` is called with `_params.dustTolerance` set to an extreme value, or an operator later calls `setDustTolerance` with one — neither path performs any bounds check (the counterexample uses the stored value `2^256 - 15`). 2. The market holds an ordinary, well-formed accounting state; in the witness `collateralNAV = 15`, `stEffectiveNAV = 3`, `jtEffectiveNAV = 1204`, `lptRawNAV = 4`, `minCoverageWAD = 1`, `marketState = PERPETUAL` — nothing extreme. 3. Any call to `maxSTDeposit` computes its coverage-bounded quote successfully, then reaches the checked addition `collateralNAV + dustTolerance` → `addNAVUnits(15, 2^256 - 15)` and reverts on overflow. 4. Any call to `maxJTWithdrawal` reverts at the identical site, before doing any other work. 5. `maxLPTWithdrawal` reverts the same way via its own `stEffectiveNAV + dustTolerance` addition whenever `minLiquidityWAD != 0`; in the witness it returned `4` only because `minLiquidityWAD = 0` short-circuited the view before that addition. 6. Kernel and entry-point gating and max-deposit/withdraw reporting that consume these quotes therefore revert, halting deposits and withdrawals market-wide. 7. Attempting to repair the parameter via `setDustTolerance` runs its `withSyncedAccounting` modifier, whose two mandatory syncs can consume the bricked quotes, so the recovery call itself may revert.
Assumptions & uncertainties	- The trigger requires a privileged write (`initialize` or `setDustTolerance`); this is operator error or a compromised/misconfigured admin, not an unprivileged attack.

- The counterexample directly demonstrates reverts in `maxSTDeposit` and `maxJTWithdrawal` only. The exposure of `maxLPTWithdrawal` is inferred from the same unguarded `stEffectiveNAV + dustTolerance` checked addition being present in that view; in this witness it was skipped because `minLiquidityWAD = 0` short-circuited the view. That leg was not itself exercised by the trace.
- The severity of the recovery problem depends on whether `withSyncedAccounting`'s syncs actually consume the reverting capacity views on the `setDustTolerance` path. The analysis states the syncs *can* consume them; whether every sync invocation necessarily does so was not established by this counterexample, so recovery may be impaired rather than provably impossible.
- The exact overflow threshold depends on the live NAV legs (`collateralNAV`, `stEffectiveNAV`); the trace pins the concrete pair `15` and `2^256 - 15`, and smaller-but-still-huge values of `dustTolerance` would be needed against larger NAVs.
- This rule was registered as an expected failure in the verification harness; the violation is a deliberate demonstration of the missing validation in the implementation as written, not a specification defect.

MEDIUM

setDustTolerance accepts an arbitrary, unbounded NAV value, letting a single governance write erase the junior tranche's recoverable drawdown and break capacity accounting

rule `dust_tolerance_stays_bounded` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.setDustTolerance` writes its argument to storage verbatim with no bound check — the only economic parameter in the contract with no validation on any write path. The prover produced a concrete counterexample in which an authorized caller, on a fully initialized live market, sets `dustTolerance` to ≈ 2^{256} – 3302 and the `withSyncedAccounting` guard approves the transaction because the guard only inspects coverage/liquidity utilizations, which are not functions of `dustTolerance` (identical bit-for-bit before and after the write).

Severity rationale

Impact high × Likelihood low — Impact is high because the forced PERPETUAL transition permanently converts a recoverable junior drawdown into a realized junior loss that cannot be reversed by restoring the parameter, while also halting fee/premium accrual and bricking the capacity views through checked-arithmetic overflow. Likelihood is low because the witnessed path requires the privileged `setDustTolerance` role (the trace shows `_checkCanCall` returning true for a legitimate role-holder), though it is raised somewhat by the fact that `initialize` writes `dustTolerance` with the same total absence of validation, making the bad state reachable by a one-off deployment misconfiguration as well.

Impact

A single unvalidated governance write permanently destroys value belonging to junior-tranche holders: an in-flight, recoverable junior drawdown is converted into a realized junior loss by the forced PERPETUAL transition, and the loss cannot be undone by restoring a sane `dustTolerance`. The same write simultaneously suppresses all premium and protocol-fee accrual (protocol and senior-side revenue loss) and makes the capacity views revert via checked-arithmetic overflow on `collateralNAV + dustTolerance` / `stEffectiveNAV + dustTolerance`, denying service to any integrator or contract that reads them. Because the `withSyncedAccounting` guard is blind to this parameter, none of this is caught or reverted at the time of the write.

Description

`dustTolerance` is the accountant's "treat as economically negligible" threshold, expressed in NAV units. It gates several state-machine and accounting decisions, notably:

- the fourth forcing condition of the market state machine, `jtImpermanentLoss <= dustTolerance`, which decides whether an outstanding junior-tranche impermanent loss is still a *recoverable drawdown* or is collapsed into a realized junior loss by forcing PERPETUAL;
- the premium / protocol-fee booking gates `stGain > dustTolerance` and `jtGain > dustTolerance`;
- the checked additions `collateralNAV + dustTolerance` and `stEffectiveNAV + dustTolerance` inside the capacity views.

Every sibling setter in the contract carries an explicit bound (`_minCoverageWAD < WAD`, `_coverageLiquidationUtilizationWAD > WAD`, `fee <= MAX_PROTOCOL_FEE_WAD`, `_validateYieldShareConfig(...)`). `setDustTolerance` carries none: its body is a bare assignment `_getRoycoDayAccountantStorage().dustTolerance = _dustTolerance;` plus an event. The same is true of the `initialize` path, which writes `$.dustTolerance = _params.dustTolerance` with no validation analogue at all.

The counterexample makes this concrete on a *running* market rather than a deployment-time artifact:

1. The prestate is a live, configured accountant — `kernel != 0`, initializer consumed, and all other configuration bounds honoured (`minCoverageWAD = WAD – 1`, `coverageLiquidationUtilizationWAD > WAD`, fees `≤ MAX_PROTOCOL_FEE_WAD`, `maxJTYieldShareWAD + maxLPTYieldShareWAD ≤ WAD`, distinct non-zero YDMs). The starting `dustTolerance` is a sane ≈ 9.9e44.
2. `AccessManagedUpgradeable._checkCanCall` returns `(true, 0)` for the caller, so this is a legitimate role-holder — not an access-control bypass.
3. `setDustTolerance(0xffff...f31a)` (≈ 2^{256} – 3302) executes and the value lands in storage unmodified: `Store at ... RoycoDayAccountantState.dustTolerance ↪ 0xffff...f31a`.
4. Both bracketing kernel syncs of `withSyncedAccounting` run and both approve. Critically, the guard's inputs are unchanged across the write: `gSyncCovU[0] == gSyncCovU[1] == 1e18`, `gSyncLiqU[0] == gSyncLiqU[1] == 1e18`, `gSyncTheta[0] == gSyncTheta[1]`.

Point 4 is the structural part of the finding. `withSyncedAccounting`'s entire vocabulary is `coverageUtilizationWAD`, `liquidityUtilizationWAD` and `coverageLiquidationUtilizationWAD`. `UtilizationLogic._computeCoverageUtilization` is a function of `(lastCollateralNAV, minCoverageWAD, lastJTEffectiveNAV)` and `_computeLiquidityUtilization` of `(lastSTEffectiveNAV, minLiquidityWAD, lastLPTRawNAV)` — neither depends on `dustTolerance`. The guard is therefore not merely *failing* to notice the change; it has nothing to notice it with, which is exactly what the identical pre/post ghost values in the trace demonstrate. Extending `withSyncedAccounting` is not a viable remediation for this parameter; the bound has to live in the setter.

Once `dustTolerance` is saturated, the downstream effects follow mechanically from the code as written: any outstanding `jtImpermanentLoss` immediately satisfies `jtImpermanentLoss <= dustTolerance`, so the state machine forces PERPETUAL and

	the junior tranche's recoverable drawdown is converted into a realized junior loss; all premium and protocol-fee bookings are simultaneously suppressed because `stGain > dustTolerance` / `jtGain > dustTolerance` can no longer hold for any realistic gain; and the checked `collateralNAV + dustTolerance` / `stEffectiveNAV + dustTolerance` additions in the capacity views revert on overflow, bricking those views. Lowering the tolerance again afterwards does not restore the erased junior claim — the drawdown has already been realized in state.
Attack path	1. A market is live and configured: `kernel` set, initializer consumed, all other config bounds valid, `dustTolerance` at a sane value (≈ 9.9e44 in the trace), and the junior tranche carries a non-zero, still-recoverable `lastJTImpermanentLoss`. 2. The holder of the role permitted to call `setDustTolerance` (per `AccessManagedUpgradeable`; the trace shows `_checkCanCall` returning `(true, 0)`) calls `setDustTolerance(0xffff...f31a)` — a value near $2^{256} - 1$. 3. `withSyncedAccounting` performs the pre-write kernel sync; the guard records `coverageUtilizationWAD = 1e18`, `liquidityUtilizationWAD = 1e18`. 4. The body assigns the argument verbatim to `$.dustTolerance`. No `_validateDustTolerance` exists. 5. The post-write kernel sync records the *identical* utilizations (`gSyncCovU[1] == gSyncCovU[0]`, `gSyncLiqU[1] == gSyncLiqU[0]`), because neither utilization is a function of `dustTolerance`. The guard approves and the transaction succeeds. 6. From this point on, `jtImpermanentLoss <= dustTolerance` is unconditionally true, so the next state-machine evaluation forces PERPETUAL and realizes the junior drawdown as a loss; `stGain > dustTolerance` / `jtGain > dustTolerance` never hold, so premium and protocol-fee bookings stop; and the capacity views revert on the checked `+ dustTolerance` additions. 7. Restoring `dustTolerance` to a sane value does not restore the junior tranche's erased recovery claim.
Assumptions & uncertainties	The write itself requires the caller to hold the `setDustTolerance` permission in the `AccessManager`, so this is a privileged-misconfiguration / compromised-or-mistaken-governance issue rather than something any external actor can trigger; the counterexample explicitly confirms a legitimate role-holder, not an access-control bypass. The magnitude of value destroyed depends on the junior tranche actually carrying an outstanding recoverable `jtImpermanentLoss` at the time — the trace leaves `lastJTImpermanentLoss` unconstrained (`*`), so the erasure consequence is inferred from the code's `jtImpermanentLoss <= dustTolerance` condition rather than witnessed end-to-end by this particular rule. What *is* directly witnessed is the unbounded write surviving the `withSyncedAccounting` guard. The equally unvalidated `initialize` path means the same broken state can also be reached accidentally at deployment, with no privileged action required afterwards. The downstream capacity-view overflow and premium-gate divergence are asserted by sibling rules (`capacity_views_never_revert`, `premium_payment_consumes_accrual_window`) against the code as written; note also that because "dust" is inherently scale-relative, an absolute ceiling alone still leaves any market whose drawdown is smaller than that ceiling exposed to the recovery-claim erasure.

MEDIUM

All ten guarded parameter setters are bricked whenever the kernel sync reverts, freezing accountant governance exactly when a config fix is the remedy

rule `guarded_setter_executable_when_sync_reverts` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.withSyncedAccounting` brackets every guarded configuration setter with two *mandatory* calls to `kernel.syncTrancheAccountingFromAccountant()`. Because that kernel entrypoint is pause-gated, reentrancy-gated and pokes the collateral oracle inside a price frame, any of those conditions failing makes the pre-op sync revert and takes the whole setter with it — before argument validation and before any state change. The prover produced a concrete counterexample in which a properly authorized caller invokes `setMinCoverage` with a legal value and the call reverts purely because the kernel sync is unavailable.

Severity rationale	Impact medium × Likelihood medium — The prover produced a concrete counterexample in which an authorized caller passing a valid argument is unconditionally reverted, proving all ten guarded setters are unreachable whenever the kernel sync fails — a state reachable by a single pauser or by an ordinary oracle staleness/circuit-break event, hence medium likelihood. Impact is medium rather than high because no funds are directly taken, but the lockout is a total governance denial of service that blocks the very repairs (dust tolerance, coverage and liquidity thresholds) needed to exit the degraded state, creating a recovery deadlock with an asymmetric role requirement for unpausing.
Impact	Complete, potentially indefinite denial of service on accountant parameter governance whenever the kernel sync is unavailable (kernel paused, oracle stale / circuit-broken, or an operation frame in flight). All ten guarded setters revert, including the ones that constitute the intended remediation for the degraded state itself — most notably `setDustTolerance`, the repair path out of the dust-tolerance denial of service, and the coverage/liquidity thresholds that govern liquidation. This creates a recovery deadlock in which the protocol cannot be reconfigured out of a bad state, and gives a single pauser leverage to freeze governance while unpausing requires a separate role. No funds are directly stolen, but positions can remain stuck under unfixable configuration while the market degrades.
Description	The `withSyncedAccounting` modifier used by the accountant's guarded setters performs a pre-op and a post-op call into `kernel.syncTrancheAccountingFromAccountant()`. Neither call is best-effort: a revert inside the kernel propagates verbatim out of the setter. The kernel entrypoint can revert for several ordinary operational reasons: the kernel is paused, the collateral oracle reverts on staleness or a circuit-breaker trip while being poked inside the price frame, or the call arrives while an operation frame is in flight and trips the reentrancy gate. In all of those states, every guarded setter is unreachable. The counterexample makes this concrete. Starting from a live, fully configured accountant (`kernel = 0xffff...ffff`, initializer consumed, `config_bounds_always_valid` invariant assumed, `block.timestamp = 2^31 - 1` above both stored clocks), a caller that the authority explicitly authorizes — `cvlCanCallWithDelay` returns `(true, 0)` at the `restricted` gate — calls `setMinCoverage(0xde0b6b3a763ffff)`, i.e. `WAD - 1`, which satisfies the setter's own `require(_minCoverageWAD < WAD)`. The trace then shows the modifier's first mandatory `CALL` to the kernel (`addr = 0xffff...ffff`, `selector = 0xef00b0e7` = `syncTrancheAccountingFromAccountant`) returning `rc = 0x0` under the modeled "kernel sync cannot run" hypothesis, and the revert propagating out of `setMinCoverage`. No store to `minCoverageWAD` occurs anywhere in the trace: the failure happens in the modifier's prologue, so the setter body never runs.

The affected surface is the ten guarded setters: `setSeniorTrancheProtocolFee`, `setJuniorTrancheProtocolFee`, `setJTYieldShareProtocolFee`, `setLPTYieldShareProtocolFee`, `setMinCoverage`, `setLiquidationCoverageUtilization`, `setMinLiquidity`, `setMaxYieldShares`, `setFixedTermDuration` and `setDustTolerance`.

Two things make this more than a cosmetic liveness gap. First, the states that disable the setters are exactly the states in which a configuration change is the intended remediation — loosening coverage/liquidity thresholds, adjusting fees, or repairing `dustTolerance`, which is itself the recovery path out of the dust-tolerance denial-of-service condition. The result is a recovery deadlock: the fix is gated on the very subsystem that is broken. Second, the authors already recognised this hazard elsewhere: the two YDM setters deliberately route their sync through `DispatchLogic._tryExecute`, with a comment noting that "a reverting sync is tolerated since this setter is the only recovery path from a sync-bricking YDM." That tolerance was never extended to the guarded setters, so the protection is inconsistent.

There is also a role asymmetry worth noting: whoever can pause the kernel can hold all accountant parameter governance hostage, while lifting the pause requires a different role — so a single pauser (or a mis-behaving oracle that nobody controls) can keep governance frozen indefinitely.

Remediation should not simply make the guard's syncs best-effort: the pre/post utilization comparison is what makes the guard-semantics properties (`guard_blocks_coverage_and_liquidity_worsening`, `no_config_induced_liquidation`, `utilization_params_only_change_under_full_guard`, `pre_change_accrual_settled_under_old_config`) hold, and those currently verify. Workable shapes are (a) an emergency, delay-gated, unguarded variant of each setter (or a single `emergencySetConfig`) behind a distinct role; (b) a degraded-mode path that, when the pre-op sync fails, skips the utilization comparison but admits only statically, unambiguously safety-increasing new values; or (c) splitting the guard so quote-affecting parameters — `dustTolerance` above all — are always repairable.

Attack path

1. The kernel enters a state where `syncTrancheAccountingFromAccountant()` reverts — the pauser pauses the kernel, or the collateral oracle goes stale / trips a circuit breaker and reverts when poked inside the price frame, or the call lands while an operation frame holds the reentrancy gate.
2. Governance (a caller the authority authorizes; in the trace `cvlCanCallWithDelay` returns `(true, 0)`) attempts a remediating configuration change, e.g. `setMinCoverage(WAD - 1)` — a value that passes the setter's own `require(_minCoverageWAD < WAD)`.
3. The `withSyncedAccounting` prologue issues its mandatory `CALL` to the kernel (`selector = 0xef00b0e7`); the kernel reverts and the revert propagates unchanged.
4. `setMinCoverage` reverts with no write to `minCoverageWAD`. The same holds for the other nine guarded setters, including `setDustTolerance`.
5. Governance cannot repair the configuration until the kernel sync is restored — which, in the pause case, requires a role distinct from the pauser, and in the oracle case is outside the protocol's control.

Assumptions & uncertainties

The counterexample encodes "the kernel sync reverts" via a summary of `syncTrancheAccountingFromAccountant` (`cvlKernelSync` taking the `gSyncReverts` branch); it does not itself prove that the kernel reverts under any particular concrete condition. The real-world reachability therefore rests on the stated properties of that entrypoint — pause gate, reentrancy gate, and an oracle poke inside a price frame — which should be re-confirmed against the kernel source. The control-flow fact the trace establishes (mandatory pre-op sync, revert propagates, setter body never runs, no state written) is independent of that and is not a prover modeling artifact: the revert occurs at the modifier's own call site, and `DispatchLogic._tryExecute` is not on this path. Severity assumes the kernel-degraded states are transient-but-real operational events rather than permanent; if the pauser role is adversarial or the oracle failure is long-lived, impact trends higher.

MEDIUM

`setLiquidationCoverageUtilization` accepts any value > WAD, letting the coverage-liquidation threshold be lowered to a hair-trigger without the accounting guard objecting

rule `liquidation_threshold_cannot_be_lowered` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.setLiquidationCoverageUtilization` validates its argument only with `require(_coverageLiquidationUtilizationWAD > WAD)` — no floor above WAD, no ceiling, no per-call delta cap and no monotonicity requirement. The Prover produced a counterexample in which a live, fully-valid market has its liquidation threshold Θ lowered (by one wei in the witness, but by construction all the way down to `WAD + 1`) while both `withSyncedAccounting` kernel syncs pass, because the guard's Θ clause is satisfied by its second disjunct `postΘ > postCoverageUtilization` on any healthy market.

Severity rationale

Impact high × Likelihood low — The counterexample shows a concrete, guard-passing state change on a live market whose configuration satisfies the proved invariant, so the break is unquestionably reachable; the consequence is direct erosion of junior-tranche principal via the self-sustaining self-liquidation bonus, hence high impact. Likelihood is low because the call requires an address the AccessManager authority grants for this selector (governance/parameter role), not an arbitrary actor, and the loss only materialises on a subsequent drawdown.

Impact

The liquidation threshold that protects the junior tranche can be pre-armed to a hair-trigger while the market is healthy. Once any coverage shortfall — down to a single wei — subsequently occurs, the JT-funded self-liquidation bonus fires and, because each bonus payment further reduces junior effective NAV and raises coverage utilization, liquidation remains armed, allowing senior redeemers to progressively extract the junior loss-absorption buffer. Junior tranche depositors lose principal that the threshold was designed to protect, and the tranche's protective buffer can be exhausted. The same missing bound also permits unbounded upward moves of Θ (effectively disabling liquidation protection for seniors), and `initialize` sets the field with no validation at all.

Description

The rule `liquidation_threshold_cannot_be_lowered` asserts that no accountant method leaves `coverageLiquidationUtilizationWAD` (Θ) below its pre-call value. The Prover refuted it on `setLiquidationCoverageUtilization(uint256)`.

In the counterexample the prestate is a live, initialized accountant (`kernel = 0xff...ff`, `initialized() == true`) whose configuration

satisfies the separately-proved invariant `config_bounds_always_valid`: `minCoverageWAD = minLiquidityWAD = 1e18 – 1`, all four protocol fees at `MAX_PROTOCOL_FEE_WAD`, `maxJTYieldShareWAD + maxLPTYieldShareWAD = WAD`, distinct non-zero YDMs, and `Θ = 0x...fe13`, comfortably above WAD. `initialize` is excluded from the parametric rule, so this is a governance action on a running market, not deployment configuration.

An authorized caller (`0x...f31a`, for which the authority returns `canCallWithDelay ↪ (true, 0)`) invokes `setLiquidationCoverageUtilization(0x...fe12)`. The `withSyncedAccounting` machinery runs in full: two real dispatches of selector `0xef00b0e7` to the kernel are observed (`gSyncCalls` goes `0 → 1 → 2`), the pre-op packet carries `coverageUtilizationWAD = 1e18`, `Θ = 0x...fe13` and the post-op packet carries `coverageUtilizationWAD = 1e18`, `Θ = 0x...fe12`. Every guard clause passes: the coverage clause (`postCov = 1e18 <= WAD`), the liquidity clause, and — critically — the Θ clause, via its second disjunct `postΘ > postCoverageUtilization`. Storage is then written to `0x...fe12` and the assertion `getCoverageLiquidationUtilizationWAD() >= theta0` evaluates to false.

The root cause is the setter's entire validation: `require(_coverageLiquidationUtilizationWAD > WAD, INVALID_COVERAGE_CONFIG())`. Because the only constraint is one-sided and bounded only by WAD, Θ can be lowered arbitrarily — in a single call or by repeated small steps — down to `WAD + 1`. At `Θ = WAD + 1` the JT-funded self-liquidation bonus arms on the first wei of coverage shortfall; since each bonus payment reduces junior effective NAV and therefore *raises* coverage utilization, liquidation stays armed once entered, so senior redeemers keep extracting junior effective NAV as a bonus — a self-sustaining drain of the junior loss-absorption buffer into senior exits.

`withSyncedAccounting` is structurally incapable of preventing this. Its Θ clause is `preΘ <= postΘ || postΘ > postCoverageUtilization`. On a healthy market the second disjunct holds for *any* admissible Θ (Θ must exceed WAD, while a healthy market's coverage utilization does not), so the guard grants unconditional permission. The guard asks an instantaneous question — "is the market in liquidation right now?" — whereas the harm from lowering Θ materialises on a *future* drawdown. This is why the witness can lower Θ with both syncs passing and the utilizations literally unchanged. The same one-sided `require` also means there is no ceiling on raises (the guard's first disjunct permits any increase), and `initialize` writes this field with no validation at all, so a setter-only fix would leave the deployment path open.

This is not a modeling artifact: the prestate satisfies the proved configuration invariant, `initialize` is filtered out, and the kernel-sync model is an over-approximation with unconstrained utilizations, so the guard's checks were shown to hold for arbitrary sync results rather than being modeled as absent. The one-wei magnitude in the witness is simply the minimal decrease; no plausible reading of the existing guard could catch it.

Attack path

1. Market is live and healthy: `initialized() == true`, `kernel != 0`, coverage utilization `= 1e18 <= WAD`, `Θ = 0x...fe13` (> WAD), configuration satisfying `config_bounds_always_valid`.
2. A caller authorized by the AccessManager (witness: `0x...f31a`, `canCallWithDelay ↪ (true, 0)`, immediate execution, zero value) calls `setLiquidationCoverageUtilization(0x...fe12)`.
3. The only validation, `require(newΘ > WAD)`, passes.
4. `withSyncedAccounting` performs both kernel syncs (selector `0xef00b0e7`, two dispatches). Coverage clause passes (`postCov = 1e18 <= WAD`), liquidity clause passes, and the Θ clause passes via `postΘ (0x...fe12) > postCov (1e18)`.
5. Storage is written: `coverageLiquidationUtilizationWAD = 0x...fe12`, strictly below the pre-call value; the rule's assertion fails.
6. Repeating steps 2–5 (or a single large step) walks Θ down to `WAD + 1`, arming the self-liquidation bonus on the first wei of future coverage shortfall; each bonus payment reduces junior effective NAV, raising coverage utilization and keeping liquidation armed.

Assumptions & uncertainties

The write-up relies on the described semantics of Θ (the coverage-utilization level at which the JT-funded self-liquidation bonus arms) and on the feedback effect that bonus payments reduce junior effective NAV and thereby raise coverage utilization; the counterexample itself only proves the parameter can be lowered past any bound above WAD while the guard passes. The economic magnitude of the drain depends on the bonus formula and on how the kernel consumes Θ, which are outside the verified contract (the kernel sync is over-approximated in the model). The caller in the witness is an address the authority grants for this selector; the exact real-world role/timelock configuration for this setter was not part of the verification scope.

MEDIUM

`setLiquidationCoverageUtilization` has no upper bound, letting governance raise the liquidation threshold Θ to an unreachable value and permanently disable the FIXED_TERM→PERPETUAL liquidation escape

rule `liquidation_threshold_raise_is_bounded` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.setLiquidationCoverageUtilization` validates the coverage liquidation threshold Θ (`coverageLiquidationUtilizationWAD`) on one side only (`require(_coverageLiquidationUtilizationWAD > WAD)`), and `initialize` writes the same field with no validation at all. The Prover produced a concrete counterexample in which an authorized caller moves Θ from `2e18` to ≈ $2^{256} - 1930$ in a single transaction on a live market, with the `withSyncedAccounting` guard passing both bracketing syncs. Because `coverageUtilizationWAD >= Θ` is one of the conditions that force the market state machine back to `PERPETUAL`, an unreachable Θ silently deletes that forcing condition and the JT-funded self-liquidation bonus can never arm.

Severity rationale

Impact high × Likelihood low — The counterexample requires an AccessManager-authorized caller (the trace shows `canCallWithDelay` returning `(true, 0)`), so it is privileged-access-only — hence low likelihood — but it needs no unusual state: it fires from a healthy, well-formed live market where every guard clause passes, and the same unbounded write is reachable via the completely unvalidated `initialize` path. The consequence is frozen tranche capital and a disabled liquidation circuit-breaker for the remainder of a fixed term, which is a high impact.

Impact

Senior and junior tranche capital (and LPT holders) can be frozen for the whole remaining fixed-term duration — up to ~194 days in the witness configuration — while the market is deeply undercollateralized, with the self-liquidation mechanism that exists to force senior exits open never arming. This is a denial of exit and a removal of the protocol's core undercollateralization circuit-breaker, exposing depositors to further losses they cannot escape. The deployment path (`initialize`) is equally unvalidated, so a market can be launched in this state from block one.

Description	Θ (`coverageLiquidationUtilizationWAD`) is the coverage-utilization threshold at which a market is considered to be in liquidation. `_previewSyncTrancheAccounting` uses `coverageUtilizationWAD >= coverageLiquidationUtilizationWAD` as one of the conditions that force the market state machine out of `FIXED_TERM` back to `PERPETUAL`, which is the state in which senior/junior deposits and redemptions and LPT redemptions are permitted and in which the JT-funded self-liquidation bonus arms to force senior exits open. The setter constrains Θ only from below: ```solidity require(_coverageLiquidationUtilizationWAD > WAD, INVALID_COVERAGE_CONFIG()); ``` There is no ceiling, no per-call delta cap, and no monotonicity constraint; `initialize` writes the same field with no validation whatsoever, so the deployment path is equally open. The counterexample refutes the bound directly. From a well-formed, live prestate (`kernel != 0`, initializer consumed, all config bounds valid, `lastMarketState = FIXED_TERM`, `fixedTermDurationSeconds = MAX_UINT24`, Θ = `0x1bc16d674ec80000` = `2e18`), an authorized caller (`canCallWithDelay` returns `(true, 0)`, i.e. immediate) calls `setLiquidationCoverageUtilization(0xff...f87e)` ≈ `2²⁵⁶ − 1930`. The value passes the `> WAD` check, is stored, and the post-state read returns it, violating the asserted ceiling of `2e18`. Critically, the `withSyncedAccounting` bracket cannot catch this. The trace shows the pre-op sync (`gSyncCalls 0 → 1`) and post-op sync (`1 → 2`) both succeeding with unchanged utilizations (`gSyncCovU[1] = 1e18 <= WAD`, `gSyncLiqU[1] = 1e18 <= WAD`), so the coverage and liquidity legs of the guard pass, and the guard's Θ clause `preTheta <= postTheta` is satisfied unconditionally by *any* raise regardless of magnitude (`2e18 <= 2²⁵⁶ − 1930`). The guard asks an instantaneous question — "is the market in liquidation right now?" — while the harm from a Θ change only materialises on a *future* loss. On a healthy market the guard has nothing to object to, so the change can be pre-armed silently. The economic consequence: once Θ is out of reach, a market that subsequently takes a junior drawdown never satisfies `coverageUtilization >= Θ` and therefore remains in `FIXED_TERM` while deeply undercollateralized, for the entire remaining fixed-term duration (in the witness, `fixedTermDurationSeconds = MAX_UINT24` ≈ 194 days). In that state the kernel blocks senior and junior deposits and redemptions and LPT redemptions, and the self-liquidation bonus that exists to force senior exits open never arms. Senior and junior capital is trapped in exactly the scenario the liquidation threshold was designed to relieve. This is the same one-sided `require` that also permits Θ to be lowered arbitrarily (the complementary `liquidation_threshold_cannot_be_lowered` counterexample lowers Θ by one wei unopposed); the two are two directions of a single defect. Remediation: introduce explicit `MIN_LIQUIDATION_UTILIZATION_WAD` / `MAX_LIQUIDATION_UTILIZATION_WAD` constants, enforce both bounds in `setLiquidationCoverageUtilization` **and** in `initialize`, add a per-call delta cap and/or a distinct timelock class for Θ so the ceiling cannot be walked to in small steps, and remove or replace the vacuous `preTheta <= postTheta` disjunct in `withSyncedAccounting`. The bound must live in the setter, not the guard, because the guard's test is instantaneous while the harm is deferred.
Attack path	1. Market is live and healthy: initializer consumed, `kernel != 0`, Θ = `2e18`, coverage and liquidity utilizations both at `1e18` (≤ WAD), `lastMarketState = FIXED_TERM`, `fixedTermDurationSeconds = MAX_UINT24`. 2. An AccessManager-authorized caller invokes `setLiquidationCoverageUtilization(≈2²⁵⁶ − 1930)`. `_checkCanCall` grants immediately (`(true, 0)`). 3. The only validation, `require(_coverageLiquidationUtilizationWAD > WAD)`, passes trivially for this enormous value. 4. The `withSyncedAccounting` bracket runs its pre-op sync (`gSyncCalls 0 → 1`) and post-op sync (`1 → 2`); both succeed. Coverage leg (`1e18 <= WAD`) and liquidity leg (`1e18 <= WAD`) pass, and the Θ clause `preTheta <= postTheta` is satisfied because the change is a raise. No revert. 5. `coverageLiquidationUtilizationWAD` is now ≈`2²⁵⁶`, i.e. practically unreachable by any real `coverageUtilizationWAD`. 6. Later, a junior drawdown drives the market deeply undercollateralized. `coverageUtilization >= Θ` can never hold, so the state machine is never forced back to `PERPETUAL`; the market stays in `FIXED_TERM`, the kernel blocks senior/junior deposits and redemptions and LPT redemptions, and the JT-funded self-liquidation bonus never arms — for the remaining fixed-term duration.
Assumptions & uncertainties	The write itself and the absence of any upper bound are proven by the trace; the downstream freeze relies on the documented role of `coverageUtilizationWAD >= coverageLiquidationUtilizationWAD` as one of the conditions in `_previewSyncTrancheAccounting` that force a return to `PERPETUAL`, and on the kernel blocking deposits/redemptions while in `FIXED_TERM` — these were not themselves re-derived from the counterexample. Exploitation requires an authorized (governance / AccessManager-granted) caller, so this is a governance-key or governance-mistake risk rather than a permissionless attack; the trace shows the call being granted with zero delay, but a real deployment may impose a timelock on this role, which would give observers a window to react (though not prevent the change). The prover notes no havoc, no unresolved calls on the causal path, and no modelling artifacts in the witness.

HIGH

Premium is booked out of senior NAV without consuming the accrual window: `premiumsPaid` gate diverges from the premium computation, and the zero-elapsed fallback pays from a fabricated one-second window

rule `premium_payment_consumes_accrual_window` in `autospec_Parameter_Governance.spec` prover run

`preOpSyncTrancheAccounting` can move a nonzero premium out of the senior slice (counterexample: `lptLiquidityPremium = 1`) while leaving `twLPTYieldShareAccruedWAD = 4552` and `lastPremiumPaymentTimestamp` unchanged. The premium legs are computed whenever `stGain != 0`, but the window-consumption block is gated on the separate predicate `stGain > dustTolerance`; additionally, when `elapsedSinceLastPremiumPayment`

== 0` the code discards the real accumulators and pays from a synthetic one-second window at the instantaneous yield-share rate, again without stamping the premium clock. The same accrual window can therefore fund an unbounded number of premium payments.

Severity rationale

Impact medium × Likelihood high — The trace shows a nonzero premium actually moved out of the senior gain with both accumulators and the premium clock untouched, so the same window can fund unlimited further premium payments — real, repeatable mis-accounting that transfers value from senior to JT/LPT, though bounded by realised gains rather than draining principal outright (medium impact). Reachability is high: no privileged access is needed, the path is taken by any kernel-routed operation whose gain falls in the unvalidated dust band, and the zero-elapsed fallback triggers on any second sync within the same block regardless of `dustTolerance`.

Impact

Senior (ST) NAV is systematically over-paid to the junior and LPT tranches. Because the accrual window is never consumed on these paths, the same accrued yield-share entitlement is charged against every subsequent gain, and under the zero-elapsed fallback each same-block sync re-prices the premium at the instantaneous clamped share rate (up to 100% of the gain when `maxLPTYieldShareWAD = 1e18`, as in the witness). The result is incorrect tranche accounting that transfers value out of senior holders — repeatedly and without bound on the number of payments per window — while the premium clock never advances, so the mispricing compounds rather than self-correcting.

Description

The intended invariant is that a sync which books a premium out of the senior slice must, in the same call, zero `twJTYieldShareAccruedWAD` / `twLPTYieldShareAccruedWAD` and set `lastPremiumPaymentTimestamp = block.timestamp`, so that no accrual window funds more than one premium payment. The Prover produced a concrete trace where a premium is booked and none of that happens.

Witness state (all bounds that `initialize` enforces are satisfied): `kernel = 0xff..ff`, both YDMs set and distinct, all four protocol fees `0`, `minCoverageWAD = minLiquidityWAD = 0`, `maxJTYieldShareWAD = 0`, `maxLPTYieldShareWAD = 1e18`, market `PERPETUAL`, no drawdown, `lastCollateralNAV = lastSTEffectiveNAV = lastJTEffectiveNAV = 0`, `dustTolerance = 1`, `twJTYieldShareAccruedWAD = 0`, `twLPTYieldShareAccruedWAD = 4552`, and both clocks stamped at the current block: `lastYieldShareAccrualTimestamp = lastPremiumPaymentTimestamp = block.timestamp = 2147480412`.

The kernel calls `preOpSyncTrancheAccounting(1)`:

1. `_accruePremiumYieldShares` sees `elapsed = 0` and early-returns the stored accumulators `(0, 4552)` without stamping.
2. In `_previewSyncTrancheAccounting`, `collateralNAV (1) > lastCollateralNAV (0)` so `gain = 1`; with `lastCollateralNAV == 0` the whole gain is attributed to senior: `stGain = 1`, `jtGain = 0`.
3. The premium branch is entered because `stGain != ZERO_NAV_UNITS`. The consumption flag is computed independently as `premiumsPaid = greaterThanNAVUnits(stGain = 1, dustTolerance = 1) = false` (visible verbatim in the trace).
4. `elapsedSinceLastPremiumPayments = 0`, so the instantaneous fallback runs: it sets the elapsed window to 1 second and overwrites the accumulator locals with the clamped instantaneous YDM shares — `min(734, 0) = 0` for JT and `min(1e18+1, 1e18) = 1e18` for LPT — discarding the genuinely accrued `4552`.
5. Premiums become `jtRiskPremium = mulDiv(1, 0, 1e18) = 0` and `lptLiquidityPremium = mulDiv(1, 1e18, 1e18) = 1`. The nonzero LPT liquidity premium is carved out of the senior gain and re-added to `stEffectiveNAV` as shares owed to the LPT (returned state: `stEffectiveNAV = 1`, `lptLiquidityPremium = 1`).
6. Back in `preOpSyncTrancheAccounting`, the `if (premiumsPaid)` block — which performs `delete $.twJTYieldShareAccruedWAD`, `delete $.twLPTYieldShareAccruedWAD` and `$.lastPremiumPaymentTimestamp = uint32(block.timestamp)` — is skipped. Post-state confirms `twLPTYieldShareAccruedWAD = 4552` and `lastPremiumPaymentTimestamp = 2147480412` unchanged, while the premium was paid.

Two independent defects are exposed by this single trace.

Defect A — the "premium booked" predicate and the "window consumed" predicate are different expressions. The premium legs are produced whenever `stGain != 0`, whereas `premiumsPaid` is `stGain > dustTolerance`. For the entire band `0 < stGain <= dustTolerance`, the protocol pays a premium out of the senior slice while the accrual window that funded it survives verbatim and is re-applied to the next gain, and the next. `dustTolerance` is an admin parameter written with no validation on any path (`initialize` assigns `$.dustTolerance = _params.dustTolerance` directly, and `setDustTolerance` performs no bound check), so governance controls how wide the over-paying band is.

Defect B — the zero-elapsed fallback pays from a fabricated window and stamps nothing. When `elapsedSinceLastPremiumPayments == 0` (two syncs in the same block, or a sync in the same block as the previous premium payment), the code throws away the genuinely accrued time-weighted accumulators, substitutes the instantaneous clamped yield shares over a synthetic one-second window, and pays a premium from that. In this witness that is what makes the premium nonzero at all: `4552` over the true zero-length window would round to nothing, but `1e18` over a synthetic second yields `1`. Nothing on this path advances `lastPremiumPaymentTimestamp` or clears the accumulators, so the fallback can be re-entered repeatedly within one block, each entry minting a fresh premium at the instantaneous rate. This defect is not gated by the dust tolerance at all — it survives `dustTolerance == 0`, which is why the companion rule `premium_payment_consumes_accrual_window_zero_dust` still verifies and cannot observe it.

Attack path

1. Market is live and initialized; `dustTolerance` is nonzero (witness: `1`) or, for Defect B, any value including `0`.
2. The premium clock is at the current block — either because a premium was just paid in this block or because a prior sync stamped it (witness: `lastPremiumPaymentTimestamp == block.timestamp`).
3. The kernel invokes `preOpSyncTrancheAccounting(collateralNAV)` with a collateral NAV above `lastCollateralNAV`, producing `stGain > 0`. Any user operation that routes through the kernel triggers this sync, so no privileged access is needed.
4. Either the gain lands in the band `0 < stGain <= dustTolerance` (Defect A), or the elapsed premium window is zero (Defect B, e.g. a second operation in the same block).
5. `_previewSyncTrancheAccounting` returns a nonzero `lptLiquidityPremium` (witness: `1`) carved out of the senior gain, while `premiumsPaid == false` leaves `twJTYieldShareAccruedWAD` / `twLPTYieldShareAccruedWAD` and `lastPremiumPaymentTimestamp` untouched.
6. Repeat steps 3–5: the same unconsumed window funds a premium on every subsequent gain, and multiple same-block syncs each mint a fresh premium at the instantaneous rate.

Assumptions & uncertainties	The counterexample uses tiny NAV magnitudes (`collateralNAV = 1`, premium `= 1`) because the Prover minimises witnesses; the per-call loss magnitude at realistic scale depends on `dustTolerance`, on the configured `maxJTYieldShareWAD` / `maxLPTYieldShareWAD`, and on the YDM-reported yield shares (modelled here by a `cvlYieldShare` summary rather than the concrete YDM implementations). The severity of Defect A scales directly with the admin-chosen `dustTolerance`, which is currently unbounded on both the `initialize` and `setDustTolerance` write paths. Defect B does not depend on `dustTolerance` at all and requires only two syncs with a zero elapsed premium window. I have not verified downstream kernel behaviour on the reported `lptLiquidityPremium` beyond the stated intent that it is minted as ST shares.

MEDIUM

MAX_PROTOCOL_FEE_WAD equals WAD, so all four protocol-fee setters accept a 100% fee that appropriates the entire tranche yield

rule `protocol_fees_cannot_reach_full_appropriation` in `autospec_Parameter_Governance.spec` prover run

`MAX_PROTOCOL_FEE_WAD` in `src/libraries/Constants.sol` is defined as `1e18` — exactly `WAD`, the whole of the quantity being shared — and all four protocol-fee setters validate with a non-strict `<=`. The Prover produced concrete counterexamples in which an authorized role holder calls `setSeniorTrancheProtocolFee`, `setJuniorTrancheProtocolFee`, `setJTYieldShareProtocolFee` or `setLPTYieldShareProtocolFee` with exactly `1e18`, the `withSyncedAccounting` guard passes silently, and the fee field is written to 100%. At that rate the entire senior residual gain, junior gain, JT risk premium and LPT liquidity premium are routed to the protocol fee recipient.

Severity rationale	Impact high × Likelihood low — Impact is high because the counterexample shows the entire tranche/LPT yield stream being redirected to the fee recipient in a single accepted transaction, with no floor guaranteed to the capital that earned it. Likelihood is low because the action requires an address authorized by the AccessManager for the fee setters (the trace's `cvlCanCallWithDelay ↪ (true, 0)`), i.e. privileged access, though no other special state or ordering is needed and the code presents no further obstacle.
Impact	A single authorized configuration transaction can set any or all of the four protocol-fee rates to 100%, after which the entire senior residual gain, the entire junior gain, the entire JT risk premium and the entire LPT liquidity premium are minted to the protocol fee recipient. Junior capital continues to absorb first loss and LPT capital continues to provide market-making depth for zero compensation, while senior holders are diluted by their own yield. Because all four rates share the same defect and no delta limit or fee-specific timelock exists, the whole future yield stream of the market can be diverted in one step; the change is invisible to the only guard on the setter path, so it produces no coverage/liquidity signal for monitoring to catch.
Description	The rule `protocol_fees_cannot_reach_full_appropriation` assumes a live, initialized accountant (`kernel != 0`, initializer consumed, `config_bounds_always_valid` holding) whose four protocol-fee fields are each strictly below `WAD`, runs an arbitrary accountant method, and asserts all four are still strictly below `WAD`. It was refuted on four separate instantiations — one per fee setter — and all four violations share a single root cause; they are not distinct problems. In every counterexample the prestate has `stProtocolFeeWAD = jtProtocolFeeWAD = jtYieldShareProtocolFeeWAD = lptYieldShareProtocolFeeWAD = 0xde0b6b3a763ffff` (`1e18 – 1`, one wei below 100%), a plausible live market rather than a saturated-storage artifact. The action is a single call to one fee setter with argument `0xde0b6b3a7640000` = `1e18` = `WAD`, from caller `0x...f35c`, for whom the summarized authority answers `cvlCanCallWithDelay ↪ (true, 0)` — an ordinary role holder exercising an ordinary permission, with `msg.value == 0`. The trace shows `AccessManagedUpgradeable._checkCanCall` passing, the `require(_fee <= MAX_PROTOCOL_FEE_WAD, MAX_PROTOCOL_FEE_EXCEEDED())` check passing on the boundary value, the store executing (e.g. `Store at ...stProtocolFeeWAD ↪ 0xde0b6b3a7640000`), and the final assertion failing on the corresponding conjunct. Two composing defects produce this: 1. **The cap is not a cap.** `MAX_PROTOCOL_FEE_WAD == 1e18 == WAD`, i.e. equal to the whole of the quantity being divided, and all four setters compare non-strictly, so `1e18` is an admissible value. Downstream each fee is applied as `x.mulDiv(feeWAD, WAD, Floor)`, which at `feeWAD == WAD` returns `x` in full: `stGain`, `jtGain`, `jtRiskPremium` and `lptLiquidityPremium` are each fully consumed and minted to the protocol fee recipient. 2. **The only guard on the setter path is structurally blind to it.** `withSyncedAccounting` inspects only `coverageUtilizationWAD`, `liquidityUtilizationWAD` and `coverageLiquidationUtilizationWAD`. No fee field appears in any of these, and the trace confirms both bracketing syncs execute (`gSyncCalls: 0 → 1 → 2`, `gSyncReverts = false`) with the recorded utilizations unchanged across the write. The guard therefore has nothing to object to. This means the guard cannot be "strengthened" for this parameter class; a bound must live in the setter itself. There is also no per-call delta limit: the witness moves `1e18 – 1 → 1e18` because that is the cheapest witness for the solver, but `0 → 1e18` in a single transaction would be equally permitted. Separately, `RoycoDayAccountant.initialize` writes all four fee fields with no validation whatsoever, so the deployment path can install 100% fees directly (the rule filters `initialize` out as vacuous on a live accountant, so this second path is not exercised by the counterexample and is reported from source reading).
Attack path	1. Market is live and healthy: `kernel != 0`, initializer consumed, `minCoverageWAD` and `minLiquidityWAD` below `WAD`, and the four protocol-fee fields at any value (in the witness, `1e18 – 1`). 2. An address holding the fee-setter role (witness: `0x...f35c`, for which the authority returns allowed with zero delay) calls one of `setSeniorTrancheProtocolFee`, `setJuniorTrancheProtocolFee`, `setJTYieldShareProtocolFee` or `setLPTYieldShareProtocolFee` with `1e18`. 3. `restricted` authorization passes; `require(_fee <= MAX_PROTOCOL_FEE_WAD)` passes because `MAX_PROTOCOL_FEE_WAD == 1e18`. 4. `withSyncedAccounting` runs its pre-op kernel sync, the field is written, and the post-op sync runs; both succeed and the compared utilizations are identical, since no fee field feeds `_computeCoverageUtilization` or `_computeLiquidityUtilization`. 5. Poststate: the fee field is `1e18`. On the next accounting cycle, `mulDiv(gain, WAD, WAD, Floor) == gain`, so the entire corresponding gain/premium is minted to the protocol fee recipient. Repeating steps 2–4 for the other three setters appropriates all four streams.

Assumptions & uncertainties

The counterexample relies on the summarized authority model (`cvlCanCallWithDelay`) returning "allowed with no delay" for the caller, i.e. it assumes a legitimately provisioned fee-setter role; it is not an access-control bypass, so realized risk depends on how tightly that role is held and whether it is behind a timelock in deployment. The downstream consequence (`mulDiv(x, WAD, WAD, Floor) == x` consuming the whole gain) is read from the fee-application code rather than exercised by this rule, which only observes the stored fee value. The `initialize` path writing all four fee fields unchecked is likewise a source observation, not part of the counterexample, since the rule excludes `initialize` as vacuous on a live accountant. The reported instance differs per run (senior, junior, JT yield-share, LPT yield-share) purely by solver choice; all four setters use the identical non-strict comparison against the identical constant.

MEDIUM

Solvency requirements minCoverageWAD / minLiquidityWAD can be set to zero in one transaction, and the accounting guard is structurally unable to object

rule `requirements_cannot_be_zeroed` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.setMinCoverage` and `setMinLiquidity` validate only an upper bound (`value < WAD`), so a single authorized call with argument `0` deletes the corresponding solvency requirement outright. The `withSyncedAccounting` guard cannot catch this because the coverage/liquidity utilizations it checks are computed *from* the parameter being zeroed, so zeroing it drives the guard's own metric to a passing value by definition. The prover produced clean counterexamples for both setters, going from the maximum legal setting (`WAD – 1`) to `0` in one transaction with both mandatory syncs succeeding.

Severity rationale

Impact high × Likelihood low — Impact is high: zeroing either requirement makes the coverage gate / maxJTWithdrawal and maxLPTWithdrawal bounds vacuous, exposing senior-tranche principal to withdrawal of the loss-absorption buffer and of all guaranteed secondary liquidity. Likelihood is low because the counterexample requires a caller the authority approves for the setter (`canCallWithDelay` returns `(true, 0)` in both traces), i.e. privileged access — but within that role the action needs no special state, no delay and only one transaction, and the role is shared with routine fee tuning.

Impact

A single authorized configuration call removes a core solvency requirement while every protective check in the accountant reports success. With the coverage requirement at zero, the junior tranche can withdraw the entire loss-absorption buffer, leaving the senior tranche unprotected against subsequent losses; with the liquidity requirement at zero, the LPT can withdraw the full venue mark, draining the secondary liquidity that senior redemptions depend on. In both cases senior-tranche principal is put directly at risk with no on-chain guard or delay standing in the way, and the change is indistinguishable at the guard level from ordinary healthy-market parameter tuning.

Description

The rule `requirements_cannot_be_zeroed` starts from a live, initialized market (`kernel != 0`, non-zero codesize, initializer consumed) whose configuration satisfies the proved `config_bounds_always_valid` invariant and whose two solvency requirements are both non-zero, and asserts that after any non-`initialize` accountant method both `minCoverageWAD` and `minLiquidityWAD` are still non-zero. It is refuted twice.

Counterexample 1 — `setMinCoverage(0)`. Prestate: `minCoverageWAD = 0xde0b6b3a763ffff` (= `WAD – 1`, the largest legal value, not a saturated-storage artifact), `minLiquidityWAD = WAD – 1`, market in `FIXED_TERM`. The caller `0x...f35c` passes `AccessManagedUpgradeable._checkCanCall` — the authority answers `(true, 0)`, i.e. an authorized, non-delayed governance action. The `withSyncedAccounting` bracket executes both syncs (`gSyncCalls` goes `0 → 1` pre-op and `1 → 2` post-op), neither reverts, and both accounting packets are recorded. Between them the trace shows `Store at … minCoverageWAD ↪ '0'`. Poststate: `getMinCoverageWAD() ↪ '0'`; the assertion fails.

Counterexample 2 — `setMinLiquidity(0)`. Identical shape from the same prestate: authorized caller, `Store at … minLiquidityWAD ↪ '0'`, both syncs complete, the trailing sync reports `state.minLiquidityWAD = 0` back to the guard and the guard accepts it. Poststate `getMinLiquidityWAD() ↪ '0'`.

Root cause (shared by both). The setters are bounded only from above:

```solidity
require(_minCoverageWAD < WAD,  INVALID_COVERAGE_CONFIG());
require(_minLiquidityWAD < WAD, INVALID_LIQUIDITY_CONFIG());
```

`0 < WAD` holds trivially, so "delete the solvency requirement entirely" is an ordinary parameter value reachable in one call, from the maximum setting, with no floor, no per-call delta cap, no separate role and no enforced delay. `initialize` admits `0` for both fields with no validation at all, so the deployment path reintroduces the same value even if only the setters were patched.

The second half of the root cause is that the guard is **self-certifying** for this class of change. `withSyncedAccounting` inspects only `coverageUtilizationWAD`, `liquidityUtilizationWAD` and `coverageLiquidationUtilizationWAD`, but `UtilizationLogic._computeCoverageUtilization(lastCollateralNAV, minCoverageWAD, lastJTEffectiveNAV)` and `_computeLiquidityUtilization(lastSTEffectiveNAV, minLiquidityWAD, lastLPTRawNAV)` are functions of the very parameters being zeroed. Driving a requirement to zero drives the corresponding utilization to a passing value *by definition*, so the "post ≤ WAD or post ≤ pre" test is satisfied for a reason that has nothing to do with the market being healthy. The guard cannot distinguish "healthy" from "unconstrained", which is visible in the traces: it sees an acceptable post-op packet while the requirement it is meant to protect has just been abolished. No strengthening of the guard predicate can fix this — any guard-level defence would have to inspect the written value and its delta, not the derived utilizations. The bound must live in the setter.

Downstream effect. With `minCoverageWAD == 0` the coverage gate and the `maxJTWithdrawal` bound become vacuous, so the junior tranche can redeem the entire loss-absorption buffer that stands between senior and loss. With `minLiquidityWAD == 0`, `maxLPTWithdrawal` returns the full venue mark, so the LPT can drain all the secondary liquidity guaranteed to senior.

Recommended remediation. (1) Add `MIN_COVERAGE_FLOOR_WAD` / `MIN_LIQUIDITY_FLOOR_WAD` to `Constants.sol` and enforce `floor <= value < WAD` in `setMinCoverage`, `setMinLiquidity` **and** `initialize`. (2) Add a per-call delta cap so even a legitimate loosening cannot travel `WAD – 1 → 0` in one transaction, as these counterexamples do. (3) If removing a requirement must remain possible, route it through a separate, more privileged, mandatorily delayed entry point with its own role, rather than a parameter tweak sharing a role with fee changes. (4) Do not attempt a guard-level fix, and document in the `withSyncedAccounting` comment that the guard is blind to parameters that define the utilizations it measures, so future setters do not inherit a false sense of protection.

Attack path

1. Market is live and healthy: `kernel != 0`, initializer consumed, `minCoverageWAD = minLiquidityWAD = WAD – 1` (maximum legal setting), state `FIXED_TERM`, config satisfying `config_bounds_always_valid`.
2. An address authorized for the setter's role (trace: `0x...f35c`; `AccessManagedUpgradeable._checkCanCall` → authority returns `(true, 0)`, immediate, no delay) calls `setMinCoverage(0)` — or, in the second instantiation, `setMinLiquidity(0)`.
3. The `withSyncedAccounting` pre-op sync runs and succeeds (`gSyncCalls: 0 → 1`), recording the pre-op utilization packet.
4. `require(value < WAD)` passes trivially for `0`; the body writes `minCoverageWAD = 0` (resp. `minLiquidityWAD = 0`) — visible as `Store at ...minCoverageWAD ↪ '0'`.
5. The post-op sync runs and succeeds (`gSyncCalls: 1 → 2`); the recomputed utilization is a passing value precisely because the parameter that defines it is now zero, so the guard raises no objection and the transaction completes.
6. The requirement is now off: `getMinCoverageWAD() ↪ '0'` (resp. `getMinLiquidityWAD() ↪ '0'`). Junior can then redeem against a vacuous coverage gate / `maxJTWithdrawal` bound, or the LPT can withdraw the full venue mark via `maxLPTWithdrawal`.

Assumptions & uncertainties

The counterexample assumes the caller holds the role the `AccessManager` authority grants for these `restricted` setters; the rule models the authority with a CVL summary (`cvlCanCallWithDelay`) that returns `(true, 0)`, so it does not prove which concrete address holds the role or whether the deployment configures a timelock delay for it. If the role is in practice held only by a timelocked, multi-sig-controlled governance contract, the action becomes observable and delayed, which reduces likelihood but does not remove the missing-floor defect (and does not help the `initialize` path, which is unvalidated). The downstream consequences (`maxJTWithdrawal` / `maxLPTWithdrawal` becoming vacuous) are taken from the property statement and the prover's source analysis rather than from a separate proved rule; the counterexample itself establishes only that both requirements can be written to zero under a fully executed guard. Some prestate values in the trace are extremal symbolic values (`Θ = MAX_UINT256`, timestamps at type maxima); these are not needed for the attack — the two requirement fields are at the plausible maximum legal value `WAD – 1`, and the write to zero is independent of the rest of the state.

HIGH

Unbounded `dustTolerance` lets a sync permanently erase a live junior-tranche recovery claim with no economic justification (FIXED_TERM → PERPETUAL write-off)

rule `sync_cannot_erase_live_junior_recovery_claim` in `autospec_Parameter_Governance.spec` prover run

`_previewSyncTrancheAccounting` treats `jtImpermanentLoss <= dustTolerance` as one of the conditions that forces the market from FIXED_TERM to PERPETUAL, and the PERPETUAL commit unconditionally zeroes `lastJTImpermanentLoss`. Because `dustTolerance` is an unbounded, unvalidated absolute NAV parameter (written verbatim by `setDustTolerance`, never validated in `initialize`), an oversized tolerance alone is sufficient to convert the junior tranche's entire outstanding recoverable drawdown into a permanently realized junior loss — even when the sync sees exactly zero PnL. The prover produced a concrete witness where 1713 units of junior recovery claim are deleted with no collateral gain whatsoever.

Severity rationale

Impact high × Likelihood medium — Impact is high because a live junior recovery claim is destroyed irreversibly with no repaying gain, transferring value away from junior depositors, and the fixed term is terminated early — the prover's trace shows the deletion happening with `collateralNAV == lastCollateralNAV`, i.e. zero economic justification. Likelihood is medium rather than high because installing the oversized tolerance requires the governance setter, but that setter is completely unvalidated, the guard is structurally blind to it, and the trigger afterwards is an ordinary sync; a market whose drawdown happens to be small can also fall into this state without any malicious intent.

Impact

The junior tranche's recoverable drawdown is permanently and irreversibly written off without ever having been repaid. In the counterexample, 1713 NAV units of junior recovery claim are deleted in a sync with exactly zero PnL, converting a recoverable position into a realized junior loss that no subsequent parameter change can restore. Simultaneously the write-off suppresses premium and protocol-fee bookings via the `stGain > D` / `jtGain > D` dust gates, and it silently terminates the fixed term (`fixedTermEndTimestamp = 0`) before it has elapsed, changing the economics junior depositors subscribed to. The loss is invisible to the `withSyncedAccounting` guard, so there is no on-chain circuit breaker.

Description

The market state machine in `_previewSyncTrancheAccounting` evaluates seven conditions that force a transition out of FIXED_TERM into PERPETUAL. One of them is `lessThanOrEqualToNAVUnits(jtImpermanentLoss, dustTolerance)` — i.e. "the outstanding junior drawdown is small enough to be treated as dust". The PERPETUAL commit branch that follows performs two semantically distinct actions on a single unconditional path: it stores `lastMarketState = PERPETUAL` and `fixedTermEndTimestamp = 0` (suspending the fixed-term observation period), *and* it stores `lastJTImpermanentLoss = 0` (destroying the ledger entry recording the junior tranche's recoverable drawdown).

Fusing these two actions means the write-off happens regardless of whether the drawdown was actually repaid. Nothing in the branch requires that a collateral gain occurred, or that the term expired; "within the dust tolerance" is treated as economically equivalent to "repaid".

The second half of the defect is that the comparison threshold is fully under governance control and is scale-free: `setDustTolerance` writes its argument verbatim with no ceiling, and `initialize` has no `_validateDustTolerance` analogue at all (unlike the existing `_validateYieldShareConfig` pattern). Because the parameter is an absolute NAV quantity rather than a fraction of market size, *any* outstanding drawdown smaller than whatever tolerance is currently installed is silently reclassified as dust on the next sync.

The counterexample isolates this mechanism unambiguously. The prestate is a live, initialized FIXED_TERM market with `lastMarketState = FIXED_TERM`, `lastJTImpermanentLoss = 1713`, `lastSTEffectiveNAV = 3238`, `lastJTEffectiveNAV = 892`,

`lastCollateralNAV = 4130` (NAV conservation holds exactly: 3238 + 892 = 4130), `fixedTermEndTimestamp = 2147483156` against `block.timestamp = 2147483155` (the term has **not** elapsed), non-zero `fixedTermDurationSeconds`, and `minCoverageWAD = 0` so coverage utilization computes to `0`, far below the liquidation threshold.

The kernel then calls `preOpSyncTrancheAccounting(4130)` with `collateralNAV == lastCollateralNAV`. Both `lessThanNAVUnits(4130, 4130)` and `greaterThanNAVUnits(4130, 4130)` return `false`, so the entire PnL waterfall is skipped — there is literally zero profit or loss in this sync, and nothing appreciated that could legitimately repay the drawdown. Of the seven forcing conditions, six evaluate false (duration non-zero; `equalsNAVUnits(3238, 0) == false`; `equalsNAVUnits(892, 0) == false`; term not elapsed; coverage utilization `0 < MAX_UINT256`; grace period over). Exactly one evaluates true: `lessThanOrEqualToNAVUnits(1713, 1713) == true`, with the admin-set tolerance sitting precisely at the outstanding drawdown. The PERPETUAL branch then writes `lastMarketState = 0`, `fixedTermEndTimestamp = 0`, and `lastJTImpermanentLoss = 0`. The rule's assertion `getLastJTImpermanentLoss() != 0` reads back `0` and fails.

The conversion is irreversible. Lowering `dustTolerance` afterwards cannot restore the deleted ledger entry — the junior tranche's recoverable drawdown has become a realized junior loss. Moreover the `withSyncedAccounting` guard on the setter path cannot detect this: it inspects only coverage utilization, liquidity utilization and the liquidation threshold, and `dustTolerance` appears in none of them. In this witness coverage utilization was `0` both before and after, so the guard is structurally blind to the value destruction.

This sits inside the same cross-cutting parameter-governance weakness as the missing fee cap, the one-sided Θ bound and the absent coverage/liquidity floors: the only gate on the setter path reads derived utilizations, so it cannot see a parameter that is absent from both. Real bounds have to live in the setters and in `initialize`.

Recommended remediation (two independent changes, both needed):
1. Bound `dustTolerance` at every write site — add a ceiling constant and a `_validateDustTolerance` internal mirroring `_validateYieldShareConfig`, called from both `setDustTolerance` and `initialize`. Note this is **necessary but not sufficient**: any market whose outstanding drawdown happens to fall below the new ceiling remains exposed, because the comparison is against an absolute parameter. A market-relative bound (a small fraction of `lastCollateralNAV` or `lastSTEffectiveNAV`), or a one-way ratchet forbidding the tolerance from ever being raised above the currently outstanding `lastJTImpermanentLoss`, closes the residual gap.
2. Separate the two actions the PERPETUAL commit fuses. Transitioning out of FIXED_TERM and destroying the junior recovery claim must be independently gated, and zeroing `lastJTImpermanentLoss` must require a genuine economic justification — a non-zero collateral gain that actually repaid the drawdown, or term expiry.

Regression obligation: `nav_conservation` and `fixed_term_lifecycle_coupling` both hold in this witness's prestate and poststate and must continue to hold after any change to the state machine; `sync_cannot_erase_live_junior_recovery_claim` flipping to VERIFIED is the acceptance test.

Attack path

1. A market is live and initialized in FIXED_TERM with an outstanding junior drawdown, e.g. `lastJTImpermanentLoss = 1713`, `lastSTEffectiveNAV = 3238`, `lastJTEffectiveNAV = 892`, `lastCollateralNAV = 4130`, `fixedTermEndTimestamp = 2147483156`, non-zero `fixedTermDurationSeconds`, `minCoverageWAD = 0`.
2. The privileged actor calls `setDustTolerance(1713)` (or any value >= the outstanding drawdown). The value is stored verbatim; no ceiling is checked, and `withSyncedAccounting` sees only coverage/liquidity utilizations, which are unchanged (`0`), so nothing reverts.
3. Any kernel-routed operation triggers `preOpSyncTrancheAccounting(4130)` at `block.timestamp = 2147483155`, i.e. before `fixedTermEndTimestamp`, with `collateralNAV == lastCollateralNAV` so the PnL waterfall is entirely skipped.
4. In the state machine, six of the seven PERPETUAL-forcing conditions are false; only `lessThanOrEqualToNAVUnits(1713, 1713)` is true.
5. The PERPETUAL branch commits `lastMarketState = PERPETUAL`, `fixedTermEndTimestamp = 0`, and `lastJTImpermanentLoss = 0`.
6. The junior recovery claim is gone. Lowering `dustTolerance` back down does not restore it.

Assumptions & uncertainties

The write path for `dustTolerance` is privileged (`setDustTolerance`, and `initialize`), so reaching the state requires a governance action — malicious, compromised, or merely careless. The finding does not depend on the tolerance being set adversarially large: the counterexample uses a tolerance exactly equal to the outstanding drawdown, so an honestly chosen "dust" figure that happens to exceed a market's current drawdown is enough. The precise downstream accounting consequence for junior redemption values depends on how `lastJTImpermanentLoss` is consumed elsewhere in the protocol, which is outside the verified contract; the demonstrated break is the unjustified, irreversible deletion of the ledger entry itself. The claim that premium and protocol-fee bookings are suppressed via the `stGain > D` / `jtGain > D` gates is taken from the specification of the attack vector rather than exercised directly by this trace, which had zero PnL.

LOW

`preOpSyncTrancheAccounting` returns hard-coded zero `lptRawNAV`/`liquidityUtilizationWAD`, making the guard's senior-liquidity-floor check vacuous

rule `sync_packet_reports_true_liquidity_state` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant._previewSyncTrancheAccounting` marshals the returned `SyncedAccountingState` with literal zero placeholders for `lptRawNAV` and `liquidityUtilizationWAD`, discarding the non-zero `$.lastLPTRawNAV` it already holds. The Certora counterexample shows the accountant returning `lptRawNAV = 0` / `liquidityUtilizationWAD = 0` while storage still reads `lastLPTRawNAV = 2^256 – 1`. Because the packet carries no indicator of whether the liquidity leg has been patched, any consumer that reads it — notably `withSyncedAccounting`'s `postOp.liquidityUtilizationWAD <= WAD` clause — is trivially satisfied and cannot fail closed.

Severity rationale

Impact medium × Likelihood low — The accountant-side defect is unconditional — the returned values are literal constants, reproducible on any call from the kernel in a well-formed live market, as the counterexample shows. However, realizing the guard

bypass additionally requires the deployed kernel to fail to patch the liquidity leg (or to patch it with a mark taken before the fee/premium share mints), which this rule does not prove; the kernel is summarized under the verification model, so the end-to-end exploit path is unverified. Hence a genuine break in a safety-critical invariant, but reachable only if the unenforced kernel convention is violated.

Impact

The senior-liquidity-floor guard clause can be satisfied vacuously. If a packet carrying the placeholder liquidity leg reaches `withSyncedAccounting`, an operation that raises senior claims — for example a `minLiquidity` increase — can settle the market below the senior liquidity floor while the guard reports approval. That undermines the protocol's core solvency invariant for the senior tranche, and downstream consumers reading `lptRawNAV` from this packet see zero instead of the true committed venue mark, corrupting any accounting derived from it.

Description

The rule `sync_packet_reports_true_liquidity_state` asserts two things about the packet the accountant hands back from `preOpSyncTrancheAccounting`: (1) the packet's `lptRawNAV` equals the mark the accountant actually holds in storage (`$.lastLPTRawNAV`), and (2) a reported `liquidityUtilizationWAD == 0` is genuine — i.e. only possible when `minLiquidityWAD == 0` or `stEffectiveNAV == 0`, rather than a fabricated constant.

The prover refuted assertion (1) with a concrete counterexample from a well-formed, initialized, live market: `kernel = 0xff...ff`, `lastMarketState = PERPETUAL`, `lastCollateralNAV = 1312 = lastSTEffectiveNAV + lastJTEffectiveNAV` (conservation holds), `lastJTImpermanentLoss = 0`, `fixedTermEndTimestamp = 0`, and crucially `lastLPTRawNAV = 2^256 – 1` — a large, non-zero committed venue mark. The kernel calls `preOpSyncTrancheAccounting(0)`. The trace walks the full body: the accrual clocks initialize, the PnL waterfall runs the loss branch down to `stEffectiveNAV = 0`, the state machine commits `PERPETUAL`, and the four NAV checkpoints are stored. The packet is then returned as `{ ..., lptRawNAV = 0, ..., liquidityUtilizationWAD = 0, ... }` while storage still reads `lastLPTRawNAV = 2^256 – 1`. The assertion `to_mathint(state.lptRawNAV) == getLastLPTRawNAV()` evaluates `0 == 2^256 – 1` and fails.

The root cause is not arithmetic, not `dustTolerance` (left symbolic and unconstrained by the solver), and not a modelling artifact: at the tail of `RoycoDayAccountant._previewSyncTrancheAccounting` the struct is marshalled with two hard-coded constants — `lptRawNAV: ZERO_NAV_UNITS` and `liquidityUtilizationWAD: 0` — accompanied by a source comment stating that the kernel is expected to refresh them after committing the fresh mark.

Three consequences follow, all visible in the trace:

1. **The packet is fabricated, not merely stale.** The accountant already holds a usable value (`$.lastLPTRawNAV = 2^256 – 1`) and deliberately reports zero instead — arbitrarily far from the truth.

2. **There is no discriminator.** A placeholder packet is structurally indistinguishable from one the kernel has patched. No downstream consumer, the guard included, can determine whether the liquidity leg is meaningful, so no consumer can fail closed on an unpatched packet.

3. **The guard's liquidity clause fails open.** `withSyncedAccounting` tests `postOp.liquidityUtilizationWAD <= WAD`. A literal `0` satisfies that unconditionally. If the packet reaching the guard is the unpatched one — or carries a mark taken before the sync's fee/premium share mints — then a `minLiquidity` increase, or any change that raises senior claims, can settle the market below the senior liquidity floor with the guard silently approving.

The safety of the whole arrangement therefore rests on an out-of-band convention (kernel marks the venue → calls `commitLiquidityProviderTrancheRawNAV` → patches the packet before returning it to the guard) that nothing in the code enforces and nothing in the packet records.

Suggested remediation, in increasing order of strength:

- In `_previewSyncTrancheAccounting`, populate `lptRawNAV` from `$.lastLPTRawNAV` and derive `liquidityUtilizationWAD` via `UtilizationLogic._computeLiquidityUtilization(stEffectiveNAV, minLiquidityWAD, $.lastLPTRawNAV)`. The packet then becomes at worst *stale* rather than *fabricated*, and a reported zero utilization is always attributable to real state.
- Add an explicit discriminator field to `SyncedAccountingState` (e.g. `bool liquidityLegCommitted`) set only by `commitLiquidityProviderTrancheRawNAV`, and make `withSyncedAccounting` revert when handed a packet whose liquidity leg was never committed. This converts the out-of-band convention into a checked obligation. Note this is an ABI change to the struct and must be coordinated with `RoycoDayBalancerV3Kernel` and `RoycoDayEntryPoint`.

Attack path

1. Prestate: an initialized, live market with `lastMarketState = PERPETUAL`, NAV conservation holding (`lastCollateralNAV = 1312 = lastSTEffectiveNAV + lastJTEffectiveNAV`), and a large non-zero committed venue mark `lastLPTRawNAV = 2^256 – 1`.
2. The kernel (`msg.sender == $.kernel`) calls `preOpSyncTrancheAccounting(0)`.
3. The sync body runs to completion: accrual clocks initialize, the PnL waterfall takes the loss branch down to `stEffectiveNAV = 0`, `PERPETUAL` is committed, and the NAV checkpoints are stored.
4. The accountant marshals and returns the packet with the literal placeholders `lptRawNAV = 0` and `liquidityUtilizationWAD = 0`, while storage still reads `lastLPTRawNAV = 2^256 – 1`. Nothing in the packet marks the liquidity leg as uncommitted.
5. If that packet reaches `withSyncedAccounting` without being patched (or with a mark taken before the sync's fee/premium share mints), the clause `postOp.liquidityUtilizationWAD <= WAD` is trivially true, and an operation raising senior claims — such as a `minLiquidity` increase — settles below the senior liquidity floor with the guard approving.

Assumptions & uncertainties

The counterexample proves only the accountant-side half: that `preOpSyncTrancheAccounting` emits a fabricated liquidity leg, and that an unpatched packet renders the guard's liquidity clause vacuous. It does **not** prove that the deployed `RoycoDayBalancerV3Kernel` fails to patch the packet — the kernel is summarized under the verification model, and this remains a named, still-unverified assumption. Closing the property end to end requires a rule on `RoycoDayBalancerV3Kernel.syncTrancheAccountingFromAccountant` asserting that the packet returned to the guard carries the freshly committed `lastLPTRawNAV` with a liquidity utilization computed *after* the fee and premium share mints. The sibling guard properties are proved for arbitrary packets (the sound direction), but the guard's real-world behaviour likewise depends on the kernel honouring this unenforced convention.

LOW

YDM swap installs an incoming Yield Distribution Module without initializing it, resurrecting stale per-market curve state when initialization calldata is empty

rule `ydm_swap_always_reinitializes_incoming_ydm` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant._initializeYDM` makes YDM initialization optional and caller-controlled: it only dispatches the initialization call when `_ydmInitializationData.length != 0`, yet `setJuniorTrancheYDM` / `setLiquidityProviderTrancheYDM` overwrite the YDM pointer unconditionally. The Prover produced a counterexample in which `setJuniorTrancheYDM(0x1965, "")` succeeds, stores `$.jtYDM = 0x1965`, and issues no call whatsoever to the incoming module — so the newly installed YDM is used with whatever per-market state (curve parameters and last-adaptation timestamp) it already held for this accountant, or with no state at all.

Severity rationale

Impact medium × Likelihood low — The counterexample requires only that the caller pass the `restricted` access-manager gate (`cvlCanCallWithDelay ↪ true, 0`), i.e. the YDM setters are privileged governance functions, so the path is reachable only by a configured governance caller — hence low likelihood. The resulting loss is bounded by the yield-share clamp and the length of the mispriced window, or is a recoverable freeze of accrual until another swap, so the impact is a conditional/recoverable value misallocation and temporary denial of service rather than an outright drain.

Impact

Two consequences follow from the same missing-initialization defect. (1) Mispriced yield share: a YDM re-installed with empty calldata retains a stale last-adaptation timestamp, so the first post-swap accrual collapses the whole intervening interval into one adaptation step and pushes the JT risk / LPT liquidity yield share to its clamp, transferring value between the Senior Tranche and the Junior / LP Tranches for that window at a moment of the swapper's choosing. (2) Bricked accrual: installing an instance with no per-market state for this accountant is accepted at swap time and fails only at the next sync, freezing accrual and the deposit/redemption flows that depend on it until another swap is executed.

Description

The rule `ydm_swap_always_reinitializes_incoming_ydm` asserts that any successful YDM swap must result in at least one call to the incoming YDM address (the ghost `gCallsToWatched` is incremented by a `CALL` hook on the watched target, pinned to `newYDM` and required to be distinct from the kernel so the best-effort sync call cannot be miscounted).

The counterexample refutes it. Starting from a live, initialized accountant (`kernel = 0x...fe82`, `jtYDM = 0x...fe13`, `lptYDM = 0x...f07c`), the caller `0x2710` invokes `setJuniorTrancheYDM(_jtYDM = 0x1965, _jtYDMInitializationData = bytes (length = 0))`. The `restricted` access check passes (`cvlCanCallWithDelay ↪ true, 0`), the sibling-YDM check passes (`0x1965 != 0x...f07c`), and the best-effort kernel sync returns failure (`cvlTryExecuteSync ↪ false`) which the accountant deliberately discards. Then `RoycoDayAccountant._initializeYDM(_ydm = 0x1965, _ydmInitializationData = bytes)` opens and closes as a single frame with **no sub-call inside it at all**, after which `$.jtYDM` is stored as `0x1965`. The final assertion fails with `gCallsToWatched ↪ '0'`.

The root cause is the body of `_initializeYDM`:

```solidity
require(_ydm != address(0), NULL_ADDRESS());
if (_ydmInitializationData.length != 0) _ydm._dispatch(DispatchMode.EXECUTE, _ydmInitializationData);
```

Initialization is conditional on caller-supplied calldata, while the pointer write in the setter is unconditional. Because YDM per-market state is held inside the YDM instance keyed by the calling accountant's address, and that state survives being swapped away, a swap performed with empty calldata installs a module that carries whatever state it accumulated during an earlier stint under this same accountant. Rotating back to a previously used adaptive YDM with empty data therefore resurrects stale curve parameters and a stale last-adaptation timestamp: the first accrual after the swap applies the entire intervening gap as a single adaptation step, snapping the JT risk / LPT liquidity yield share toward its clamp and mispricing the premium for that window — a value transfer between the Senior Tranche and the Junior / LP Tranches whose timing is chosen by whoever performs the swap.

The complementary case is equally reachable from the same defect: installing an instance that has never been initialized for this accountant also succeeds (the pointer is written, nothing is validated), and the failure surfaces only at the next sync, when accrual — and the deposits/redemptions that depend on it — revert. In the trace the sync leg already reports failure and is discarded, so the swap completes with a target that has never been touched or probed in any way.

Note that the swap performs no validation of the target beyond `!= address(0)` and `!= sibling YDM`: no interface probe, no allowlist, no post-swap smoke test. The counterexample's target `0x1965` is an arbitrary address that is never called.

Attack path

1. The accountant is live and configured: `kernel = 0x...fe82`, `jtYDM = 0x...fe13`, `lptYDM = 0x...f07c`, initializer already consumed.
2. A YDM instance (`0x1965` in the trace) that was previously installed and initialized under this same accountant is no longer the active `jtYDM`; its per-market state, keyed by the accountant address, still lives inside it, including a now-stale last-adaptation timestamp.
3. A caller permitted by the access manager (`0x2710`, `cvlCanCallWithDelay ↪ true, 0`) calls `setJuniorTrancheYDM(0x1965, "")` — empty initialization calldata.
4. The zero-address and sibling-YDM checks pass; the best-effort kernel sync reverts and is discarded (`cvlTryExecuteSync ↪ false`).
5. `_initializeYDM(0x1965, "")` skips the dispatch entirely because `_ydmInitializationData.length == 0` — the trace shows the frame with no sub-call — and `$.jtYDM = 0x1965` is stored anyway (`gCallsToWatched ↪ 0`).
6. On the next accrual, the resurrected YDM adapts using the stale timestamp, applying the whole intervening gap in one step and clamping the JT/LPT yield share, mispricing the premium for that window.

Assumptions & uncertainties

The finding relies on the documented design fact that YDM per-market state is stored in the YDM instance keyed by the calling accountant's address and is not cleared when the module is swapped away; the counterexample itself proves only that no call is made to the incoming module, since the YDM is modelled and not concretely deployed in this run. The magnitude of the resulting

mispricing depends on the specific YDM implementation's adaptation curve and clamp, and on the length of the interval between the module's last adaptation and the first accrual after re-installation. The setters are `restricted`, so a governance caller (or a compromised/misconfigured authority) is required. Recommended remediation: reject empty `_ydmInitializationData`, or better, have the accountant construct the initialization calldata itself from a fixed accountant-controlled selector (which also removes the arbitrary-calldata primitive executed under the accountant's privileged identity); additionally reject `$.kernel`, `address(this)` and the tranche addresses as swap targets, probe the target's interface (e.g. a validated `staticcall` to `previewYieldShare` and/or a governance allowlist), and smoke-test the newly installed module so a bricked or uninitialized YDM reverts at swap time rather than at the next sync.

MEDIUM

YDM swap discards the best-effort settlement result, leaving an un-accrued window that the incoming yield-distribution model retroactively re-prices

rule `ydm_swap_leaves_no_unaccrued_window` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.setJuniorTrancheYDM` (and identically `setLiquidityProviderTrancheYDM`) settles the outgoing model's outstanding accrual through `_tryExecute` and then throws the returned `(bool success, bytes result)` away. The Prover produced a concrete trace in which the sync reports failure, the YDM pointer is nevertheless replaced, and `lastYieldShareAccrualTimestamp` is left behind the current block — so the window that actually elapsed under the outgoing model is later priced by the incoming model, transferring value between the senior tranche and the JT/LPT premiums at a moment governance chooses.

Severity rationale

Impact high × Likelihood low — The counterexample shows a completed swap leaving a real, non-zero accrual window open, which the next accrual prices with the wrong model — a directional, caller-timed transfer of senior NAV into the JT/LPT premiums, and unbounded in the `clock == 0` deployment variant, hence high impact. Likelihood is low because the swap is gated by the `restricted` governance role, even though the sync-failure precondition (reverting outgoing YDM, paused kernel, stale oracle) is precisely the situation the best-effort call was written to survive.

Impact

An accrual window that ran under one yield-distribution model is retroactively re-priced at another model's yield share. The resulting JT risk premium and LPT liquidity premium are paid out of senior effective NAV at a rate that never applied during the period, so senior-tranche value is transferred to (or withheld from) the junior and liquidity-provider tranches. The direction and magnitude are chosen by whoever performs the swap, since they select both the timing (window length) and the incoming model (the yield share applied). In the deployment-time variant, where the accrual clock is left at `0` by `initialize`, the window can span the entire market lifetime, making the mis-priced premium arbitrarily large.

Description

The rule `ydm_swap_leaves_no_unaccrued_window` asserts the natural post-condition of a model swap: after the swap, `lastYieldShareAccrualTimestamp == block.timestamp`, i.e. no accrual window attributable to the outgoing model is left open. The Prover refuted it with a clean, fully attributable counterexample.

Prestate in the counterexample: a live, initialized accountant (`kernel != 0`), `lastYieldShareAccrualTimestamp = 2147480345`, `block.timestamp = 2147480346` — a genuine one-second window is outstanding, and both accumulators (`twJTYieldShareAccruedWAD`, `twLPTYieldShareAccruedWAD`) are non-zero. The rule's preconditions (`getAccrualClock() != 0` and `getAccrualClock() < block.timestamp`) explicitly exclude the never-accrued `clock == 0` corner, so this is the ordinary mid-life stale-clock case.

Execution: the caller passes the `restricted` check (`canCallWithDelay` returns `(true, 0)`), the setter reaches

```solidity
$.kernel._tryExecute(abi.encodeCall(IRoycoDayKernel.syncTrancheAccountingFromAccountant, ()));
_initializeYDM(_jtYDM, _jtYDMInitializationData);
$.jtYDM = _jtYDM;
```

and `_tryExecute` returns `('false', empty)` — the modeled failure of the settlement sync (outgoing YDM reverting inside `yieldShare`, a paused kernel, a stale oracle, or an in-flight operation frame; `_tryExecute` is documented as reporting the outcome so that "the caller decides what a failure means"). The trace then shows control passing straight into `_initializeYDM` with no branch, no revert and no compensating write; the accountant makes no use of the `false` at all. `Store at ...jtYDM ↪ '0x266'` replaces the model pointer, and the poststate still reads `lastYieldShareAccrualTimestamp = 2147480345` with the accumulators untouched. The assertion therefore fails: the accrual clock is one second behind the current block after a completed swap.

The consequence is a mis-attribution of accrual. The next `_accruePremiumYieldShares` computes `elapsed = block.timestamp - $.lastYieldShareAccrualTimestamp` and multiplies the **incoming** model's instantaneous `yieldShare` by a window that actually elapsed under the **outgoing** model, booking the result into the JT risk premium and the LPT liquidity premium out of senior effective NAV. Because the two YDM setters carry no `withSyncedAccounting` bracket, nothing else observes or repairs the open window, and the party performing the swap chooses both the moment (hence the window length) and the incoming model (hence the yield share applied to it) — the mis-pricing is directional and timeable rather than random.

A related instance of the same defect exists at deployment: `initialize` never stamps `lastYieldShareAccrualTimestamp` or `lastPremiumPaymentTimestamp`, so the accrual clock sits at `0` until the first accrual. In that state the entire market lifetime is a single un-accrued window available to be priced by whatever model is installed first. The rule's added precondition deliberately moved past this corner so the finding would rest on the mid-life case, but the corner is real.

Note the genuine design tension: the swap must remain executable when the sync is bricked — that is the stated purpose of `_tryExecute` here and is separately verified by `ydm_swap_succeeds_despite_reverting_sync`. A correct fix therefore cannot simply make the settlement mandatory; it must bind the returned `success` flag and, on failure, close the window before

overwriting the pointer (e.g. stamping `$.lastYieldShareAccrualTimestamp = uint32(block.timestamp)`, forfeiting the unsettled premium rather than mis-attributing it), surface the flag in the update events so a forced recovery swap is distinguishable from a clean one, and stamp both clocks in `initialize`.

Attack path

1. A market is live with an outstanding accrual window: `lastYieldShareAccrualTimestamp` is behind `block.timestamp` (in the counterexample, by one second, with non-zero `twJT`/`twLPT` accumulators).
2. The settlement sync is in a failing state — the outgoing YDM reverts in `yieldShare`, the kernel is paused, the oracle is stale, or an operation frame is in flight (modeled by `gSyncReverts == true`, with `_tryExecute` returning `('false', empty)`).
3. An actor holding the YDM-setter role calls `setJuniorTrancheYDM(newYDM, "")` — the `restricted` check passes (`canCallWithDelay ↪ (true, 0)`).
4. `_tryExecute` reports failure; the return value is discarded. `_initializeYDM` runs (with empty calldata the dispatch is skipped entirely) and `$.jtYDM` is overwritten with the new model.
5. Poststate: `jtYDM = 0x266` but `lastYieldShareAccrualTimestamp` is unchanged at `2147480345` while `block.timestamp` is `2147480346` — an open window now attributed to the new model.
6. The next `_accruePremiumYieldShares` multiplies the new model's `yieldShare` by that elapsed window and books the premium out of senior effective NAV.

Assumptions & uncertainties

The counterexample models the sync failure via the `cvlTryExecuteSync` summary and the `gSyncReverts` ghost; the property break is in the accountant's handling of the reported failure, not in the summary itself (no revert explanation appears in the trace and the swap completes normally). The size of the resulting value transfer depends on the length of the open window, the difference between the outgoing and incoming yield shares, and the `maxJTYieldShareWAD` / `maxLPTYieldShareWAD` caps — the counterexample proves the mis-attribution occurs but not a specific loss figure. Reaching the setter requires the governance role gated by `restricted`; the sync-failure precondition, by contrast, is exactly the state the best-effort call exists to tolerate. The related `initialize` observation (both clocks left at `0`) is drawn from the prover's analysis of an earlier run rather than from this trace, whose preconditions exclude that corner.

MEDIUM

YDM setters accept arbitrary targets (kernel, accountant, tranches) and make initialization optional, allowing a bricked market and an authenticated arbitrary-call gadget

rule `ydm_swap_target_must_be_a_genuine_ydm` in `autospec_Parameter_Governance.spec` prover run

`RoycoDayAccountant.setJuniorTrancheYDM` / `setLiquidityProviderTrancheYDM` validate the incoming yield-distribution-model address only against `address(0)` and the sibling YDM. The Prover produced a non-reverting trace in which an authorized caller installs the market's own `kernel` address as `jtYDM`, with empty initialization calldata so that `_initializeYDM`'s dispatch is skipped entirely — no code check, no initialization, no smoke test.

Severity rationale

Impact high × Likelihood low — The counterexample is a clean, non-reverting execution that leaves the market's kernel installed as the junior tranche YDM, which both bricks all accrual/sync (and hence deposits and redemptions) and, via the caller-supplied `_dispatch`, yields arbitrary authenticated calls against the sole asset custodian — hence high impact. Likelihood is low because the setters sit behind the AccessManaged authority gate, so the actor must hold the governing role (or that role must be compromised or make a mistaken/malicious-input call).

Impact

Two impacts follow from the same validation gap. (1) Denial of service: installing an address with no per-market YDM state makes the next `yieldShare` call revert, freezing accrual, sync, deposits and redemptions for the whole market until another swap is performed. (2) Unauthorized privileged control: the unconstrained low-level initialization dispatch is an arbitrary-calldata call made under the accountant's identity, and the kernel — the market's sole asset custodian — authorizes on `msg.sender == accountant`, so the setter can be used as an authenticated call gadget against custody of user funds.

Description

The rule `ydm_swap_target_must_be_a_genuine_ydm` asserts that after a successful, authorized YDM swap the installed address is neither the market's kernel nor the accountant itself. The counterexample refutes the first assertion with a complete, non-reverting execution:

1. `_msgSender()` is `0x131d`; `AccessManagedUpgradeable._checkCanCall` returns authorized with zero delay, so the caller passes the access-managed gate.
2. `$.lptYDM` loads as `0xff...fe13` while the incoming `_jtYDM` is a different address, so the sole input check in the setter — `require(_jtYDM != $.lptYDM, YDMS_CANNOT_BE_IDENTICAL())` — passes.
3. `$.kernel` loads as the *same* address as the incoming `_jtYDM`. Nothing in the setter compares the argument against `$.kernel`, `address(this)`, or the tranche addresses.
4. The best-effort sync succeeds (`gSyncCalls: 0 → 1`).
5. `_initializeYDM(_ydm, _ydmInitializationData)` is entered with `_ydmInitializationData.length == 0`. Its only check is `require(_ydm != address(0))`, and the `if (_ydmInitializationData.length != 0)` guard is false, so the `_dispatch` is skipped and the function returns immediately with **no sub-call at all** — the incoming instance is never exercised, not even for code existence.
6. `$.jtYDM` is stored as the kernel's address and the call returns successfully. The assertion `newYDM != getKernel()` then evaluates to `false`.

Two mutually reinforcing gaps are visible directly in the source:

(a) The target is essentially unvalidated. There is no ERC-165 probe, no trial `previewYieldShare`/`yieldShare` call, no governance allowlist of blessed implementations, and no exclusion of the protocol's own privileged addresses (the kernel, the accountant itself, the senior/junior/LP tranches). Any non-zero address distinct from the sibling YDM is accepted.

(b) Initialization is optional and caller-controlled. Because `_initializeYDM` dispatches only when the supplied bytes are non-empty, the caller can guarantee the swap never fails on a bogus target simply by passing empty calldata — the one code path that would have exercised the new instance is skipped by the caller's own choice of argument.

The consequences are twofold. First, an address with no per-market YDM state is installed: the next

	`_accruePremiumYieldShares` reverts inside `IYDM($.jtYDM).yieldShare(...)` (e.g. `FixedYDM`'s `UNINITIALIZED_YDM`), which bricks every accrual and every sync and therefore every deposit and redemption in the market until another swap is executed. Second, and more severe, when initialization calldata *is* supplied, `_ydm._dispatch(DispatchMode.EXECUTE, _ydmInitializationData)` is an arbitrary-calldata call issued under the accountant's own identity — and the kernel grants trust on `msg.sender == accountant`. Pointing that primitive at the kernel, exactly the address the witness installs, converts a governance parameter setter into a general-purpose authenticated call gadget against the market's sole asset custodian. Note also that these setters carry no `withSyncedAccounting` bracket, so no accounting guard observes the change.
Attack path	1. An actor authorized to call `setJuniorTrancheYDM` (per the trace, `_checkCanCall` returns authorized with zero delay) invokes it. 2. The actor passes `_jtYDM = $.kernel` (any address other than `$.lptYDM` and non-zero is accepted) — the `YDMS_CANNOT_BE_IDENTICAL` check passes and no kernel/self/tranche exclusion exists. 3. The actor passes empty `_jtYDMInitializationData`, so `_initializeYDM` skips the `_dispatch` entirely: no initialization, no code check, no smoke test, and no way for the swap to fail on a bogus target. 4. `$.jtYDM` is written to the kernel's address and the call returns successfully. The market's yield distribution model is now an address with no per-market YDM state; the next accrual/sync reverts, freezing deposits and redemptions. 5. Variant for the privileged-call half: supply non-empty `_jtYDMInitializationData` instead, and `_ydm._dispatch(DispatchMode.EXECUTE, data)` executes attacker-chosen calldata against the kernel with `msg.sender == accountant`.
Assumptions & uncertainties	The trace relies on a summarized authority (`cvlCanCallWithDelay` returning true), so the caller must hold the role gating the YDM setters — i.e. the swap path is privileged/governance-reachable rather than open to anyone; this is why likelihood is not rated high. The counterexample directly demonstrates only the "kernel installed as jtYDM with empty init data" state; the accountant-as-YDM assertion was never reached because the first assertion failed first, but the validation logic is identical, so that case is equally admissible. The described DoS chain (revert inside `yieldShare`, e.g. `FixedYDM`'s `UNINITIALIZED_YDM`) and the kernel's `msg.sender == accountant` trust model come from the surrounding code as described in the analysis rather than from the trace itself. `setLiquidityProviderTrancheYDM` mirrors the same logic and is expected to be equally affected.

general General VIOLATED EDITED SOURCE

All properties.

RULE	STATUS	DESCRIPTION	
max_st_deposit_coverage_safe:187	VERIFIED	If m = maxSTDeposit(state) is neither zero nor the MAX_NAV_UNITS 'unbounded' sentinel and state.minCoverageWAD != 0, then (state.collateralNAV + m + dustTolerance) * minCoverageWAD <= state.jtEffectiveNAV * WAD. Consequently an ST deposit of exactly m NAV (which raises both collateralNAV and stEffectiveNAV by m) settles with coverage utilization <= WAD, i.e. the reported capacity can always be consumed in full without tripping the kernel's post-op coverage gate.	pr
max_st_deposit_liquidity_safe:213	VERIFIED	If m = maxSTDeposit(state) is neither zero nor the MAX_NAV_UNITS sentinel and state.minLiquidityWAD != 0, then (state.stEffectiveNAV + m + dustTolerance) * minLiquidityWAD <= state.lptRawNAV * WAD, so an ST deposit of exactly m settles with liquidity utilization <= WAD. maxSTDeposit must be bounded by BOTH the coverage and the liquidity headroom, never just the coverage leg.	pr
max_jt_withdrawal_coverage_safe:237	VERIFIED	If y = maxJTWithdrawal(state) != 0 then (state.jtEffectiveNAV - y) * WAD >= (state.collateralNAV + dustTolerance - y) * minCoverageWAD: withdrawing exactly y from the junior tranche (which reduces both jtEffectiveNAV and collateralNAV by y) leaves the coverage requirement satisfied with the dust pad still in hand.	pr
max_jt_withdrawal_bounded_by_junior_claim:260	VERIFIED	• For any NAV-conserving input state (collateralNAV == stEffectiveNAV + jtEffectiveNAV), maxJTWithdrawal(state) <= state.jtEffectiveNAV and <= state.collateralNAV. The junior tranche must never be told it can withdraw more NAV than its own claim on the collateral (the closed form divides by (WAD - minCoverageWAD), which can amplify the surplus above the junior claim if the bound is mis-derived). • All three functions are permissionless and accept an arbitrary in-memory SyncedAccountingState with no check that it originated from a real sync or that it is NAV-conserving (collateralNAV == stEffectiveNAV + jtEffectiveNAV) or that lptRawNAV corresponds to the actual venue mark. With a non-conserving struct (e.g. jtEffectiveNAV > collateralNAV) maxJTWithdrawal can exceed the junior claim, and downstream conversion of that NAV into shares can yield a maxRedeem exceeding the	pr

RULE	STATUS	DESCRIPTION	
		tranche's total supply — an overstated, unfulfillable capacity for any integrator that reads the accountant directly.	
`max_lpt_withdrawal_liquidity_safe:282`	VERIFIED	maxLPTWithdrawal(state) <= state.lptRawNAV always; and if state.minLiquidityWAD != 0 and z = maxLPTWithdrawal(state) != 0 then (state.lptRawNAV - z) * WAD >= (state.stEffectiveNAV + dustTolerance) * minLiquidityWAD, i.e. removing exactly z of venue depth leaves the senior liquidity floor satisfied with the dust pad intact.	pr
`zero_min_coverage_capacity_degeneracy:304`	VERIFIED	When a requirement is switched off the corresponding capacity bound must vanish exactly: minCoverageWAD == 0 implies the coverage leg of maxSTDeposit is unbounded (MAX_NAV_UNITS) and maxJTWithdrawal == jtEffectiveNAV; minLiquidityWAD == 0 implies the liquidity leg of maxSTDeposit is unbounded and maxLPTWithdrawal == lptRawNAV exactly (no dust deduction). This is the capacity-math half of the 'zero-min-liquidity market reduces to a plain senior/junior market' reduction.	pr
`zero_min_liquidity_capacity_degeneracy:325`	VERIFIED	When a requirement is switched off the corresponding capacity bound must vanish exactly: minCoverageWAD == 0 implies the coverage leg of maxSTDeposit is unbounded (MAX_NAV_UNITS) and maxJTWithdrawal == jtEffectiveNAV; minLiquidityWAD == 0 implies the liquidity leg of maxSTDeposit is unbounded and maxLPTWithdrawal == lptRawNAV exactly (no dust deduction). This is the capacity-math half of the 'zero-min-liquidity market reduces to a plain senior/junior market' reduction.	pr
`coverage_violation_saturates_capacity_to_zero:348`	VERIFIED	If the supplied state already violates a requirement, the corresponding capacity must saturate to exactly zero and never underflow or wrap to a large figure: coverage utilization > WAD implies maxSTDeposit == 0 and maxJTWithdrawal == 0; liquidity utilization > WAD implies maxSTDeposit == 0 and maxLPTWithdrawal == 0. (These are the degenerate cases where the 'safe to consume' claims do not apply, so zero is the only correct answer.)	pr
`liquidity_violation_saturates_capacity_to_zero:367`	VERIFIED	If the supplied state already violates a requirement, the corresponding capacity must saturate to exactly zero and never underflow or wrap to a large figure: coverage utilization > WAD implies maxSTDeposit == 0 and maxJTWithdrawal == 0; liquidity utilization > WAD implies maxSTDeposit == 0 and maxLPTWithdrawal == 0. (These are the degenerate cases where the 'safe to consume' claims do not apply, so zero is the only correct answer.)	pr
`no_spurious_zero_max_st_deposit:388`	VERIFIED	Conversely, a healthy market with strict slack beyond the dust pad must report strictly positive capacity: floor(jtEffectiveNAV*WAD/minCoverageWAD) > collateralNAV + dustTolerance and floor(lptRawNAV*WAD/minLiquidityWAD) > stEffectiveNAV + dustTolerance imply maxSTDeposit > 0; lptRawNAV > ceil((stEffectiveNAV+dustTolerance)*minLiquidityWAD/WAD) implies maxLPTWithdrawal > 0; jtEffectiveNAV > ceil((collateralNAV+dustTolerance)*minCoverageWAD/WAD) implies maxJTWithdrawal > 0. A spurious zero silently freezes every deposit/redemption path that sizes operations off these views.	pr

RULE	STATUS	DESCRIPTION	
`no_spurious_zero_max_lpt_withdrawal:403`	VERIFIED	Conversely, a healthy market with strict slack beyond the dust pad must report strictly positive capacity: floor(jtEffectiveNAV*WAD/minCoverageWAD) > collateralNAV + dustTolerance and floor(lptRawNAV*WAD/minLiquidityWAD) > stEffectiveNAV + dustTolerance imply maxSTDeposit > 0; lptRawNAV > ceil((stEffectiveNAV+dustTolerance)*minLiquidityWAD/WAD) implies maxLPTWithdrawal > 0; jtEffectiveNAV > ceil((collateralNAV+dustTolerance)*minCoverageWAD/WAD) implies maxJTWithdrawal > 0. A spurious zero silently freezes every deposit/redemption path that sizes operations off these views.	pr
`no_spurious_zero_max_jt_withdrawal:416`	VERIFIED	Conversely, a healthy market with strict slack beyond the dust pad must report strictly positive capacity: floor(jtEffectiveNAV*WAD/minCoverageWAD) > collateralNAV + dustTolerance and floor(lptRawNAV*WAD/minLiquidityWAD) > stEffectiveNAV + dustTolerance imply maxSTDeposit > 0; lptRawNAV > ceil((stEffectiveNAV+dustTolerance)*minLiquidityWAD/WAD) implies maxLPTWithdrawal > 0; jtEffectiveNAV > ceil((collateralNAV+dustTolerance)*minCoverageWAD/WAD) implies maxJTWithdrawal > 0. A spurious zero silently freezes every deposit/redemption path that sizes operations off these views.	pr
`max_st_deposit_monotonicity:436`	VERIFIED	Capacity must never grow as the market becomes riskier: maxSTDeposit is nondecreasing in jtEffectiveNAV and lptRawNAV and nonincreasing in collateralNAV and stEffectiveNAV; maxJTWithdrawal is nondecreasing in jtEffectiveNAV and nonincreasing in collateralNAV; maxLPTWithdrawal is nondecreasing in lptRawNAV and nonincreasing in stEffectiveNAV; and all three are nonincreasing in dustTolerance. A monotonicity break would mean a loss (falling J or P) can widen the admitted operation size.	pr
`max_jt_withdrawal_monotonicity:464`	VERIFIED	Capacity must never grow as the market becomes riskier: maxSTDeposit is nondecreasing in jtEffectiveNAV and lptRawNAV and nonincreasing in collateralNAV and stEffectiveNAV; maxJTWithdrawal is nondecreasing in jtEffectiveNAV and nonincreasing in collateralNAV; maxLPTWithdrawal is nondecreasing in lptRawNAV and nonincreasing in stEffectiveNAV; and all three are nonincreasing in dustTolerance. A monotonicity break would mean a loss (falling J or P) can widen the admitted operation size.	pr
`max_lpt_withdrawal_monotonicity:489`	VERIFIED	Capacity must never grow as the market becomes riskier: maxSTDeposit is nondecreasing in jtEffectiveNAV and lptRawNAV and nonincreasing in collateralNAV and stEffectiveNAV; maxJTWithdrawal is nondecreasing in jtEffectiveNAV and nonincreasing in collateralNAV; maxLPTWithdrawal is nondecreasing in lptRawNAV and nonincreasing in stEffectiveNAV; and all three are nonincreasing in dustTolerance. A monotonicity break would mean a loss (falling J or P) can widen the admitted operation size.	pr
`capacity_views_write_no_storage:522`	VERIFIED	The three capacity views must be pure functions of the caller-supplied SyncedAccountingState plus dustTolerance: they write no storage and read no other accountant storage field (in particular not the stale lastCollateralNAV / lastSTEffectiveNAV / lastJTEffectiveNAV / lastLPTRawNAV checkpoints or the stored coverage/liquidity parameters). If any NAV or ratio were taken from storage instead of the passed struct, the kernel would size operations against pre-sync state while the gate enforces post-sync state.	pr
`capacity_ignores_accountant_storage:545`	VERIFIED	The three capacity views must be pure functions of the caller-supplied SyncedAccountingState plus dustTolerance: they write no storage and read no other accountant storage field (in particular not the stale lastCollateralNAV / lastSTEffectiveNAV / lastJTEffectiveNAV / lastLPTRawNAV checkpoints or the	pr

RULE	STATUS	DESCRIPTION
		stored coverage/liquidity parameters). If any NAV or ratio were taken from storage instead of the passed struct, the kernel would size operations against pre-sync state while the gate enforces post-sync state.
`max_st_deposit_is_exactly_the_tighter_padded_leg:1070`	VERIFIED	• The three capacity views must be pure functions of the caller-supplied SyncedAccountingState plus dustTolerance: they write no storage and read no other accountant storage field (in particular not the stale lastCollateralNAV / lastSTEffectiveNAV / lastJTEffectiveNAV / lastLPTRawNAV checkpoints or the stored coverage/liquidity parameters). If any NAV or ratio were taken from storage instead of the passed struct, the kernel would size operations against pre-sync state while the gate enforces post-sync state. • The capacity figures must be TIGHT, not merely safe: the under-report relative to the exact requirement boundary must be bounded by the dust pad plus at most one rounding unit. Concretely, for a NAV-conserving state: (a) the coverage leg of maxSTDeposit must equal saturatingSub(floor(jtEffectiveNAV*WAD/minCoverageWAD) - collateralNAV, dustTolerance) — the exact largest x with (collateralNAV+x)*minCoverageWAD <= jtEffectiveNAV*WAD, minus the pad, never less; (b) the liquidity leg must equal saturatingSub(floor(lptRawNAV*WAD/minLiquidityWAD) - stEffectiveNAV, dustTolerance); (c) maxLPTWithdrawal >= (lptRawNAV - ceil(stEffectiveNAV*minLiquidityWAD/WAD)) - ceil(dustTolerance*minLiquidityWAD/WAD); (d) maxJTWithdrawal >= floor((jtEffectiveNAV*WAD - collateralNAV*minCoverageWAD)/(WAD - minCoverageWAD)) minus at most one surplus unit amplified by WAD/(WAD-minCoverageWAD) plus the dust term. A systematically or unboundedly conservative figure (e.g. from a mis-inverted closed form, a wrong rounding side, or the wrong WAD divisor) still satisfies every 'safe to consume' bound while silently freezing senior deposits and junior/LPT exits for everything that sizes maximal fills off these views.
`max_lpt_withdrawal_is_exactly_the_padded_closed_form:1085`	VERIFIED	• The three capacity views must be pure functions of the caller-supplied SyncedAccountingState plus dustTolerance: they write no storage and read no other accountant storage field (in particular not the stale lastCollateralNAV / lastSTEffectiveNAV / lastJTEffectiveNAV / lastLPTRawNAV checkpoints or the stored coverage/liquidity parameters). If any NAV or ratio were taken from storage instead of the passed struct, the kernel would size operations against pre-sync state while the gate enforces post-sync state. • The capacity figures must be TIGHT, not merely safe: the under-report relative to the exact requirement boundary must be bounded by the dust pad plus at most one rounding unit. Concretely, for a NAV-conserving state: (a) the coverage leg of maxSTDeposit must equal saturatingSub(floor(jtEffectiveNAV*WAD/minCoverageWAD) - collateralNAV, dustTolerance) — the exact largest x with (collateralNAV+x)*minCoverageWAD <= jtEffectiveNAV*WAD, minus the pad, never less; (b) the liquidity leg must equal saturatingSub(floor(lptRawNAV*WAD/minLiquidityWAD) - stEffectiveNAV, dustTolerance); (c) maxLPTWithdrawal >= (lptRawNAV - ceil(stEffectiveNAV*minLiquidityWAD/WAD)) - ceil(dustTolerance*minLiquidityWAD/WAD); (d) maxJTWithdrawal >= floor((jtEffectiveNAV*WAD - collateralNAV*minCoverageWAD)/(WAD - minCoverageWAD)) minus at most one surplus unit amplified by WAD/(WAD-minCoverageWAD) plus the dust term. A systematically or unboundedly conservative figure (e.g. from a mis-inverted closed form, a wrong rounding side, or the wrong WAD divisor) still satisfies every 'safe to consume' bound while silently freezing senior deposits and junior/LPT exits for everything that sizes maximal fills off these views.

RULE	STATUS	DESCRIPTION	
`max_jt_withdrawal_tightness:1111`	VERIFIED	• The three capacity views must be pure functions of the caller-supplied SyncedAccountingState plus dustTolerance: they write no storage and read no other accountant storage field (in particular not the stale lastCollateralNAV / lastSTEffectiveNAV / lastJTEffectiveNAV / lastLPTRawNAV checkpoints or the stored coverage/liquidity parameters). If any NAV or ratio were taken from storage instead of the passed struct, the kernel would size operations against pre-sync state while the gate enforces post-sync state. • The capacity figures must be TIGHT, not merely safe: the under-report relative to the exact requirement boundary must be bounded by the dust pad plus at most one rounding unit. Concretely, for a NAV-conserving state: (a) the coverage leg of maxSTDeposit must equal saturatingSub(floor(jtEffectiveNAV*WAD/minCoverageWAD) - collateralNAV, dustTolerance) — the exact largest x with (collateralNAV+x)*minCoverageWAD <= jtEffectiveNAV*WAD, minus the pad, never less; (b) the liquidity leg must equal saturatingSub(floor(lptRawNAV*WAD/minLiquidityWAD) - stEffectiveNAV, dustTolerance); (c) maxLPTWithdrawal >= (lptRawNAV - ceil(stEffectiveNAV*minLiquidityWAD/WAD)) - ceil(dustTolerance*minLiquidityWAD/WAD); (d) maxJTWithdrawal >= floor((jtEffectiveNAV*WAD - collateralNAV*minCoverageWAD)/(WAD - minCoverageWAD)) minus at most one surplus unit amplified by WAD/(WAD-minCoverageWAD) plus the dust term. A systematically or unboundedly conservative figure (e.g. from a mis-inverted closed form, a wrong rounding side, or the wrong WAD divisor) still satisfies every 'safe to consume' bound while silently freezing senior deposits and junior/LPT exits for everything that sizes maximal fills off these views.	pr
`capacity_views_never_revert:581`	VERIFIED	For every state whose config fields satisfy the accountant's own configuration invariants (minCoverageWAD < WAD, minLiquidityWAD < WAD) and NAV magnitudes within realistic bounds, all three capacity views terminate without reverting; in particular the (WAD - minCoverageWAD) denominator in maxJTWithdrawal is never zero. If any governance path could set minCoverageWAD == WAD, maxJTWithdrawal (and the kernel's JT maxRedeem view built on it) would revert permanently.	pr
`gate_admits_max_st_deposit_within_drift_budget:641`	VERIFIED	The kernel derives inkindMaxDeposit / inkindMaxRedeemable / lptMaxRedeemableMultiAsset from a PRE-op preview sync state, while the authoritative requirement checks run on the POST-op state, after the sync's fee and liquidity-premium senior share mints and after a freshly committed venue mark. If the dust padding does not dominate the resulting drift in stEffectiveNAV / lptRawNAV plus the NAV->collateral-asset and NAV->share conversion rounding performed by DepositLogic/RedemptionLogic, an operation sized at exactly the reported maximum can revert on the post-op coverage/liquidity gate (breaking keeper and entry-point maximal fills), or the reported maximum can be materially below the true boundary in a way that is exploitable for gate-boundary griefing.	pr

RULE	STATUS	DESCRIPTION	
`gate_admits_max_jt_withdrawal_within_drift_budget:679`	VERIFIED	The kernel derives inkindMaxDeposit / inkindMaxRedeemable / lptMaxRedeemableMultiAsset from a PRE-op preview sync state, while the authoritative requirement checks run on the POST-op state, after the sync's fee and liquidity-premium senior share mints and after a freshly committed venue mark. If the dust padding does not dominate the resulting drift in stEffectiveNAV / lptRawNAV plus the NAV->collateral-asset and NAV->share conversion rounding performed by DepositLogic/RedemptionLogic, an operation sized at exactly the reported maximum can revert on the post-op coverage/liquidity gate (breaking keeper and entry-point maximal fills), or the reported maximum can be materially below the true boundary in a way that is exploitable for gate-boundary griefing.	pr
`gate_admits_max_lpt_withdrawal_within_drift_budget:705`	VERIFIED	The kernel derives inkindMaxDeposit / inkindMaxRedeemable / lptMaxRedeemableMultiAsset from a PRE-op preview sync state, while the authoritative requirement checks run on the POST-op state, after the sync's fee and liquidity-premium senior share mints and after a freshly committed venue mark. If the dust padding does not dominate the resulting drift in stEffectiveNAV / lptRawNAV plus the NAV->collateral-asset and NAV->share conversion rounding performed by DepositLogic/RedemptionLogic, an operation sized at exactly the reported maximum can revert on the post-op coverage/liquidity gate (breaking keeper and entry-point maximal fills), or the reported maximum can be materially below the true boundary in a way that is exploitable for gate-boundary griefing.	pr
`gate_admits_max_st_deposit_under_prorata_collateral_dust:740`	VERIFIED	The kernel derives inkindMaxDeposit / inkindMaxRedeemable / lptMaxRedeemableMultiAsset from a PRE-op preview sync state, while the authoritative requirement checks run on the POST-op state, after the sync's fee and liquidity-premium senior share mints and after a freshly committed venue mark. If the dust padding does not dominate the resulting drift in stEffectiveNAV / lptRawNAV plus the NAV->collateral-asset and NAV->share conversion rounding performed by DepositLogic/RedemptionLogic, an operation sized at exactly the reported maximum can revert on the post-op coverage/liquidity gate (breaking keeper and entry-point maximal fills), or the reported maximum can be materially below the true boundary in a way that is exploitable for gate-boundary griefing.	pr
`dust_pad_dominates_dust_sized_senior_share_mint:831`	VIOLATED	The kernel derives inkindMaxDeposit / inkindMaxRedeemable / lptMaxRedeemableMultiAsset from a PRE-op preview sync state, while the authoritative requirement checks run on the POST-op state, after the sync's fee and liquidity-premium senior share mints and after a freshly committed venue mark. If the dust padding does not dominate the resulting drift in stEffectiveNAV / lptRawNAV plus the NAV->collateral-asset and NAV->share conversion rounding performed by DepositLogic/RedemptionLogic, an operation sized at exactly the reported maximum can revert on the post-op coverage/liquidity gate (breaking keeper and entry-point maximal fills), or the reported maximum can be materially below the true boundary in a way that is exploitable for gate-boundary griefing.	pr
`dust_pad_dominates_dust_sized_venue_mark_drift:868`	VIOLATED	The kernel derives inkindMaxDeposit / inkindMaxRedeemable / lptMaxRedeemableMultiAsset from a PRE-op preview sync state, while the authoritative requirement checks run on the POST-op state, after the sync's fee and liquidity-premium senior share mints and after a freshly committed venue mark. If the dust padding does not dominate the resulting drift in stEffectiveNAV / lptRawNAV plus the NAV->collateral-asset and NAV->share conversion rounding performed by DepositLogic/RedemptionLogic, an operation sized at exactly the reported maximum can revert on the post-op coverage/liquidity gate (breaking keeper and entry-point maximal fills), or the reported maximum can be materially	pr

RULE	STATUS	DESCRIPTION	
		below the true boundary in a way that is exploitable for gate-boundary griefing.	
`max_jt_withdrawal_bounded_for_unvalidated_caller_state:910`	VIOLATED	All three functions are permissionless and accept an arbitrary in-memory SyncedAccountingState with no check that it originated from a real sync or that it is NAV-conserving (collateralNAV == stEffectiveNAV + jtEffectiveNAV) or that lptRawNAV corresponds to the actual venue mark. With a non-conserving struct (e.g. jtEffectiveNAV > collateralNAV) maxJTWithdrawal can exceed the junior claim, and downstream conversion of that NAV into shares can yield a maxRedeem exceeding the tranche's total supply — an overstated, unfulfillable capacity for any integrator that reads the accountant directly.	pr
`dust_pad_acts_exactly_as_extra_collateral_and_senior_nav:940`	VERIFIED	dustTolerance is admin-settable with no upper bound (and settable to zero) under a role separate from the coverage/liquidity parameter setters, and it enters all three capacity formulas additively. A large dustTolerance drives maxSTDeposit, maxJTWithdrawal and maxLPTWithdrawal to zero on a perfectly healthy market, freezing every flow that sizes itself off these views; a zero dustTolerance removes the padding that is the sole reason these figures under-report, re-opening the 'operation sized at the reported maximum reverts on post-op rounding' failure the padding exists to prevent.	pr
`dust_tolerance_under_reports_by_at_most_the_pad:968`	VERIFIED	dustTolerance is admin-settable with no upper bound (and settable to zero) under a role separate from the coverage/liquidity parameter setters, and it enters all three capacity formulas additively. A large dustTolerance drives maxSTDeposit, maxJTWithdrawal and maxLPTWithdrawal to zero on a perfectly healthy market, freezing every flow that sizes itself off these views; a zero dustTolerance removes the padding that is the sole reason these figures under-report, re-opening the 'operation sized at the reported maximum reverts on post-op rounding' failure the padding exists to prevent.	pr
`healthy_market_reports_positive_capacity_for_any_dust_tolerance:1026`	VIOLATED	dustTolerance is admin-settable with no upper bound (and settable to zero) under a role separate from the coverage/liquidity parameter setters, and it enters all three capacity formulas additively. A large dustTolerance drives maxSTDeposit, maxJTWithdrawal and maxLPTWithdrawal to zero on a perfectly healthy market, freezing every flow that sizes itself off these views; a zero dustTolerance removes the padding that is the sole reason these figures under-report, re-opening the 'operation sized at the reported maximum reverts on post-op rounding' failure the padding exists to prevent.	pr
`zero_dust_tolerance_capacity_still_gate_safe:1031`	VERIFIED	dustTolerance is admin-settable with no upper bound (and settable to zero) under a role separate from the coverage/liquidity parameter setters, and it enters all three capacity formulas additively. A large dustTolerance drives maxSTDeposit, maxJTWithdrawal and maxLPTWithdrawal to zero on a perfectly healthy market, freezing every flow that sizes itself off these views; a zero dustTolerance removes the padding that is the sole reason these figures under-report, re-opening the 'operation sized at the reported maximum reverts on post-op rounding' failure the padding exists to prevent.	pr
`max_st_deposit_monotone_in_requirement_ratios:1132`	VERIFIED	Tightening a requirement ratio must never widen capacity: maxSTDeposit must be nonincreasing in state.minCoverageWAD and in state.minLiquidityWAD; maxLPTWithdrawal must be nonincreasing in state.minLiquidityWAD; and, for NAV-conserving states (jtEffectiveNAV <= collateralNAV), maxJTWithdrawal must be nonincreasing in state.minCoverageWAD. The junior case is the non-trivial one: minCoverageWAD appears both in the required-coverage numerator (raising it shrinks the surplus) and in the (WAD - minCoverageWAD) divisor (raising it amplifies whatever surplus remains), so a governance increase of the coverage requirement could counterintuitively increase the advertised junior exit size if the closed form is mis-derived or if the input state is not conservation-respecting.	pr

RULE	STATUS	DESCRIPTION	
`max_lpt_withdrawal_monotone_in_min_liquidity:1151`	VERIFIED	Tightening a requirement ratio must never widen capacity: maxSTDeposit must be nonincreasing in state.minCoverageWAD and in state.minLiquidityWAD; maxLPTWithdrawal must be nonincreasing in state.minLiquidityWAD; and, for NAV-conserving states (jtEffectiveNAV <= collateralNAV), maxJTWithdrawal must be nonincreasing in state.minCoverageWAD. The junior case is the non-trivial one: minCoverageWAD appears both in the required-coverage numerator (raising it shrinks the surplus) and in the (WAD - minCoverageWAD) divisor (raising it amplifies whatever surplus remains), so a governance increase of the coverage requirement could counterintuitively increase the advertised junior exit size if the closed form is mis-derived or if the input state is not conservation-respecting.	pr
`max_jt_withdrawal_monotone_in_min_coverage_up_to_rounding:1180`	VERIFIED	Tightening a requirement ratio must never widen capacity: maxSTDeposit must be nonincreasing in state.minCoverageWAD and in state.minLiquidityWAD; maxLPTWithdrawal must be nonincreasing in state.minLiquidityWAD; and, for NAV-conserving states (jtEffectiveNAV <= collateralNAV), maxJTWithdrawal must be nonincreasing in state.minCoverageWAD. The junior case is the non-trivial one: minCoverageWAD appears both in the required-coverage numerator (raising it shrinks the surplus) and in the (WAD - minCoverageWAD) divisor (raising it amplifies whatever surplus remains), so a governance increase of the coverage requirement could counterintuitively increase the advertised junior exit size if the closed form is mis-derived or if the input state is not conservation-respecting.	pr
`max_jt_withdrawal_monotone_in_min_coverage:1209`	VERIFIED	Tightening a requirement ratio must never widen capacity: maxSTDeposit must be nonincreasing in state.minCoverageWAD and in state.minLiquidityWAD; maxLPTWithdrawal must be nonincreasing in state.minLiquidityWAD; and, for NAV-conserving states (jtEffectiveNAV <= collateralNAV), maxJTWithdrawal must be nonincreasing in state.minCoverageWAD. The junior case is the non-trivial one: minCoverageWAD appears both in the required-coverage numerator (raising it shrinks the surplus) and in the (WAD - minCoverageWAD) divisor (raising it amplifies whatever surplus remains), so a governance increase of the coverage requirement could counterintuitively increase the advertised junior exit size if the closed form is mis-derived or if the input state is not conservation-respecting.	pr
`capacity_ignores_market_and_utilization_fields:1240`	VERIFIED	Each capacity bound must be a function of only the requirement-relevant fields of the supplied state (collateralNAV, stEffectiveNAV, jtEffectiveNAV, lptRawNAV, minCoverageWAD, minLiquidityWAD) plus dustTolerance. Holding those fixed and varying state.marketState, state.coverageUtilizationWAD, state.liquidityUtilizationWAD, or state.coverageLiquidationUtilizationWAD must not change any of the three outputs. In particular no coverage-liquidation or FIXED_TERM condition may unlock venue depth beyond the senior liquidity floor in maxLPTWithdrawal, nor relax the coverage-derived bounds: a state-conditional 'shortcut' branch would let LP capital withdraw market-making depth exactly when senior exits depend on it (or, symmetrically, would let a stale/degenerate utilization field widen a bound the kernel's post-op gate then rejects).	pr
`stale_low_lpt_mark_st_deposit_still_gate_safe:1287`	VERIFIED	preOpSyncTrancheAccounting deliberately returns lptRawNAV == 0 and liquidityUtilizationWAD == 0 as documented placeholders (the venue mark is committed separately, after the sync's fee/premium share mints), yet maxSTDeposit's liquidity leg and maxLPTWithdrawal read state.lptRawNAV as authoritative and cannot distinguish a placeholder from a genuinely empty venue. Any consumer that passes a state packet whose venue mark has not been refreshed evaluates floor(0*WAD/minLiquidityWAD) == 0 and reports maxSTDeposit == 0 for every market with a nonzero liquidity requirement — a silent, total freeze of senior deposit sizing and of entry-point ST fills — and, symmetrically, a mark that	pr

RULE	STATUS	DESCRIPTION	
		differs from the one the kernel's post-op liquidity gate enforces yields an LPT withdrawal figure that is either unconsumable or larger than the gate admits. Every call site should be checked to pass a state whose lptRawNAV equals the mark in force at gate-evaluation time.	
`stale_low_lpt_mark_lpt_withdrawal_still_gate_safe:1311`	VERIFIED	preOpSyncTrancheAccounting deliberately returns lptRawNAV == 0 and liquidityUtilizationWAD == 0 as documented placeholders (the venue mark is committed separately, after the sync's fee/premium share mints), yet maxSTDeposit's liquidity leg and maxLPTWithdrawal read state.lptRawNAV as authoritative and cannot distinguish a placeholder from a genuinely empty venue. Any consumer that passes a state packet whose venue mark has not been refreshed evaluates floor(0*WAD/minLiquidityWAD) == 0 and reports maxSTDeposit == 0 for every market with a nonzero liquidity requirement — a silent, total freeze of senior deposit sizing and of entry-point ST fills — and, symmetrically, a mark that differs from the one the kernel's post-op liquidity gate enforces yields an LPT withdrawal figure that is either unconsumable or larger than the gate admits. Every call site should be checked to pass a state whose lptRawNAV equals the mark in force at gate-evaluation time.	pr
`placeholder_lpt_mark_zeroes_liquidity_bounded_capacity:1333`	VERIFIED	preOpSyncTrancheAccounting deliberately returns lptRawNAV == 0 and liquidityUtilizationWAD == 0 as documented placeholders (the venue mark is committed separately, after the sync's fee/premium share mints), yet maxSTDeposit's liquidity leg and maxLPTWithdrawal read state.lptRawNAV as authoritative and cannot distinguish a placeholder from a genuinely empty venue. Any consumer that passes a state packet whose venue mark has not been refreshed evaluates floor(0*WAD/minLiquidityWAD) == 0 and reports maxSTDeposit == 0 for every market with a nonzero liquidity requirement — a silent, total freeze of senior deposit sizing and of entry-point ST fills — and, symmetrically, a mark that differs from the one the kernel's post-op liquidity gate enforces yields an LPT withdrawal figure that is either unconsumable or larger than the gate admits. Every call site should be checked to pass a state whose lptRawNAV equals the mark in force at gate-evaluation time.	pr
`jt_deposit_of_any_size_admitted_when_coverage_breached:1389`	VIOLATED	The capacity surface exposes maximums for ST deposits, JT withdrawals and LPT withdrawals but nothing corresponding to the kernel's JT_DEPOSIT gate, which requires the post-op coverage utilization to settle strictly below the liquidation threshold (the 'restore health in one shot' rule). Junior deposits are therefore advertised as unbounded/unconditional while, whenever coverage utilization has breached the threshold, every junior deposit smaller than the full recapitalization size reverts on the post-op gate. The consequence is the same class of failure the dust padding exists to prevent — an advertised, unconsumable capacity — and it bites precisely during recovery: keepers and the async queue cannot size a fillable junior deposit, queued partial fills can never complete, and an adversary who shaves coverage between request and execution can make an already-escrowed junior deposit permanently unexecutable.	pr
`jt_deposit_monotonically_restores_coverage:1406`	VERIFIED	The capacity surface exposes maximums for ST deposits, JT withdrawals and LPT withdrawals but nothing corresponding to the kernel's JT_DEPOSIT gate, which requires the post-op coverage utilization to settle strictly below the liquidation threshold (the 'restore health in one shot' rule). Junior deposits are therefore advertised as unbounded/unconditional while, whenever coverage utilization has breached the threshold, every junior deposit smaller than the full recapitalization size reverts on the post-op gate. The consequence is the same class of failure the dust padding exists to prevent — an advertised, unconsumable capacity — and it bites precisely during recovery: keepers and the async queue cannot size a fillable junior deposit, queued partial fills can never complete, and an adversary who shaves coverage	pr

RULE	STATUS	DESCRIPTION	
		between request and execution can make an already-escrowed junior deposit permanently unexecutable.	
`config_bounds_always_valid:577`	VERIFIED	For an initialized accountant (kernel != 0), the configuration must always satisfy the same bounds `initialize` enforces, no matter which setter last ran: minCoverageWAD < WAD; minLiquidityWAD < WAD; coverageLiquidationUtilizationWAD > WAD; each of stProtocolFeeWAD, jtProtocolFeeWAD, jtYieldShareProtocolFeeWAD, lptYieldShareProtocolFeeWAD <= MAX_PROTOCOL_FEE_WAD; maxJTYieldShareWAD + maxLPTYieldShareWAD <= WAD; jtYDM != lptYDM with both non-zero. No setter may install a value that `initialize` would have rejected. (minCoverageWAD < WAD is what keeps maxJTWithdrawal's WAD - minCoverage denominator non-zero; the yield-share sum bound is what guarantees the risk + liquidity premiums always fit inside the senior gain; Θ > WAD is what makes the liquidation threshold breachable only after real losses.)	pr
`kernel_and_grace_stamp_immutable_after_init:201`	VERIFIED	The `kernel` address and `fixedTermCommenceableAtTimestamp` are written exactly once, by `initialize`, and can never be modified by any parameter setter, YDM swap, or sync. A rotatable kernel pointer would void the `onlyRoycoKernel` gate on every NAV-mutating function (and the guard would sync against an attacker-chosen kernel); a movable grace stamp would let governance re-arm or re-suppress FIXED_TERM eligibility for a mature market.	pr
`initializer_consumed_once_kernel_is_set:493`	VERIFIED	The `kernel` address and `fixedTermCommenceableAtTimestamp` are written exactly once, by `initialize`, and can never be modified by any parameter setter, YDM swap, or sync. A rotatable kernel pointer would void the `onlyRoycoKernel` gate on every NAV-mutating function (and the guard would sync against an attacker-chosen kernel); a movable grace stamp would let governance re-arm or re-suppress FIXED_TERM eligibility for a mature market.	pr
`config_fields_have_dedicated_writers:63`	VERIFIED	A caller that the AccessManager authority denies must not be able to change any configuration field (fees, minCoverage, Θ, minLiquidity, max yield shares, fixed-term duration, dust tolerance, either YDM). Conversely, the kernel-only sync entrypoints (preOpSyncTrancheAccounting, commitLiquidityProviderTrancheRawNAV, postOpSyncTrancheAccounting) and the views must never write any configuration field: each config field's only writers are `initialize` and its dedicated `restricted` setter.	pr
`denied_caller_cannot_change_config:63`	VERIFIED	A caller that the AccessManager authority denies must not be able to change any configuration field (fees, minCoverage, Θ, minLiquidity, max yield shares, fixed-term duration, dust tolerance, either YDM). Conversely, the kernel-only sync entrypoints (preOpSyncTrancheAccounting, commitLiquidityProviderTrancheRawNAV, postOpSyncTrancheAccounting) and the views must never write any configuration field: each config field's only writers are `initialize` and its dedicated `restricted` setter.	pr
`guard_blocks_coverage_and_liquidity_worsening:63`	VERIFIED	Every guarded parameter change that succeeds must leave the market no worse off on both requirements, measured by the synced state before and after the change: (post coverageUtilization <= WAD) OR (post coverageUtilization <= pre coverageUtilization), and (post liquidityUtilization <= WAD) OR (post liquidityUtilization <= pre liquidityUtilization). In particular, a market that satisfies its coverage (resp. liquidity) requirement before the call can never be left in violation of it by a parameter change; a market already in violation can only be moved in the improving direction.	pr
`no_config_induced_liquidation:63`	VERIFIED	No parameter change may move the market from a non-liquidation state (coverageUtilization < coverageLiquidationUtilization) into a liquidation state (coverageUtilization >= coverageLiquidationUtilization) —	pr

RULE	STATUS	DESCRIPTION	
		neither by raising minCoverage (which raises coverage utilization) nor by lowering the liquidation threshold Θ. This matters because entering liquidation arms the JT-funded self-liquidation bonus for senior redeemers, i.e. a config change alone must never be able to start draining junior effective NAV into senior exit bonuses.	
`utilization_params_only_change_under_full_guard:201`	VERIFIED	Any state transition that writes minCoverageWAD, minLiquidityWAD, or coverageLiquidationUtilizationWAD must be bracketed by a full accounting sync before the write and another after it, with the coverage/liquidity/liquidation guard evaluated against the post-change sync result. No code path (setter, initializer-after-init, or otherwise) may write these three fields without the guard, since the guard is the only mechanism preventing a config change from violating the coverage/liquidity requirements.	pr
`pre_change_accrual_settled_under_old_config:63`	VERIFIED	PnL and yield-share accrual that occurred before a parameter change must be settled under the OLD configuration: raising a protocol fee, changing the max yield shares, or changing minCoverage/minLiquidity must not increase the protocol fee or the JT/LPT premium charged on collateral gain and on the accrual window that predates the transaction. Formally, the settling sync must run before the field write (so the trailing sync sees zero unsettled gain), and the yield-share accrual clock must be checkpointed at the old parameters. A reversed ordering would let governance retroactively fee/premium already-earned senior yield.	pr
`ydm_swap_succeeds_despite_reverting_sync:896`	VERIFIED	A YDM replacement must be executable by an authorized caller even when the market sync reverts (e.g. because the outgoing YDM reverts in yieldShare and bricks every sync), i.e. the pre-swap sync must be best-effort and never a precondition for the swap. The swap must still revert when the incoming YDM is the zero address or equals the other tranche's YDM, must re-initialize the incoming instance for this market, and must leave the stored YDM address unchanged if the initialization dispatch reverts (no partial swap).	pr
`ydm_swap_rejects_zero_or_duplicate_ydm:924`	VERIFIED	A YDM replacement must be executable by an authorized caller even when the market sync reverts (e.g. because the outgoing YDM reverts in yieldShare and bricks every sync), i.e. the pre-swap sync must be best-effort and never a precondition for the swap. The swap must still revert when the incoming YDM is the zero address or equals the other tranche's YDM, must re-initialize the incoming instance for this market, and must leave the stored YDM address unchanged if the initialization dispatch reverts (no partial swap).	pr
`lpt_ydm_swap_rejects_zero_or_duplicate_ydm:934`	VERIFIED	A YDM replacement must be executable by an authorized caller even when the market sync reverts (e.g. because the outgoing YDM reverts in yieldShare and bricks every sync), i.e. the pre-swap sync must be best-effort and never a precondition for the swap. The swap must still revert when the incoming YDM is the zero address or equals the other tranche's YDM, must re-initialize the incoming instance for this market, and must leave the stored YDM address unchanged if the initialization dispatch reverts (no partial swap).	pr
`ydm_swap_is_atomic:948`	VERIFIED	A YDM replacement must be executable by an authorized caller even when the market sync reverts (e.g. because the outgoing YDM reverts in yieldShare and bricks every sync), i.e. the pre-swap sync must be best-effort and never a precondition for the swap. The swap must still revert when the incoming YDM is the zero address or equals the other tranche's YDM, must re-initialize the incoming instance for this market, and must leave the stored YDM address unchanged if the initialization dispatch reverts (no partial swap).	pr
`ydm_swap_target_must_be_a_genuine_ydm:973`	VIOLATED	The YDM setters validate only non-zero and distinct-from-the-other-YDM, and the initialization dispatch is skipped entirely when the supplied initialization calldata is empty. A swap with empty data — or with an address that is not a YDM at all (the kernel, the accountant itself, a tranche) — installs an instance	pr

RULE	STATUS	DESCRIPTION	
		that has no per-market state, so yieldShare/previewYieldShare fail shut (e.g. FixedYDM's UNINITIALIZED_YDM), bricking every accrual, every sync, and therefore all deposits and redemptions until another swap. The same unconstrained low-level initialization dispatch is an arbitrary-calldata call made under the accountant's privileged identity (the kernel trusts msg.sender == accountant), so the target must be constrained to a genuine YDM.	
`ydm_swap_leaves_no_unaccrued_window:1004`	VIOLATED	Because the pre-swap sync is best-effort, a swap can complete while lastYieldShareAccrualTimestamp remains stale (the sync reverted, e.g. the outgoing YDM reverts, or the kernel is paused / the oracle is stale). The next accrual then multiplies the incoming YDM's instantaneous share by the whole elapsed window that actually ran under the outgoing YDM, retroactively re-pricing the JT risk and LPT liquidity premiums for that period — a value transfer between ST and JT/LPT that governance can time. A correct swap should leave no un-accrued window attributable to the outgoing model.	pr
`dust_tolerance_stays_bounded:1038`	VIOLATED	setDustTolerance accepts any NAV value with no bound, and the withSyncedAccounting guard inspects only coverage/liquidity utilizations — never the impermanent loss or the market state. A large dust tolerance makes the state machine's `jtImpermanentLoss <= dustTolerance` condition true, forcing PERPETUAL and permanently erasing the junior tranche's recoverable drawdown (turning it into a realized junior loss), while simultaneously suppressing all premium and protocol-fee bookings via the stGain > D / jtGain > D gates. The value destroyed for JT is not detectable by the guard and cannot be undone by lowering the tolerance again.	pr
`sync_cannot_erase_live_junior_recovery_claim:1078`	VIOLATED	setDustTolerance accepts any NAV value with no bound, and the withSyncedAccounting guard inspects only coverage/liquidity utilizations — never the impermanent loss or the market state. A large dust tolerance makes the state machine's `jtImpermanentLoss <= dustTolerance` condition true, forcing PERPETUAL and permanently erasing the junior tranche's recoverable drawdown (turning it into a realized junior loss), while simultaneously suppressing all premium and protocol-fee bookings via the stGain > D / jtGain > D gates. The value destroyed for JT is not detectable by the guard and cannot be undone by lowering the tolerance again.	pr
`capacity_views_never_revert:1113`	VIOLATED	An extreme dustTolerance makes the checked NAV additions inside the capacity views (collateralNAV + dustTolerance in maxSTDeposit/maxJTWithdrawal, stEffectiveNAV + dustTolerance in maxSTDeposit/maxLPTWithdrawal) revert, so every capacity quote the kernel and entry point rely on for gating and for max-deposit/withdraw reporting reverts — a denial of service on the whole market from a single unvalidated admin parameter, which is also hard to recover from because the recovery setter itself runs two full syncs.	pr
`guarded_setter_executable_when_sync_reverts:1143`	VIOLATED	The guard performs two mandatory (non-best-effort) calls into the kernel's sync entrypoint, which is pause-gated, reentrancy-gated, and pokes the collateral oracle inside a price frame. Consequently a paused kernel, an oracle that reverts on staleness/circuit-break, or an in-flight operation frame makes every guarded parameter setter revert — precisely in the scenarios where a configuration fix (dust tolerance, coverage/liquidity thresholds, fees) may be the intended remediation. A single pauser can therefore indefinitely freeze all accountant parameter governance, while unpausing requires a different role.	pr
`sync_packet_reports_true_liquidity_state:1173`	VIOLATED	The accountant returns liquidityUtilizationWAD (and lptRawNAV) as zero placeholders from its pre-op sync, relying on the kernel to overwrite them with the freshly committed venue mark before returning the packet to the guard. If the packet reaching the guard carries the unpatched placeholder (or a mark taken before the sync's fee/premium share mints), the liquidity clause `postOp.liquidityUtilizationWAD <= WAD` is trivially satisfied and a minLiquidity increase (or any change	pr

RULE	STATUS	DESCRIPTION	
		raising senior claims) can settle the market below the senior liquidity floor undetected.	
`requirements_cannot_be_zeroed:1211`	VIOLATED	Setting minCoverage or minLiquidity to zero drives the corresponding utilization to zero by definition, so the guard's 'not worse than 100%' test is trivially satisfied while the requirement itself is removed: with minCoverage == 0 the coverage gate and maxJTWithdrawal bound become vacuous (junior can redeem the entire loss-absorption buffer out from under senior), and with minLiquidity == 0 maxLPTWithdrawal returns the full venue mark (the LPT can drain all secondary liquidity guaranteed to senior). The guard cannot distinguish 'healthy' from 'unconstrained', so this governance action needs its own bound or delay-based justification.	pr
`premium_payment_consumes_accrual_window:1252`	VIOLATED	Whenever a sync actually books a premium out of the senior slice — i.e. it moves a nonzero JT risk premium into jtEffectiveNAV and/or reports a nonzero lptLiquidityPremium for the kernel to mint as ST shares — the time-weighted yield-share accumulators (twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) must be zeroed and lastPremiumPaymentTimestamp advanced to the current block in the same call. No accrual window may fund more than one premium payment. This is a parameter-governance property because the consumption gate is `premiumsPaid = (stGain > dustTolerance)` — an admin-set, unvalidated parameter — while the premium NAV movement itself is not gated on it: an oversized dustTolerance makes every gain with stGain <= D pay a premium out of a window that is never consumed, so the same accrued window is re-applied to every subsequent gain, systematically over-paying JT/LPT out of senior NAV.	pr
`premium_payment_consumes_accrual_window_zero_dust:1262`	VERIFIED	Whenever a sync actually books a premium out of the senior slice — i.e. it moves a nonzero JT risk premium into jtEffectiveNAV and/or reports a nonzero lptLiquidityPremium for the kernel to mint as ST shares — the time-weighted yield-share accumulators (twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) must be zeroed and lastPremiumPaymentTimestamp advanced to the current block in the same call. No accrual window may fund more than one premium payment. This is a parameter-governance property because the consumption gate is `premiumsPaid = (stGain > dustTolerance)` — an admin-set, unvalidated parameter — while the premium NAV movement itself is not gated on it: an oversized dustTolerance makes every gain with stGain <= D pay a premium out of a window that is never consumed, so the same accrued window is re-applied to every subsequent gain, systematically over-paying JT/LPT out of senior NAV.	pr
`fixed_term_end_not_movable_by_duration_change:1294`	VERIFIED	A change to fixedTermDurationSeconds must not alter the end timestamp of a fixed term already in progress: the term end is stamped only on the PERPETUAL -> FIXED_TERM entry transition, so if the market is FIXED_TERM before and after the setter call, fixedTermEndTimestamp must be unchanged. The single permitted effect on a running term is the zero-duration case, which terminates it. If a duration change could re-stamp or extend a live term, governance could repeatedly bump the duration to keep ST/JT deposits and redemptions and LPT redemptions frozen indefinitely, trapping senior and junior capital.	pr
`sync_does_not_restamp_running_fixed_term:1324`	VERIFIED	A change to fixedTermDurationSeconds must not alter the end timestamp of a fixed term already in progress: the term end is stamped only on the PERPETUAL -> FIXED_TERM entry transition, so if the market is FIXED_TERM before and after the setter call, fixedTermEndTimestamp must be unchanged. The single permitted effect on a running term is the zero-duration case, which terminates it. If a duration change could re-stamp or extend a live term, governance could repeatedly bump the duration to keep ST/JT deposits and redemptions and LPT redemptions frozen indefinitely, trapping senior and junior capital.	pr

RULE	STATUS	DESCRIPTION	
`setters_never_write_nav_checkpoints:63`	VERIFIED	A parameter setter's net effect on the accounting checkpoints (lastCollateralNAV, lastSTEffectiveNAV, lastJTEffectiveNAV, lastJTImpermanentLoss, lastMarketState, fixedTermEndTimestamp) must be exactly what the two bracketing accounting syncs, evaluated at the new configuration, would produce. In particular no setter may write the NAV checkpoints directly, and any direct write of lastJTImpermanentLoss / lastMarketState / fixedTermEndTimestamp inside a setter (the zero-duration branch of setFixedTermDuration) must coincide with the result the market state machine would force at the new configuration, so that at the end of every setter call NAV conservation (C == S + J) and the state/IL coupling (PERPETUAL <=> IL == 0 <=> fixedTermEndTimestamp == 0) still hold. A setter that mutated checkpoints outside the state machine could break conservation or leave an internally inconsistent lifecycle state that the kernel's gates then act on.	pr
`zero_duration_reset_is_internally_consistent:1382`	VERIFIED	A parameter setter's net effect on the accounting checkpoints (lastCollateralNAV, lastSTEffectiveNAV, lastJTEffectiveNAV, lastJTImpermanentLoss, lastMarketState, fixedTermEndTimestamp) must be exactly what the two bracketing accounting syncs, evaluated at the new configuration, would produce. In particular no setter may write the NAV checkpoints directly, and any direct write of lastJTImpermanentLoss / lastMarketState / fixedTermEndTimestamp inside a setter (the zero-duration branch of setFixedTermDuration) must coincide with the result the market state machine would force at the new configuration, so that at the end of every setter call NAV conservation (C == S + J) and the state/IL coupling (PERPETUAL <=> IL == 0 <=> fixedTermEndTimestamp == 0) still hold. A setter that mutated checkpoints outside the state machine could break conservation or leave an internally inconsistent lifecycle state that the kernel's gates then act on.	pr
`nav_conservation:510`	VERIFIED	A parameter setter's net effect on the accounting checkpoints (lastCollateralNAV, lastSTEffectiveNAV, lastJTEffectiveNAV, lastJTImpermanentLoss, lastMarketState, fixedTermEndTimestamp) must be exactly what the two bracketing accounting syncs, evaluated at the new configuration, would produce. In particular no setter may write the NAV checkpoints directly, and any direct write of lastJTImpermanentLoss / lastMarketState / fixedTermEndTimestamp inside a setter (the zero-duration branch of setFixedTermDuration) must coincide with the result the market state machine would force at the new configuration, so that at the end of every setter call NAV conservation (C == S + J) and the state/IL coupling (PERPETUAL <=> IL == 0 <=> fixedTermEndTimestamp == 0) still hold. A setter that mutated checkpoints outside the state machine could break conservation or leave an internally inconsistent lifecycle state that the kernel's gates then act on.	pr
`fixed_term_lifecycle_coupling:525`	VERIFIED	A parameter setter's net effect on the accounting checkpoints (lastCollateralNAV, lastSTEffectiveNAV, lastJTEffectiveNAV, lastJTImpermanentLoss, lastMarketState, fixedTermEndTimestamp) must be exactly what the two bracketing accounting syncs, evaluated at the new configuration, would produce. In particular no setter may write the NAV checkpoints directly, and any direct write of lastJTImpermanentLoss / lastMarketState / fixedTermEndTimestamp inside a setter (the zero-duration branch of setFixedTermDuration) must coincide with the result the market state machine would force at the new configuration, so that at the end of every setter call NAV conservation (C == S + J) and the state/IL coupling (PERPETUAL <=> IL == 0 <=> fixedTermEndTimestamp == 0) still hold. A setter that mutated checkpoints outside the state machine could break conservation or leave an internally inconsistent lifecycle state that the kernel's gates then act on.	pr

RULE	STATUS	DESCRIPTION	
`liquidation_threshold_cannot_be_lowered:1429`	VIOLATED	coverageLiquidationUtilizationWAD (Θ) is bounded only from below by `> WAD`, with no floor above WAD and no bound on how far a single change may move it. While the market is healthy, governance can set Θ = WAD + 1 and pass the guard (its Θ clause is satisfied because postΘ > postCoverageUtilization), pre-arming the self-liquidation bonus so that the very first wei of coverage shortfall lets senior redeemers extract junior effective NAV as a bonus. Because each bonus payment reduces J and therefore raises coverage utilization, liquidation stays armed, giving a self-sustaining drain of the junior loss-absorption buffer into senior exits. A guard that only checks 'not in liquidation at the moment of the change' cannot detect this pre-arming.	pr
`protocol_fees_cannot_reach_full_appropriation:1463`	VIOLATED	MAX_PROTOCOL_FEE_WAD equals WAD, so all four fee setters accept a 100% rate, and the withSyncedAccounting guard inspects only coverage/liquidity utilization — never the fee fields — nor bounds the size of a single change. One fee-setter action can therefore route the entire senior residual gain (stProtocolFee), the entire junior gain (jtProtocolFee), the entire JT risk premium (jtYieldShareProtocolFee) and the entire LPT liquidity premium (lptYieldShareProtocolFee) to the protocol fee recipient as minted shares, so junior capital bears first-loss and LPT capital provides market-making depth for exactly zero compensation while senior holders are diluted by their own gain. A cap set at 100% provides no protection, and the change is invisible to the only guard on the setter path.	pr
`ydm_swap_always_reinitializes_incoming_ydm:1498`	VIOLATED	The YDM initialization dispatch is skipped entirely when the supplied initialization calldata is empty, and YDM per-market state is keyed by the calling accountant's address and survives being swapped away. Installing an instance that was already initialized for this accountant in an earlier stint (e.g. rotating back to a previously used adaptive YDM) with empty calldata therefore succeeds without resetting anything: the accountant resurrects stale curve parameters and a stale last-adaptation timestamp. The first accrual after the swap then applies the whole intervening gap as one adaptation step, snapping the JT risk / LPT liquidity yield share to its clamp and mispricing the premium for that window — a governance-timed value transfer between ST and JT/LPT. This is the complement of installing a never-initialized instance (which fails shut): here the swap succeeds but with state accrued outside the current installation.	pr
`governance_calls_preserve_accrual_state:63`	VERIFIED	A restricted parameter setter or a YDM swap must not modify the yield-share accrual state — the two time-weighted accumulators (twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) and the two clocks (lastYieldShareAccrualTimestamp tA, lastPremiumPaymentTimestamp tP) — other than exactly as the accrual/premium-payment logic inside its own bracketing accounting sync(s) would. Concretely, after any governance call: tP <= tA <= block.timestamp still holds, tA/tP are only ever advanced to the current block, and neither accumulator grows by more than (max yield share) * (elapsed since the previous accrual). This matters because minCoverageWAD, minLiquidityWAD, fixedTermDurationSeconds, lastMarketState, fixedTermEndTimestamp, tA and tP are all packed into a single storage slot that three of the setters write. A packed-write regression that pushes tP past the current block makes the premium leg's `block.timestamp - lastPremiumPaymentTimestamp` subtraction underflow, so every pre-op sync reverts and the entire market (deposits, redemptions, keeper syncs) is permanently bricked with no recovery path — the guarded setters themselves require two successful syncs, and the best-effort YDM swap never touches tP. Conversely, shrinking tP or inflating an accumulator directly inflates `stGain * tw / (elapsed * WAD)`, overpaying the JT risk and LPT liquidity premiums out of senior effective NAV.	pr

RULE	STATUS	DESCRIPTION	
`sync_maintains_accrual_clock_bounds:1561`	VERIFIED	A restricted parameter setter or a YDM swap must not modify the yield-share accrual state — the two time-weighted accumulators (twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) and the two clocks (lastYieldShareAccrualTimestamp tA, lastPremiumPaymentTimestamp tP) — other than exactly as the accrual/premium-payment logic inside its own bracketing accounting sync(s) would. Concretely, after any governance call: tP <= tA <= block.timestamp still holds, tA/tP are only ever advanced to the current block, and neither accumulator grows by more than (max yield share) * (elapsed since the previous accrual). This matters because minCoverageWAD, minLiquidityWAD, fixedTermDurationSeconds, lastMarketState, fixedTermEndTimestamp, tA and tP are all packed into a single storage slot that three of the setters write. A packed-write regression that pushes tP past the current block makes the premium leg's `block.timestamp - lastPremiumPaymentTimestamp` subtraction underflow, so every pre-op sync reverts and the entire market (deposits, redemptions, keeper syncs) is permanently bricked with no recovery path — the guarded setters themselves require two successful syncs, and the best-effort YDM swap never touches tP. Conversely, shrinking tP or inflating an accumulator directly inflates `stGain * tw / (elapsed * WAD)`, overpaying the JT risk and LPT liquidity premiums out of senior effective NAV.	pr
`sync_accrual_growth_is_bounded:1586`	VERIFIED	A restricted parameter setter or a YDM swap must not modify the yield-share accrual state — the two time-weighted accumulators (twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) and the two clocks (lastYieldShareAccrualTimestamp tA, lastPremiumPaymentTimestamp tP) — other than exactly as the accrual/premium-payment logic inside its own bracketing accounting sync(s) would. Concretely, after any governance call: tP <= tA <= block.timestamp still holds, tA/tP are only ever advanced to the current block, and neither accumulator grows by more than (max yield share) * (elapsed since the previous accrual). This matters because minCoverageWAD, minLiquidityWAD, fixedTermDurationSeconds, lastMarketState, fixedTermEndTimestamp, tA and tP are all packed into a single storage slot that three of the setters write. A packed-write regression that pushes tP past the current block makes the premium leg's `block.timestamp - lastPremiumPaymentTimestamp` subtraction underflow, so every pre-op sync reverts and the entire market (deposits, redemptions, keeper syncs) is permanently bricked with no recovery path — the guarded setters themselves require two successful syncs, and the best-effort YDM swap never touches tP. Conversely, shrinking tP or inflating an accumulator directly inflates `stGain * tw / (elapsed * WAD)`, overpaying the JT risk and LPT liquidity premiums out of senior effective NAV.	pr
`premium_clock_behind_accrual_clock:1512`	VERIFIED	A restricted parameter setter or a YDM swap must not modify the yield-share accrual state — the two time-weighted accumulators (twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) and the two clocks (lastYieldShareAccrualTimestamp tA, lastPremiumPaymentTimestamp tP) — other than exactly as the accrual/premium-payment logic inside its own bracketing accounting sync(s) would. Concretely, after any governance call: tP <= tA <= block.timestamp still holds, tA/tP are only ever advanced to the current block, and neither accumulator grows by more than (max yield share) * (elapsed since the previous accrual). This matters because minCoverageWAD, minLiquidityWAD, fixedTermDurationSeconds, lastMarketState, fixedTermEndTimestamp, tA and tP are all packed into a single storage slot that three of the setters write. A packed-write regression that pushes tP past the current block makes the premium leg's `block.timestamp - lastPremiumPaymentTimestamp` subtraction underflow, so every pre-op sync reverts and the entire market (deposits,	pr

RULE	STATUS	DESCRIPTION	
		redemptions, keeper syncs) is permanently bricked with no recovery path — the guarded setters themselves require two successful syncs, and the best-effort YDM swap never touches tP. Conversely, shrinking tP or inflating an accumulator directly inflates `stGain * tw / (elapsed * WAD)`, overpaying the JT risk and LPT liquidity premiums out of senior effective NAV.	
`liquidation_threshold_raise_is_bounded:1638`	VIOLATED	coverageLiquidationUtilizationWAD (Theta) is bounded only from below (> WAD) and the guard's Theta clause is satisfied by any raise (preTheta <= postTheta), unconditionally and regardless of magnitude — trivially so while the market is healthy. Because `coverageUtilization >= Theta` is one of the conditions that force the market state machine back to PERPETUAL, raising Theta to a practically unreachable value silently deletes that forcing condition. A market that subsequently takes a junior drawdown then remains in FIXED_TERM — where the kernel blocks senior and junior deposits and redemptions and LPT redemptions — while it is deeply undercollateralized, for the whole remaining fixed-term duration, and the JT-funded self-liquidation bonus that exists to force senior exits open never arms. Governance can pre-arm this state at a moment when the guard has nothing to object to, trapping senior and junior capital exactly in the scenario the liquidation threshold was designed to relieve. A guard that only asks whether the market is in liquidation at the instant of the change cannot detect a Theta change that only takes effect on a future loss.	pr
`nav_conservation:339`	VERIFIED	The accountant's checkpointed collateral NAV always equals the sum of the two checkpointed tranche effective NAVs: $.lastCollateralNAV == $.lastSTEffectiveNAV + $.lastJTEffectiveNAV (all in NAV units, unsigned). This holds in every state, including the uninitialized all-zero state.	pr
`perpetual_implies_no_fixed_term_end:347`	VERIFIED	If the accountant's checkpointed market state is PERPETUAL ($.lastMarketState == MarketState.PERPETUAL) then the stored fixed-term end timestamp is zero ($.fixedTermEndTimestamp == 0).	pr
`perpetual_implies_zero_jt_impermanent_loss:355`	VERIFIED	If the accountant's checkpointed market state is PERPETUAL ($.lastMarketState == MarketState.PERPETUAL) then the checkpointed junior tranche impermanent loss is zero ($.lastJTImpermanentLoss == 0 NAV units). Equivalently, a non-zero recorded JT impermanent loss can only exist while the market is in the FIXED_TERM state.	pr
`fixed_term_implies_nonzero_jt_impermanent_loss:363`	VERIFIED	If the accountant's checkpointed market state is FIXED_TERM ($.lastMarketState == MarketState.FIXED_TERM) then the checkpointed junior tranche impermanent loss is non-zero ($.lastJTImpermanentLoss != 0 NAV units): a fixed term is only ever recorded while an outstanding junior drawdown exists.	pr
`zero_fixed_term_duration_implies_perpetual:389`	VERIFIED	If the configured fixed-term duration is zero ($.fixedTermDurationSeconds == 0) then the checkpointed market state is PERPETUAL ($.lastMarketState == MarketState.PERPETUAL): a market configured with no fixed-term duration can never be recorded as being in a fixed term.	pr
`initializer_consumed_once_kernel_is_set:263`	VERIFIED	If the configured fixed-term duration is zero ($.fixedTermDurationSeconds == 0) then the checkpointed market state is PERPETUAL ($.lastMarketState == MarketState.PERPETUAL): a market configured with no fixed-term duration can never be recorded as being in a fixed term.	pr
`uninitialized_accountant_is_pristine:234`	VERIFIED	If the configured fixed-term duration is zero ($.fixedTermDurationSeconds == 0) then the checkpointed market state is PERPETUAL ($.lastMarketState == MarketState.PERPETUAL): a market configured with no fixed-term duration can never be recorded as being in a fixed term.	pr
`coverage_config_bounds:403`	VERIFIED	For an initialized accountant (i.e. whenever $.kernel != address(0)), the coverage configuration is within its admissible	pr

RULE	STATUS	DESCRIPTION	
		range: $.minCoverageWAD < WAD (1e18) and $.coverageLiquidationUtilizationWAD > WAD (1e18).	
`min_liquidity_bound:411`	VERIFIED	For an initialized accountant (i.e. whenever $.kernel != address(0)), the minimum liquidity requirement is strictly below 100%: $.minLiquidityWAD < WAD (1e18).	pr
`max_yield_shares_sum_bounded:419`	VERIFIED	The configured maximum JT and LPT yield shares always sum to at most 100% of senior appreciation: $.maxJTYieldShareWAD + $.maxLPTYieldShareWAD <= WAD (1e18). This holds in every state including the uninitialized all-zero state, and implies each of them individually is at most WAD.	pr
`protocol_fees_bounded:427`	VERIFIED	Each of the four configured protocol fee percentages is at most the maximum protocol fee: $.stProtocolFeeWAD <= MAX_PROTOCOL_FEE_WAD (1e18), $.jtProtocolFeeWAD <= MAX_PROTOCOL_FEE_WAD, $.jtYieldShareProtocolFeeWAD <= MAX_PROTOCOL_FEE_WAD and $.lptYieldShareProtocolFeeWAD <= MAX_PROTOCOL_FEE_WAD. This holds in every state including the uninitialized all-zero state.	pr
`ydms_distinct_and_nonzero:438`	VERIFIED	For an initialized accountant (i.e. whenever $.kernel != address(0)), both yield distribution models are configured and distinct: $.jtYDM != address(0), $.lptYDM != address(0) and $.jtYDM != $.lptYDM.	pr
`initialization_all_or_nothing:477`	VERIFIED	The accountant is either fully uninitialized or fully initialized with respect to the two fields that initialize() must set to non-zero values: $.kernel != address(0) if and only if $.coverageLiquidationUtilizationWAD != 0.	pr

Appendix: cut short by the run budget (1)

The run's budget ran out while these components were being formalized. Anything they published was accepted without the usual validation gates, so neither the encodings nor their verification results are reliable; nothing here contributes to the property groups, outcome counts, or coverage figures above.

NAV Sync Engine PARTIAL DRAFT PUBLISHED

Of 47 inferred properties: 47 drafted but never verified.

Unvalidated draft kept for inspection: `certora/autospec_NAV_Sync_Engine.spec.unverified` · last prover run

▶ Per-property breakdown (47)

Source modifications

The components below were verified against modified source: their outcomes are claims about the modified code, not the code as it exists on disk. Each modification was made to work around a construct the verification tooling cannot handle; the editor's account of every edit and the cumulative diff from the on-disk source follow.

Capacity Math (2 edits)

Edit 1 — `1961d423c490a80b11fe77b83123c58d0e6482647df7171902b39cc4dadbc7c1`

Single-file change: `certora/harnesses/RoycoDayAccountantHarness.sol`, previously an empty `contract RoycoDayAccountantHarness is RoycoDayAccountant {}`, now additionally declares verification-only plumbing. Nothing under `src/` was touched, and the existing contract declaration/import line is unchanged. Added, in order:

1. `struct RawSyncedAccountingState` — a mirror of `SyncedAccountingState` with identical field order and names, every field typed plain `uint256`.
2. `_toSyncedAccountingState(RawSyncedAccountingState memory) internal pure returns (SyncedAccountingState memory)` — field-by-field conversion. `NAV_UNIT` fields are `NAV_UNIT.wrap(...)`; `marketState` is mapped `0 => MarketState.PERPETUAL`, anything else `=> MarketState.FIXED_TERM` (no raw enum cast); `fixedTermEndTimestamp` gets an explicit `uint32(...)` truncating cast; the four `uint256` fields pass through.
3. `maxSTDepositRaw`, `maxJTWithdrawalRaw`, `maxLPTWithdrawalRaw` — `external view`, each takes the raw struct, converts it, calls the real inherited capacity view on `this` (they are `external`, so `this.` is required), and returns `NAV_UNIT.unwrap(...)` of the result.
4. `coverageUtilizationRaw(uint256,uint256,uint256)` and `liquidityUtilizationRaw(uint256,uint256,uint256)` — `external pure`, wrapping the NAV arguments and forwarding to `UtilizationLogic._computeCoverageUtilization` / `_computeLiquidityUtilization` (the same library functions the accountant itself calls), returning the raw `uint256`.
5. `dustToleranceRaw() external view returns (uint256)` — reads `_getRoycoDayAccountantStorage().dustTolerance` and unwraps it.

6. `harnessSetDustTolerance(uint256) external` — writes `$.dustTolerance` directly, with no access control and no `withSyncedAccounting` modifier.
7. `harnessSetCapacityIrrelevantState(...) external` — ten `uint256` parameters, writing `lastCollateralNAV`, `lastSTEffectiveNAV`, `lastJTEffectiveNAV`, `lastLPTRawNAV`, `lastJTImpermanentLoss`, `minCoverageWAD` (`uint64(...)`), `minLiquidityWAD` (`uint64(...)`), `coverageLiquidationUtilizationWAD`, `lastMarketState` (same 0/non-zero mapping), `fixedTermEndTimestamp` (`uint32(...)`); `dustTolerance` is deliberately not written. Also unrestricted and unsynced.
8. `capacityAtPreviewedStateRaw(uint256) external view` — calls the real `previewSyncTrancheAccounting` (public, so called directly), then the three real capacity views on `this`, and returns nine `uint256`s: the three capacities plus the previewed state's `collateralNAV`, `stEffectiveNAV`, `jtEffectiveNAV`, `lptRawNAV`, `minCoverageWAD`, `minLiquidityWAD`.

Comments on items 6 and 7 state explicitly, in a banner and per-function, that these writers exist only on the Certora harness and are not part of the deployed contract's surface.

No new imports were needed: the file's existing plain `import "src/accountant/RoycoDayAccountant.sol";` transitively brings `NAV_UNIT`, `MarketState`, `SyncedAccountingState` and `UtilizationLogic` into scope, and `NAV_UNIT.wrap` / `.unwrap` avoid needing `toUint256`, which the accountant does not import.

HEADLINE FOR THE AUDITOR: items 6 and 7 add two unauthenticated storage writers to the harness's external surface. Any parametric rule or invariant that ranges over all methods of `RoycoDayAccountantHarness` will now see them, so the harness no longer models the deployed contract's access control or the `withSyncedAccounting` guard on `setDustTolerance`. That is precisely what was requested (to quantify over the admin-settable configuration in one rule), but it must be filtered out wherever it is not wanted.

Soundness argument

Charter category: Exposure (additive harness surface), confined to the existing verification harness file.

Additive with respect to `src/`: no file under `src/` is modified, so every existing function's reads, writes, reverts, events and external calls are bit-identical. Nothing in the added code re-implements accounting: the three capacity wrappers and the bundler call the real `maxSTDeposit` / `maxJTWithdrawal` / `maxLPTWithdrawal` / `previewSyncTrancheAccounting`, and the two utilization wrappers call the same `UtilizationLogic` internal functions the accountant's own post-op sync and YDM drivers call. No `require`, `assert` or `revert` was added or removed anywhere.

Faithfulness of the conversion. `NAV_UNIT` is a `uint256` UDVT, so `NAV_UNIT.wrap` / `unwrap` are exact, total, and free of arithmetic. The four already-`uint256` fields pass through untouched. Two fields are narrower than `uint256` and are handled as instructed:

- `marketState` is mapped rather than cast, so an unconstrained input never yields an out-of-range enum value; the mapping is total (0 => PERPETUAL, else FIXED_TERM) and, restricted to inputs 0 and 1, agrees exactly with the enum's encoding. The only divergence from a hypothetical exact cast is on inputs >= 2, where an exact cast has no defined value at all — those inputs are collapsed onto FIXED_TERM instead.
- `fixedTermEndTimestamp` is truncated with an explicit `uint32(...)`; the same treatment is applied to `minCoverageWAD` / `minLiquidityWAD` (`uint64`) and `fixedTermEndTimestamp` in the storage writer. Truncation is exact for every value the real field can hold, so no representable state is unreachable and no unrepresentable state is fabricated; a spec that supplies an over-wide value gets the low bits rather than a revert. These casts are the only arithmetic in the diff and are pure glue for the change of representation — they have no counterpart in the original code and cannot revert.

Calling the capacity views through `this` preserves their semantics exactly (a staticcall into the real external function, including the ABI round-trip of the struct); it is required because they are declared `external`. The `view` / `pure` mutability of each wrapper matches the effective mutability of what it forwards to.

Where the harness could behave differently from the deployed contract — stated plainly rather than argued away: items 6 and 7 write storage with no authorization and without the `withSyncedAccounting` pre/post coverage-and-liquidity check that gates the real `setDustTolerance`, `setMinCoverage`, `setMinLiquidity` and `setLiquidationCoverageUtilization`. They can therefore place the accountant in configurations the deployed contract's setters would reject (e.g. `minCoverageWAD >= WAD`, `coverageLiquidationUtilizationWAD <= WAD`, or a `lastMarketState` / `fixedTermEndTimestamp` combination no sync would produce), and they can be invoked by any caller. This is an over-approximation of reachable state for the capacity views the author wants to prove things about, but it is an unsound model of the contract's access control and of `withSyncedAccounting`; any rule about who may change configuration, or about the setters' guard, must exclude these two methods.

Edit 2 — `5f05bf4294116c5a05e6c6915a77ae134e0dc3d8bd9fef6955a4586671067345`

Two files touched.

1. `src/accountant/RoycoDayAccountant.sol` — pure hoist of the three capacity views' bodies into new `internal view` helpers, each external function becoming a one-line forwarder:
   - `maxSTDeposit(SyncedAccountingState memory state) external view override(IRoycoDayAccountant)` now reads `return _maxSTDeposit(state);`; the previous body moved verbatim into new `function _maxSTDeposit(SyncedAccountingState memory state) internal view returns (NAV_UNIT)`.
   - Same for `maxJTWithdrawal` -> new `_maxJTWithdrawal(SyncedAccountingState memory) internal view returns (NAV_UNIT)`.
   - Same for `maxLPTWithdrawal` -> new `_maxLPTWithdrawal(SyncedAccountingState memory) internal view returns (NAV_UNIT)`. Each new helper carries a short NatSpec block; the existing `@inheritdoc` / `@dev` derivation comments stay on the external functions. No code inside the moved bodies was altered (including the `_getRoycoDayAccountantStorage().dustTolerance` reads, which stay inside the helpers).
2. `certora/harnesses/RoycoDayAccountantHarness.sol` — the four wrappers now call the new internal helpers instead of hopping through `this.`:
   - `maxSTDepositRaw` / `maxJTWithdrawalRaw` / `maxLPTWithdrawalRaw` call `_maxSTDeposit` / `_maxJTWithdrawal` / `_maxLPTWithdrawal`.
   - `capacityAtPreviewedStateRaw` likewise replaces its three `this.maxX(state)` calls with `_maxX(state)`; its `previewSyncTrancheAccounting` call is unchanged (already a direct `public view` call).
   - Four doc comments reworded from "the real `maxSTDeposit`" to "the body of the real `maxSTDeposit`" (etc.) so the harness documentation stays accurate. No `this.` remains anywhere in the harness.

I did NOT widen `maxSTDeposit` / `maxJTWithdrawal` / `maxLPTWithdrawal` from `external view` to `public view`, which was the request's first-choice shape: my charter freezes the visibility of existing external functions, so I took the equivalent alternative the request itself offered (internal helper + one-line external forwarder). The external ABI, selectors, mutability and storage layout are untouched either way.

Soundness argument

Charter category: **Refactor** (hoisting a named computation out of its surrounding function into a standalone internal function), plus the mechanical call-site update inside the existing harness.

Behavior preservation, per view: the entire body was moved verbatim into an `internal view` function whose single parameter is exactly the original function's single parameter, and the external function's body is a tail call returning that helper's value. Same storage reads (`_getRoycoDayAccountantStorage()` , `$.dustTolerance`), in the same order; same arithmetic operations with the same rounding modes; same early `return state.lptRawNAV` short-circuit in `_maxLPTWithdrawal` ; no writes existed and none were introduced; no `require` / `assert` / `revert` added or removed; no external calls in any of the three. The revert surface is therefore identical: the only reverts these bodies can produce come from the arithmetic inside them, which is unchanged operation-for-operation.

External surface: the three functions keep their names, parameter and return types, `external` visibility, `view` mutability and `override(IRoycoDayAccountant)` specifiers, so `IRoycoDayAccountant` is still satisfied and the deployed ABI/selectors are unchanged. The new helpers are `internal` , so they add nothing to the external surface — the harness's own `*Raw` wrappers (added previously) remain the only new external entry points, and they exist only on the Certora harness, not on the deployed accountant.

One deliberate deviation from the "everything it consumes arrives as a parameter" guidance: the helpers still read `dustTolerance` from the ERC-7201 storage struct rather than taking it as an argument. That is intentional — passing it in would let a caller (including the spec) evaluate the formula at a dust tolerance other than the stored one, i.e. would change what the harness wrappers actually exercise. Keeping the read inside makes the helper's result identical to the external view's result on every state and every input.

Call-site effect in the harness: replacing `this.maxX(state)` with `_maxX(state)` removes a `STATICCALL` to the harness's own address. Because these are `view` functions with no external calls and no state writes, the two forms compute the same value from the same storage; the only observable differences are gas and the (unobservable here) call depth. `msg.sender` / `msg.value` are not read by these bodies, so the switch from self-call context to internal-call context cannot change the result.

▶ Cumulative diff from on-disk source

Coverage: 92 CVL rules across 1 high-level properties (53–53 properties each). Coverage complete: **Yes**. 6 rule(s) not attributed to any property were dropped. 1 component(s) were cut short by the run budget (see the appendix).